

05 — Reinforcement Learning จากศูนย์: ทำไม Tron1 ถึงต้องใช้ PPO

นี่อธิบาย RL ตั้งแต่ไม่มีพื้นฐานเลย จนถึงเข้าใจว่าทุกบสรกัถใน `agents/limx_rsl_rl_ppo_cfg.py` มาจากไหน — เขียนให้อ่านรวดเดียวจบตามลำดับ โดยแต่ละส่วนใช้สิ่งที่ส่วนก่อนหน้าสร้างไว้ ไม่มีการอ้างถึงสิ่งที่ยังไม่ได้ อธิบาย คณิตศาสตร์เขียนเป็น LaTeX (เปิดดูใน VS Code preview หรือ Obsidian จะเห็นสมการเรนเดอร์)

ภาค 0 — ก่อนเริ่ม

0.1 วิธีอ่านโน้ตนี้ และอภิธานศัพท์

ทุกคำ ตัวย่อ สัญลักษณ์ คำคงที่ และเลขสมการ นิยามทีเดียวใน [00-glossary.md](#) — โน้ตนี้ *สอน* (ภาพ · ที่มา · การ derive · ความหมาย) ไม่ได้ นิยามซ้ำ · ถ้าสะดุดคำไหน เปิดอภิธานศัพท์ ก่อนอ่านต่อ · ตัวย่อทุกตัวมีชื่อเต็มในอภิธานศัพท์หัวข้อ 1 และภาคที่เรียบเรียงใหม่แล้วจะเขียนชื่อเต็ม ครั้งแรกที่ใช้ด้วย

ภาค	ตอบคำถามอะไร	หัวข้อ
1. โทษ	ปัญหาที่ต้องแก้คืออะไร เขียนให้เป็นคณิตศาสตร์ยัง	MDP, Markov, step/episode, สมการเป้าหมาย, value function, reward/return/value
2. คำตอบ	สิ่งที่เราจะไม่หา (policy) หน้าตาเป็นยังไง เชื่อมกับมอเตอร์ยัง	neural network, $\mu/a/\pi(a s)$, การสุ่ม, ก่อแปลงค่าไปมอเตอร์, act vs act_inference
3. ทางเดิน	ทำไมวิธี RL คลาสสิกใช้กับ Tron1 ไม่ได้	tabular, Monte Carlo vs TD (TD error), on-policy / importance sampling
4. PPO	PPO ประกอบขึ้นจากชิ้นส่วนอะไรบ้าง	policy gradient, actor-critic, encoder, GAE, loss ของ PPO (clip/KL/entropy/value), หน่วยนับของฟังก์ชันลอจิสติก
5. สรุป	เชื่อมทุกอย่างกลับเข้ากับ config จริง	ตาราง hyperparameter, สิ่งที่ยังไม่ได้พูดถึง

ถ้าอ่านครั้งแรก — อ่านเรียงตามลำดับ ภาค 1 → 5 แต่ละภาคจบในตัวเอง **ถ้ากลับมาหาอะไรเฉพาะจุด** — ใช้ตารางข้างบนหาภาคที่ใช่ก่อน

0.2 ตัวอย่างประจําโน้ต — ตาราง 2 env × 4 step

โน้ต 05, 06, 07 ใช้ตัวอย่างเดียวกัน: หุ่นจำลอง 2 ตัว เดิน 4 step action 1 มิติ (อ่านว่าเป็นข้อเข้าซ้ายเมื่อต้องการซื้อข้อต่อ · ตัวเลขไร้หน่วย — การคูณ 0.25 และ PD เป็นของ Tron1 จริงเท่านั้น) · เลิกพอเขียนทุกเลขลงกระดาษ · โน้ตนี้หยิบมาทีละช่อง โน้ต 07 ไล่ทั้ง iteration · เฉลยครบทุกค่าอยู่ที่ [00-glossary.md](#) หัวข้อ 8 · โน้ต 06 §3.5.5 ขยายช่อง e0 t0 เป็น 8 ข้อต่อ

ค่าคงที่	ตัวอย่าง	Tron1 จริง
จำนวน env B · จำนวน step T	$2 \cdot 4$	$2048 \cdot 24$
$\gamma \cdot \lambda$	$0.9 \cdot 0.8$	$0.99 \cdot 0.95$
σ (ความกว้างการสุ่ม)	0.5 คงที่	เริ่ม 1.0 แล้วเรียนรู้
ϵ (ขอบ clip)	0.2	0.2
epoch × minibatch	2×2	5×4

ช่องเรียกว่า **e0 t0 ... e1 t3** (env, step) · สิ่งที network คาย (μ, V) · ลูกเต๋าของ policy (ϵ — สุ่มใน `.sample()` ไม่ใช่โลกให้) · สิ่งทีโลกให้มา ($r, done$):

		t=0	t=1	t=2	t=3	V_{last}
env 0	μ	0.20	0.30	0.10	0.40	
	ϵ	+0.40	-0.20	+0.80	0.00	
	r	1.0	0.5	1.0	0.5	
	V	5.0	4.5	4.0	3.5	3.0
	done	0	0	0	0	
env 1	μ	-0.10	0.00	0.20	-0.30	
	ϵ	-0.60	+0.80	+0.20	+0.40	
	r	0.5	1.0	-2.0	0.0	
	V	3.0	2.0	1.0	4.0	3.5
	done	0	0	1	0	

env 1 ลืมหลัง step ที่ 2 (reward -2, done = 1) แล้วเริ่ม episode ใหม่ที่ t = 3 · **ช่องมาตรฐาน** ทีโน้ตหยิบ บ่อยทีสุดคือ **e0 t0: $\mu = 0.20$, $\epsilon = +0.40$, $r = 1.0$, $V = 5.0$** · ยังไม่ต้องเข้าใจว่าแต่ละแถว หมายถึงอะไร — ภาค 1 และ 2 จะแนะนำทีละแถวเมื่อถึงเวลา

ภาค 1 — โจทย์คืออะไร

ภาคนี้ยังไม่พูดถึงวิธีแก้เลย — เป้าหมายคือทำให้ปัญหา "หุ่นยนต์เดิน" กลายเป็น โจทย์คณิตศาสตร์ที่เขียนเป็นสมการได้ · ทุกคำที่ขึ้นตัวหนาครั้งแรกมีนิยามใน [00-glossary.md](#) และตัวเลขทุกตัวมาจากตาราง 2×4 ในหัวข้อ 0.2

1.1 ภาพรวม: RL กำลังแก้ปัญหาอะไร

ลองนึกภาพหุ่นยนต์ Tron1 ยืนอยู่บนพื้นราบ ทุก ๆ 0.02 วินาที (50 ครั้งต่อวินาที) มันต้องตัดสินใจว่าจะสั่งมอเตอร์ข้อต่อทั้ง 8 จุดเท่าไร ถ้าสั่งผิด มันล้ม ถ้าสั่งถูก มันเดินไปข้างหน้าได้ตามความเร็วที่สั่ง

ไม่มีใครเขียนสูตรคณิตศาสตร์ที่บอกว่า "ถ้าเอียงไปทางนี้ ให้สั่งมอเตอร์ข้อต่อเท่านี้" ได้ตรง ๆ เพราะระบบมันซับซ้อนเกินไป (แรงเสียดทาน, มวลกระจาย, momentum ของแต่ละขา ฯลฯ) สิ่งที่เราทำได้แทนคือ **ให้หุ่นยนต์ลองเดินในซิมูเลเตอร์ แล้วให้คะแนน (reward) กำกับ "ทุกครั้งที่ตัดสินใจ" ว่าการตัดสินใจครั้งนั้นดีหรือแย่** กำขำ แบบนี้เป็นล้าน ๆ ครั้ง จากนั้นให้มันปรับพฤติกรรมเพื่อเก็บคะแนนสะสมให้มากที่สุด — นี่คือแก่นของ **reinforcement learning (RL)** ทั้งหมด

ตัวเลขสองตัวในย่อหน้าแรกไม่ได้เดาขึ้น: 0.02 วินาทีมาจากฟิสิกส์ที่ก้าว 0.005 วินาที (sim.dt, limx_base_env_cfg.py:501) สี่ครั้งต่อการตัดสินใจหนึ่งครั้ง (decimation = 4, :497) และ 8 ข้อต่อมาจาก ActionsCfg ที่สั่ง "ทุกข้อต่อ" ของหุ่น (:116-121)

คะแนนที่ใช้ปรับ weight จริง ๆ ไม่ใช่ reward ดิบของแต่ละ step — หัวข้อ 1.7 จะแยกให้เห็นว่า "คะแนน" มีสี่ระดับ และตัวที่ใช้เรียนรู้คือระดับที่สี่

คำถามที่ RL ทุกวิธีพยายามตอบคือคำถามเดียวกัน: **"อยู่ในสถานการณ์นี้ ควรทำอะไร เพื่อให้ได้ผลตอบแทนสะสมในระยะยาวมากที่สุด"** ต่างกันแค่ *วิธีคำนวณคำตอบ* ซึ่ง ขึ้นอยู่กับว่าปัญหามีรูปร่างแบบไหน — และรูปร่างของปัญหานี้แหละที่ตัดวิธีเก่า ๆ ออกไปที่ละตัว (ภาค 3)

สรุปหัวข้อ — RL คือการให้คะแนนทุกครั้งที่ตัดสินใจ แล้วปรับพฤติกรรมให้คะแนนสะสมสูงสุด

1.2 MDP — แบบฟอร์มสำหรับอธิบายปัญหา

ก่อนจะพูดถึงวิธีแก้ปัญหา ต้องมีภาษากลางเรียกชิ้นส่วนของปัญหาก่อน ภาษากลางนั้น เรียกว่า **Markov Decision Process (MDP)**

MDP ไม่ใช่อัลกอริทึม ไม่ใช่วิธีแก้ปัญหา และไม่ใช่สิ่งที่ต้องคำนวณ — มันคือ **แบบฟอร์มสำหรับอธิบายปัญหา** เท่านั้นเอง

เปรียบเทียบกับ URDF

URDF ไม่ใช่หุ่นยนต์ ไม่ใช่ controller และไม่ใช่ simulator — มันคือ **รูปแบบมาตรฐานสำหรับอธิบายว่าหุ่นยนต์ตัวหนึ่งมีหน้าตาอย่างไร** (link มีอะไรบ้าง joint ต่อกันยังไง มวลเท่าไร) พอเขียน URDF เสร็จ เครื่องมือต่าง ๆ (Gazebo, MoveIt, rviz) ก็อ่านมันได้ เพราะทุกตัวรู้จักฟอร์แมตนี้

MDP คือ URDF ของปัญหาการตัดสินใจ — เป็นแบบฟอร์มที่บอกว่า "ถ้าจะอธิบาย ปัญหาตัดสินใจแบบเป็นลำดับ ต้องระบุช่องเหล่านี้ให้ครบ"

แบบฟอร์ม MDP มี 5 ช่อง — และแต่ละช่องเป็นของคนละชนิด

นี่คือตารางแรกของโน้ตที่มีคอลัมน์ **ชนิด** — คำถาม "มันเป็นของประเภทไหน" จะถูกถาม กับทุกสัญลักษณ์ตั้งแต่นี้ไป เพราะสัญลักษณ์ที่หน้าตาคล้ายกันมักเป็นคนละประเภท

ช่อง	ชนิด	คำถามที่ต้องตอบ	Tron1 ระบุว่าอะไร	ที่
$\mathcal{S}$ สถานะ (states)	เซต	สถานการณ์ที่เป็นไปได้มีอะไรบ้าง	เซนเซอร์ให้ตัวเลข 36 ค่าต่อ step (มุมข้อต่อ ความเร็ว ทิศแรงโน้มถ่วง ฯลฯ) — ส่วน input ที่ actor เห็นจริงคือ 42 ค่า ซึ่งประกอบขึ้นในหัวข้อ 2.2	PolicyCfg limx_base_env_cfg.py:130-146
$\mathcal{A}$ action	เซต	ทำอะไรได้บ้าง	offset ของมอเตอร์ข้อต่อ 8 ค่า — ต่อเนื่อง สั่งได้ทุกค่าจริง	ActionsCfg :116-121
$P(s' s, a)$ transition	การแจกแจง	ทำ action นี้แล้ว สถานการณ์จะเปลี่ยนไปอย่างไร	ฟิสิกส์ของ PhysX — เราไม่มีสูตร มีแต่ซิมูเลเตอร์ที่ก้าวทีละ 0.005 s	sim.dt :501
$R(s, a)$ ฟังก์ชันรางวัล	ฟังก์ชัน $\rightarrow$ scalar	ให้คะแนนยังไง	20 พอยต์รวมกัน ให้ทุก step — เขียนมั่วลงเล็บบเสมอ เพราะจะมี R_t อีกตัวในภาค 4	RewardsCfg :368

ช่อง	ชนิด	คำถามที่ต้องตอบ	Tron1 ระบุว่าอะไร	ที่
γ discount	ค่าคงที่	อนาคตสำคัญเท่าปัจจุบันไหม	0.99	gamma limx_rsl_rl_ppo_cfg.py:102

กรอกครบ 5 ช่องเมื่อไหร่ → **ปัญหาของเรา "เป็น" MDP** และกันที่รู้ว่าเป็น MDP เรา ก็รู้ว่าเครื่องมือทั้งตระกูล RL (Q-learning, PPO, ฯลฯ) ใช้กันมันได้ เพราะ เครื่องมือพวกนั้นถูกออกแบบมาให้กินฟอร์แมตนี้พอดี

นอกจาก 5 ช่องข้างบน ยังมีอีกคำที่ต้องรู้จัก — **ไม่ใช่ส่วนหนึ่งของแบบฟอร์ม แต่เป็นสิ่งที่เรากำลังจะไปหา:**

สัญลักษณ์	ชนิด	ความหมาย
π policy	ฟังก์ชัน	กฎที่แปลงสถานะเป็น action — คำตอบ ของปัญหา ไม่ใช่ส่วนหนึ่งของโจทย์ · หน้าตารางของมันคือฟังก์คา 2

ตาราง 2×4 ของเราคือ MDP ที่ถูกเล่นไปแล้ว 4 step — แถว r คือสิ่งที่ $R(s, a)$ ค้นออกมา แต่ละช่อง ส่วน a จะปรากฏในหัวข้อ 2.4 เมื่อรู้ว่ามันสุ่มมายังไง · สถานะ s ไม่ได้เขียนไว้ใน ตารางเพราะ toy ไม่มีเซนเซอร์ — ให้ถือว่า s คือ "อะไรก็ตามที่ทำให้ critic พุด 5.0 และ actor พุด 0.20" (สองตัวนี้จะรู้จักในหัวข้อ 1.6 และ 2.1)

ปัญหาที่ ไม่ใช่ MDP — ดูจากตัวเปรียบเทียบจะชัดขึ้น

ปัญหา	เป็น MDP ไหม	เหตุผล
จำแนกรูปแมว/หมา	$\times$	ไม่มีลำดับเวลา — ภาพรูปนี้เสร็จก็จบ ไม่มีผลต่อรูปถัดไป
เครื่องสลอต (multi-armed bandit)	$\times$ (เกือบ)	มีช่อง $\mathcal{A}$ กับ R แต่ ไม่มี $\mathcal{S}$ และ P — ดึงคันโยกแล้วเครื่องไม่เปลี่ยนสถานะ: action จึงไม่มีผลระยะยาวให้ต้องประเมิน · นี่คือเหตุผลที่วิธีของ bandit ใช้กับหุ่นเดินไม่ได้: การก้าวขาครั้งนี้เปลี่ยนท่าของก้าวถัดไปเสมอ
แก้สมการหาคำตอบเดียว	$\times$	ไม่มีการตัดสินใจซ้ำ ๆ ตามเวลา
หมากรุก	$\checkmark$	$\mathcal{S}$ = กระดาน, $\mathcal{A}$ = การเดินหมาก, P = กติกา, R = แพ้/ชนะ — ครบ แต่เป็นแบบ discrete (นับได้)
Tron1 เดิน	$\checkmark$	ครบทุกช่อง และเป็นแบบ continuous ทั้ง $\mathcal{S}$ และ $\mathcal{A}$

แล้วทำไมต้องตั้งชื่อให้มันด้วย

เพราะ **การรู้ว่ามีรูปร่างแบบไหน = รู้ว่าเครื่องมือไหนใช้ได้** เหมือนที่วิศวกร เห็นสมการแล้วบอกได้ว่า "นี่คือสมการกำลังสอง" — พอรู้ว่าเป็นกำลังสองก็รู้ทันที ว่าใช้สูตรอะไรได้ โดยไม่ต้องคิดใหม่ตั้งแต่ต้น

พอเรบอกว่า "**Tron1 เป็น MDP ที่มีสถานะและ action แบบต่อเนื่อง**" ประโยคเดียว นี้ตัดตัวเลือกออกไปครึ่งหนึ่งทันที (วิธีแบบตาราง (tabular) ใช้ไม่ได้ — หัวข้อ 3.1) และ ชี้อย่างตระกูลที่เหลือ (policy gradient → PPO) นี่คือประโยชน์ทั้งหมดของการมี คำว่า MDP อยู่

สรุปหัวข้อ — MDP คือ URDF ของปัญหาตัดสินใจ: แบบฟอร์ม 5 ช่อง ไม่ใช่วิธีแก้ · policy คือคำตอบ ที่อยู่นอกแบบฟอร์ม

1.3 Markov property — เชื่อนไขที่ซ่อนอยู่ในชื่อ

สังเกตว่าใน 5 ช่องข้างบน ไม่มีช่องไหนชื่อ "Markov" เลย แล้วทำไม MDP ถึงย่อมาจาก Markov? ก่อนตอบด้วยสูตร ดูสถานการณ์นี้ก่อน

หุ่นสองตัวที่เห็นเหมือนกันทุกตัวเลข แต่ควรทำคนละอย่าง

สมมติมีหุ่นยนต์สองตัว อยู่ในสถานการณ์ที่ observation ทั้ง 36 ค่า **เหมือนกันเป๊ะทุกตัวเลข:**

	หุ่นยนต์ A	หุ่นยนต์ B
joint_pos (มุมข้อต่อ 8 ค่า)	เหมือนกันเป๊ะ	เหมือนกันเป๊ะ
base_ang_vel (ความเร็วเชิงมุม)	เหมือนกันเป๊ะ	เหมือนกันเป๊ะ
proj_gravity (ทิศแรงโน้มถ่วง)	เหมือนกันเป๊ะ	เหมือนกันเป๊ะ
ความเร็วลำตัวจริง (วัดไม่ได้!)	1.0 m/s พุ่งไปข้างหน้า	0.0 m/s หยุดนิ่ง
action ที่ควรทำ	ต้องเตรียมรับแรง/ก้าวขาจับน้ำหนัก	ต้องออกแรงเริ่มก้าว

สองตัวนี้ **ควรทำ action ที่ต่างกัน แต่ observation ที่ policy เห็นเหมือนกัน ทุกประการ** — policy ไม่มีทางแยกแยะได้เลยว่าตอนนี้เจอเคสไหน · สิ่งที่ยหายไปคือ ข้อมูลจากอดีต (ก้าวที่แล้วมันพุ่งมาหรือยืนนิ่ง) ซึ่งสถานะปัจจุบันไม่ได้เก็บไว้

เขียนเงื่อนไขนี้เป็นสูตร

สัญลักษณ์ $P(\cdot | \cdot)$ อ่านว่า "ความน่าจะเป็นของสิ่งซ้ายมือ **เมื่อรู้**ว่า สิ่งขวามือ เป็นแบบนี้" — ชัดต้งคือคำว่า "เมื่อรู้" (อภิธานศัพท์หัวข้อ 7 มีรายละเอียดของชัดต้ง)

$$P(s_{t+1} | s_t, a_t) = P(s_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0)$$

(1)

ความหมาย: อนาคตขึ้นอยู่กับ **สถานะปัจจุบันเท่านั้น** — รู้ s_t กับ a_t แล้ว การรู้ประวัติทั้งหมดก่อนหน้านี้เพิ่มเติม **ไม่ได้ช่วยทำนายอะไรได้ดีขึ้น** เพราะ "สถานะปัจจุบันสรุปทุกอย่างที่จำเป็นต้องรู้เอาไว้หมดแล้ว" · หุ่น A กับ B ข้างบนละเมิด สมการนี้: ข้างซ้ายเท่ากันสำหรับทั้งคู่ (observation เดียวกัน) แต่ ข้างขวาต่างกัน (ประวัติต่างกัน)

Markov property ไม่ใช่ช่องให้กรอก แต่เป็น "เงื่อนไข" ที่แบบฟอร์ม MDP บังคับไว้เฉยๆ — เป็นข้อสมมติที่ต้องเป็นจริง ถ้าจะเอาเครื่องมือ RL มาใช้ได้อย่างถูกต้อง · เปรียบเทียบกับ URDF อีกครั้ง: URDF มีเงื่อนไขว่าโครงสร้างหุ่นยนต์ต้องเป็น tree ไม่มี loop — ถ้าหุ่นยนต์ของคุณ มี loop คุณก็ยังไม่ "เขียน URDF" ได้ แต่มันจะอธิบายหุ่นยนต์ได้ไม่ครบ และเครื่องมือที่อ่านมันจะ คำนวณผิดพลาด

ทำไม Tron1 ถึงไม่ Markov แท้

Remark. สาเหตุที่ความเร็วลำตัวหายไปจาก observation ไม่ใช่ความผิดพลาด ในการออกแบบ แต่เป็นข้อจำกัดทางกายภาพ: หุ่นยนต์จริงมีแค่ IMU (วัดความเร่ง และความเร็วเชิงมุม) กับ joint encoder (วัดมุมข้อต่อ) — ไม่มีเซนเซอร์ตัวไหน วัด "ตอนนี้ลำตัวเคลื่อนที่ไปข้างหน้ากี่เมตรต่อวินาที" ได้โดยตรง (ไม่มี GPS ในอาคาร ไม่มีกล้อง motion capture ติดตัวไปด้วย) · ดูได้จาก config: `base_lin_vel` อยู่ใน observation ของ **critic** (`limx_base_env_cfg.py:183`) แต่ไม่อยู่ใน `PolicyCfg (:130-146)` · ชื่อเรียกของปัญหาแบบนี้: **POMDP** (Partially Observable MDP) — MDP ที่มองเห็นสถานะได้แค่บางส่วน · ทางแก้มาตรฐานของ POMDP คือ **เอาประวัติย้อนหลังมาช่วย** ซึ่ง Tron1 ทำผ่าน `HistoryObsCfg (:154-176, ย้อน 10 step) + encoder` — อธิบายเต็ม ๆ ในหัวข้อ 4.4 หลังจากรู้จัก neural network แล้ว

แล้วถ้าไม่ Markov จะพังตรงไหน

ไม่ใช่แค่ "ไม่สวยงามในทางทฤษฎี" แต่ **เครื่องมือ RL แบบทุกชิ้นที่จะเจอต่อ จากนี้ ตั้งอยู่บนเงื่อนไขนี้ทั้งหมด** ถ้ามันไม่จริง เครื่องมือให้คำตอบผิด

เครื่องมือ	ตั้งอยู่บน Markov ตรงไหน	ถ้าไม่ Markov จะพังยังไง
Value function $V(s)$ — สมการ (3), หัวข้อ 1.6	นิยามบอกว่า "ค่าของสถานะ s " — แปลว่าค่ามันต้องขึ้นกับ s อย่างเดียว	หุ่น A กับ B มี s เดียวกันแต่ค่าจริงต่างกัน — $V(s)$ ตัวเดียวอธิบายทั้งคู่ไม่ได้
สมการ Bellman — สมการ (4)	เขียนว่า $V(s) = \mathbb{E}[r + \gamma V(s')]$ — ใช้ s' ตัวเดียวแทนอนาคตทั้งหมด	s' ไม่พอสรุปอนาคต ค่าที่คำนวณต่อ ๆ กันเพี้ยนสะสม
Bootstrapping / TD / GAE (generalized advantage estimation) — หัวข้อ 3.2, 4.4	อาศัยสมการ Bellman โดยตรง	ค่า $V(s_{t+1})$ ที่ยืมมาใช้แทนอนาคต ผิดตั้งแต่ต้น
Policy $\pi(a s)$	สมมติว่า action ที่ดีที่สุดถูกกำหนดโดย s เพียงอย่างเดียว	หุ่น A กับ B ต้องการ action ต่างกันจาก s เดียวกัน — policy ทำได้อย่างเดียวคือประนีประนอม

พูดรวบยอด: **Markov property คือสิ่งที่คำ RL ทั้งหมด** ถ้าเสานี้เอียง ทุกอย่างที่เราสร้างกับข้างบนก็เอียงตามหมด นี่คือเหตุผลที่ Tron1 ไม่ปล่อยปัญหานี้ทิ้งไว้เฉย ๆ แต่ลงทุนสร้าง encoder ขึ้นมาซ่อมมันโดยเฉพาะ (หัวข้อ 4.4)

Remark. จุดที่ควรเข้าใจให้ตรง: encoder **ไม่ได้ทำให้ปัญหากลับมาเป็น Markov แท้ ๆ 100%** — ความเร็วลำตัวที่ประมาณจากประวัติ 10 step ยังเป็นค่าประมาณ ที่มี error อยู่ดี สิ่งที่เราได้คือทำให้ **"ใกล้เคียง" Markov มากพอ** ที่เครื่องมือทั้งหมดข้างบนจะทำงานได้ดีในทางปฏิบัติ — RL บนหุ่นจริง ทั้งหมดอยู่ในโหมดนี้ ไม่ใช่โหมด Markov สมบูรณ์แบบตามทฤษฎี · ตาราง 2×4 ของเรา ถือว่า Markov เพราะไม่มีเซนเซอร์ให้ขาด

สรุปหัวข้อ — Markov = "สถานะปัจจุบันสรุปอดีตไว้หมดแล้ว" · Tron1 ละเมิดเพราะวัดความเร็ว ลำตัวไม่ได้ จึงต้องซ่อมด้วยประวัติ + encoder

1.4 หน่วยเวลา: step กับ episode

ก่อนจะเขียนสมการเป้าหมายได้ ต้องมีหน่วยนับเวลาที่ชัดเจนก่อน สองคำนี้เป็นคนละ ระดับกัน คล้ายกับ "เฟรมหนึ่งเฟรม" เทียบกับ "หนังทั้งเรื่อง":

คำ	ชนิด	คือหน่วยอะไร	ใน Tron1	ที่
step (policy step)	หน่วยนับ	การตัดสินใจ ครั้งเดียว — หนึ่งรอบของ: สังเกตสถานะ → policy คำนวณ action → สั่ง action → ฟลิกสับ → ได้ reward	หนึ่ง step กินเวลา 0.02 วินาที (50 Hz) — ภายในนั้นฟลิกสับจริงยับ 4 ครั้งย่อยที่ 200 Hz แต่ policy ตัดสินใจแค่ครั้งเดียว	<code>decimation = 4</code> <code>limx_base_env_cfg.py:497</code> , <code>sim.dt = 0.005 : 501</code> · หนึ่ง step ในโค้ดคือ <code>alg.act</code> แล้ว <code>env.step</code> <code>on_policy_runner.py:175-177</code>

คำ	ชนิด	คือหน่วยอะไร	ใน Tron1	ที่
episode	หน่วยนับ	การเล่นต่อเนื่องหลาย step ตั้งแต่หุ่นยนต์เริ่มขึ้น จนจบ (ครบเวลา หรือล้ม)	สูงสุด 1000 step ติดกัน (1000 × 0.02 s = 20 วินาที)	episode_length_s = 20.0 : 498

พูดง่าย ๆ **episode ประกอบด้วย step หลายอันเรียงต่อกัน** — step คือหน่วย เวลาที่เล็กที่สุดที่ policy ได้ตัดสินใจ ส่วน episode คือ "หนึ่งรอบการเล่น ทั้งหมด" ที่นับ step สะสมไปเรื่อย ๆ จนกว่าจะจบด้วยเหตุผลใดเหตุผลหนึ่ง:

- time_out** — เดินครบ 1000 step โดยไม่ล้ม → episode จบแบบดี (เรียกว่า *truncation*: อนาคตยังมี แต่เราหยุด) · TerminationsCfg · limx_base_env_cfg.py:450
- base_contact** — ลำตัวแตะพื้น (ล้ม) ก่อนครบเวลา → episode จบแบบล้มเหลว (เรียกว่า *termination*: อนาคตถูกตัดจริง) · limx_base_env_cfg.py:451-454

ทั้งสองกรณีทำให้ค่า **done** ของ step สุดท้ายเป็น 1 (ที่เหลือเป็น 0) — ธงตัวนี้คือสิ่งเดียวที่ episode กิ่งไว้ให้คณิตศาสตร์ในภาค 3-4 ใช้ · แต่อัลกอริทึม **ปฏิบัติต่อสองสาเหตุนี้ต่างกัน** ซึ่งจะเห็นตอนถึงหัวข้อ 4.5 (สรุปสั้น: กรณีหมดเวลา ppo.py:163-168 บวก γV กลับเข้าไปใน reward ก่อนเก็บ — อภิธานศัพท์หัวข้อ 2 แถว termination/truncation)

ดูจากตาราง 2 × 4

คอลัมน์ env 1 ของตัวอย่างประจําไม้ติมี episode อยู่สองอัน:

env 1:	t=0	t=1	t=2	t=3
	r=0.5	r=1.0	r=-2.0 done=1	r=0.0 done=0
	└── episode ที่ 1 (3 step, จบเพราะล้ม) ─┘			└── episode ที่ 2 เริ่ม (ต่อไปตารางหน้า)

ที่ t = 2 หุ่นล้ม (reward -2, done = 1) แล้ว reset — ช่อง t = 3 จึงเป็นการขึ้นใหม่ในท่าเริ่มต้น (V กระโดดจาก 1.0 เป็น 4.0 เพราะยืนตรงอีกครั้ง — จะเข้าใจตัวเลข V ในหัวข้อ 1.6) · ส่วน env 0 ยังไม่ล้มเลยใน 4 step episode ของมันยาวข้ามไปตารางถัดไป

Remark. จุดที่มักงงคือ แต่ละ environment (จากทั้ง 2048 ตัวที่รันขนาน กัน) อยู่ที่ step คนละจุดของ episode คนละอันได้พร้อมกัน — บางตัวเพิ่งเริ่มที่ step 5 บางตัวใกล้จบที่ step 998 บางตัวเพิ่งล้มแล้วรีเซ็ตกลับไป step 0 ทั้งหมดนี้เกิดพร้อมกันในเวลาจริงเดียวกัน — ผลที่ตามมาสำคัญมากและอธิบายใน หัวข้อ 4.7 (หน่วยของฟังก์ชันอัลกอริทึม) · ในตาราง 2 × 4 ก็เห็นแล้ว: env 0 กับ env 1 อยู่ใน episode คนละอันตลอดเวลา

ยังมีหน่วยที่สามชื่อ iteration ซึ่งเป็นหน่วยของ*การเรียนรู้* ไม่ใช่หน่วย ของ*ปัญหา* — เก็บไว้พูดในหัวข้อ 4.7 เพราะต้องรู้จักอัลกอริทึมก่อน

สรุปหัวข้อ — step คือหนึ่งเฟรม episode คือหนึ่งเรื่อง · done = 1 คือเฟรมสุดท้าย และเป็นสิ่งเดียว ที่ episode ส่งต่อให้สมการ

1.5 สมการเป้าหมาย — เขียนโจทย์ให้เป็นคณิตศาสตร์

พูดเป็นภาษาธรรมชาติก่อน: **เราอยากได้ policy ที่พอเอาไปรันหุ่นยนต์เต็ม episode (1000 step) แล้ว บวกคะแนน reward ทุก step เข้าด้วยกัน จะได้ผลรวมสูงที่สุด โดยเฉลี่ย** (พูดว่า "โดยเฉลี่ย" เพราะทั้งการสุ่ม action และการสุ่มเริ่มต้น episode ทำให้แต่ละครั้งได้ผลรวมไม่เท่ากันเป๊ะ) แค่นั้นเอง — สมการด้านล่างเป็น แค่การเขียนประโยคนี้อให้เป็นสัญลักษณ์:

ส่วนของสมการ	ชนิด	อ่านว่า	ความหมาย
$\sum_{t=0}^T \gamma^t r_t$	scalar	ผลรวมของ reward ทุก step ตั้งแต่ $t = 0$ ถึง $t = T$ (จบ episode) ถ่วงด้วย γ^t	คะแนนรวมของหนึ่ง episode
$\mathbb{E}[\cdot]$	ตัวดำเนินการ	"ค่าคาดหวัง" (expectation) — ค่าเฉลี่ยถ้าเล่นซ้ำหลาย ๆ รอบ	ในโค้ดคือ .mean() บนตัวอย่างจำนวนมาก
$\arg \max_{\pi}$	ตัวดำเนินการ	"หา π ที่ทำให้สิ่งข้างหลังมีค่ามากที่สุด"	นี่คือคำว่า "เรซู"
π^*	ฟังก์ชัน	policy ที่ดีที่สุดที่หาได้ (ถวากำกับตามธรรมเนียม)	สิ่งที่ 15,000 iteration พยายามเข้าใกล้

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^T \gamma^t r_t \right]$$
(2)

ความหมายรวม: "ในบรรดา policy ที่เป็นไปได้ทั้งหมด ให้เลือกตัวที่พอเอาไปรัน หุ่นยนต์แล้ว ผลรวมคะแนนเฉลี่ยทั้ง episode สูงที่สุด" — ทุกบรรทัดของ train.py ที่รันอยู่ 15,000 iteration (max_iterations, limx_rsl_rl_ppo_cfg.py:82; ลูปที่ on_policy_runner.py:170) คือการพยายามไต่เข้าใกล้คำตอบของสมการนี้ทีละก้าวเล็ก ๆ

γ (gamma, discount factor) ใน γ^t เป็นตัวเลข 0 ถึง 1 ที่บอกว่า reward ในอนาคตมีค่าน้อยกว่า reward ตอนนี้อย่างไร — เหมือนดอกเบี้ย กลับด้าน · Tron1 ใช้ gamma = 0.99 (:102) แปลว่า reward ที่จะได้อีก 100 step ข้างหน้า ยังมีน้ำหนักราว $0.99^{100} \approx 0.37$ เท่าของ reward วันนี้ — โหมด

ถูกบังคับให้มองการณ์ไกลพอสมควร แต่ไม่โลจนไม่สนใจปัจจุบัน · ตาราง 2×4 ใช้ $\gamma = 0.9$ ซึ่งให้ความรู้สึกเดียวกันในสเกลที่เล็กกว่า: $0.9^{10} \approx 0.35$ — มอง 10 step ข้างหน้าด้วยน้ำหนักพอ ๆ กับที่ Tron1 มอง 100 step

สมการ (2) นี้คือใจยกทั้งหมด ทุกอัลกอริทึมที่จะพูดถึงต่อไปนี้ — Q-learning, Monte Carlo, PPO — ล้วนพยายามแก้สมการเดียวกันนี้ แต่ วิธีประมาณคำตอบ ต่างกัน

สรุปหัวข้อ — โจทย์คือหา π ที่ทำให้ผลรวม reward ถ่วง γ สูงสุดโดยเฉลี่ย · "tru" = **arg max** ที่ทำที่ละก้าว

1.6 Value function, สมการ Bellman — และ Q-function

ค่าของสถานะ: V

ก่อนจะหาว่า action ไหนดี มีสิ่งหนึ่งที่มีประโยชน์มากกว่านั้นอีก: รู้ว่า *สถานะ* ปัจจุบันดีแค่ไหนโดยรวม (ไม่ต้องรู้ action ก่อน) เช่น หุ่นยนต์ที่กำลังยืนตรงนี่ ๆ "ดีกว่า" หุ่นยนต์ที่เอียง 45 องศาใกล้ล้ม แม้จะยังไม่ได้ตัดสินใจอะไรเลยก็ตาม

ค่านี้เรียกว่า **value function** — ชนิดของมันคือ **ฟังก์ชัน** ที่รับสถานะหนึ่งตัว คืนตัวเลข หนึ่งตัว:

$$V^\pi : \mathcal{S} \rightarrow \mathbb{R}$$

ด้วย π อ่านว่า "ภายใต้ policy นี้" — ค่าของสถานะขึ้นกับว่าใครเป็นคนเดินต่อจากตรงนั้น · นิยามว่าเป็นผลรวม reward ที่คาดว่าจะได้ *ถ้าเริ่มจากสถานะ s แล้วเดินตาม policy π ไปเรื่อย ๆ*.

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s \right] \tag{3}$$

ในตาราง 2×4 แถว V คือ **ค่าทำนาย**ของฟังก์ชันนี้สำหรับสถานะที่ env อยู่ในแต่ละ step — env 0 ทำนายว่า 5.0, 4.5, 4.0, 3.5 · ใครเป็นคนทำนาย จะรู้ในหัวข้อ 4.2 (ตัวที่ชื่อ critic — self.critic(...) actor_critic.py:170-172)

สมการ Bellman — เขียน V ให้เวียนเกิด

Value function เขียนใหม่แบบ **เวียนเกิด (recursive)** ได้ — เพราะ "ผลรวม reward จากตอนนี้ไปตลอด" เท่ากับ "reward ก้าวเดียวข้างหน้า บวกกับ (ผลรวมที่เหลือ คูณ discount)":

ขั้นตอน	เหตุผล
$V^\pi(s) = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s \right]$	นิยาม (3)
$= \mathbb{E} \left[r_t + \gamma \sum_{k=0}^{\infty} \gamma^k r_{t+1+k} \mid s_t = s \right]$	แยก reward แรกออกจากผลรวมที่เหลือ แล้วดึง γ หนึ่งตัวออกมา
$= \mathbb{E} \left[r_t + \gamma V^\pi(s_{t+1}) \mid s_t = s \right]$	ผลรวมที่เหลือคือนิยาม (3) ของ V^π ที่สถานะถัดไปพอดี

$$V^\pi(s) = \mathbb{E} \left[r_t + \gamma V^\pi(s_{t+1}) \mid s_t = s \right] \tag{4}$$

ความหมาย: ค่าของสถานะปัจจุบัน = รางวัลทันทีที่ได้ + ค่าของสถานะถัดไป (ถ่วงน้ำหนักด้วย discount) พุดง่าย ๆ คือ "ตอนนี้แค่ไหน ก็ต้องรวมด้วยว่า มันพา ไปสู่อะไรที่ดีแค่ไหนด้วย"

ตรวจกับตาราง: ที่ e0 t0 ข้างขวาของ (4) คือ $1.0 + 0.9 \times 4.5 = 5.05$ ส่วนข้างซ้าย (ที่ critic ทำนาย) คือ **5.0** — ต่างกัน **0.05** · ถ้า critic ทำนายได้สมบูรณ์แบบ สองข้างจะเท่ากัน โดยเฉลี่ย (ระวังสับสนได้หลายค่า ตารางให้มาแค่ค่าเดียว) **ส่วนต่างนี้จึงเป็นตัววัดว่า critic เดาพลาดไปเท่าไร** — เก็บเลข 0.05 นี้ไว้ หัวข้อ 3.2 จะดึงชื่อให้มัน

สมการนี้เป็นรากฐานของ RL เกือบทุกสาขา และเป็นจุดที่ทำให้เกิดแนวคิดสำคัญเรื่อง **bootstrapping** — การประมาณค่าปัจจุบัน โดยอ้างอิงจากค่าประมาณของอนาคต แทนที่จะ รอผลจริงทั้งหมด · ในตาราง ค่า V_{last} (3.0 และ 3.5) คือ bootstrap ที่ขอบตาราง: แถว $t = 4$ ไม่มีให้ดู จึงให้ critic ทำนายแทน

ค่าของสถานะเมื่อเลือกก้าวแรกไว้: Q

ตอนนี้ลองนึกถึงหุ่นสองตัวที่**ทำเดียวกันเป๊ะ** เอียงไปข้างหน้าเท่ากัน — แต่ตัวหนึ่งกำลังจะ ดันเข้าซ้ายรับน้ำหนัก (a_1) อีกตัวกำลังจะก้าวขวา (a_2) · สถานะเดียวกัน ก้าวแรกต่างกัน อนาคตต่างกัน · $V(s)$ ตอบเรื่องนี้ไม่ได้ เพราะมันเฉลี่ยทุกก้าวแรกที่ π อาจเลือก

ฟังก์ชันที่ตอบได้เรียกว่า **action-value function Q** — รับสถานะ**และ** action คืนตัวเลขหนึ่งตัว:

$$Q^\pi : \mathcal{S} \times \mathcal{A} \longrightarrow \mathbb{R}$$

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s, a_t = a \right] \quad (5)$$

ต่างจาก (3) แค่มี $a_t = a$ เพิ่มหลังขีดตั้ง — "เมื่อรู้ว่าก้าวแรกคือ a " · ทำแบบเดียวกับ Bellman จะได้ความสัมพันธ์ก้าวเดียว:

ขั้นตอน	เหตุผล
$Q^\pi(s, a) = \mathbb{E} [r_t + \gamma \sum_{k=0}^{\infty} \gamma^k r_{t+1+k} \mid s_t = s, a_t = a]$	แยก reward แยกออกเหมือนเดิม
$= \mathbb{E} [r_t + \gamma V^\pi(s_{t+1}) \mid s_t = s, a_t = a]$	หลังก้าวแรก policy กลับมาเลือกเองตามปกติ พวกรวมที่เหลือจึงเป็น V^π ไม่ใช่ Q^π

$$Q^\pi(s, a) = \mathbb{E} [r_t + \gamma V^\pi(s_{t+1}) \mid s_t = s, a_t = a] \quad (6)$$

ในตาราง: ที่ e0 t0 ก้าวแรกที่ทำให้ $r = 1.0$ และพาไป $V(s_1) = 4.5$ ดังนั้นค่าประมาณ ก้าวเดียวของ $Q(s_0, a_0)$ คือ $1.0 + 0.9 \times 4.5 = 5.05$ — เลขเดิมจากการตรวจ Bellman ข้างบน ไม่บังเอิญ: (6) กับ (4) ต่างกันแค่ (4) เลี้ยวทุกก้าวแรก ส่วน (6) ล็อกก้าวแรกไว้

Remark. Tron1 ไม่เคยคำนวณ Q เลย — ค้นทั้ง rs1_x1/ ไม่มี Q-function ที่ไหน · เหตุผล ที่ต้องนิยามไว้คือหวัชข้อถัดไปต้องใช้มันนิยาม advantage แล้วภาค 4 จะประมาณ advantage ด้วยวิธีที่ไม่ต้องมี Q จริง ๆ · ส่วน V มีตัวตนจริงในโค้ด: self.critic (actor_critic.py:92-111)

สรุปหวัชข้อ — V ถูกว่า "ทำได้ดีแค่ไหน" · Q ถูกว่า "ทำดี ถ้าเริ่มด้วยก้าวนี้ ดีแค่ไหน" · Bellman บอกว่าค่าวันนี้ = รางวัลก้าวเดียว + $\gamma \times$ ค่าพรุ่งนี้

1.7 Reward, Return, Value, Advantage — สี่ระดับของ "คะแนน"

สี่คำนี้ฟังดูคล้ายกันแต่อยู่คนละระดับกันชัดเจน — และชนิดต่างกันด้วย:

คำ	ชนิด	คือคะแนน ณ ตอนไหน	รู้ค่าได้ ตอนไหน	ในตาราง 2 × 4	ที่
reward r_t	scalar ต่อ step	คะแนนหนึ่ง step เดียว ที่ได้ทันทีหลังทำ action	รู้ทันที	แถว $r \cdot e0 t0 = 1.0$	RewardsCfg limx_base_env_cfg.py:368
return G_t	scalar ต่อ step (คำนวณได้เมื่อ episode จบ)	ผลรวม reward ตั้งแต่ step นี้ ไปจนจบ episode ถ่วงด้วย γ — ข้อเท็จจริง	รู้ต่อเมื่อ episode จบ	env 1 episode แรก: $G_0 = 0.5 + 0.9(1.0) + 0.81(-2.0) = -0.22$	ไม่มีใน repo · ที่ใกล้ที่สุดคือ Mean reward = ผลรวมต่อ episode แบบไม่ถ่วง γ (on_policy_runner.py:200-205, พิมพ์ :326) จึงไม่ใช่ G_t
value $V(s)$	ฟังก์ชัน → scalar	ค่าคาดหวังของ return ถ้าเริ่มจากสถานะนี้ — เป็นค่าทำนาย	รู้ทันที (แต่อาจผิด)	แถว $V \cdot$ env 1 ทำนาย $V(s_0) = 3.0$	self.critic actor_critic.py:170-172
advantage $A(s, a)$	ฟังก์ชัน → scalar	ก้าวนี้ดีกว่าที่คาดจากสถานะนี้เท่าไร	ต้องมี Q (หรือตัวประมาณ)	e0 t0: $\approx 5.05 - 5.0 = +0.05$ (ประมาณหยาบ)	ประมาณจริงในหวัชข้อ 4.5 · ใช้ที่ ppo.py:256

ความสัมพันธ์ระหว่างสามคำแรกเขียนเป็นสมการได้สั้น ๆ:

$$V^\pi(s) = \mathbb{E} [G_t \mid s_t = s] \quad (7)$$

ความหมาย: value คือค่าเฉลี่ยที่คาดหวังของ return ถ้าเล่นจากสถานะ นั้นซ้ำ ๆ หลายรอบ ส่วน return คือตัวเลขจริงที่เกิดขึ้นในรอบใดรอบหนึ่ง โดยเฉพาะ (ซึ่งจะแกว่งรอบ ๆ ค่า value เพราะมีความสุ่มของ action และของ environment เข้ามา) พูดเป็นภาพ: reward คือ "คะแนนเต็มเดียวที่เพิ่งได้", return คือ "ผลรวมคะแนนทั้งเกมที่จบไปแล้วจริง ๆ", value คือ "การเดาล่วงหน้าว่า เกมจะจบด้วยคะแนนรวมเท่าไร ตอนที่ยังเล่นอยู่"

ดูจากตาราง: episode แรกของ env 1 จบแล้วจริง ๆ (3 step) จึงคำนวณ return ได้ตรง ๆ: $G_0 = -0.22$ แต่ตอน $t = 0$ critic ทำนายไว้ว่า $V(s_0) = 3.0$ — ทำนายเกินจริงไป 3.22 เพราะมันไม่รู้ว่าจะอีกสองก้าวจะล้ม · return คือข้อเท็จจริง value คือคำทำนาย และส่วนต่างระหว่าง สองอย่างนี้คือสิ่งที่ critic ต้องเรียนรู้ (หัวข้อ 4.3)

Remark. สมการ Bellman (4) จริง ๆ แล้วคือการเขียนนิยาม (7) ใหม่แบบ เวียนเกิด — "ค่าคาดหวังของ return จากสถานะนี้" เท่ากับ "reward ก้าวแรก บวกค่าคาดหวังของ return จากสถานะถัดไป" นั่นเอง

ระดับที่สี่ — advantage

คำถามที่ RL ต้องตอบตอนเรียนรู้ไม่ใช่ "สถานะนี้ดีไหม" (นั่นคือ V) แต่คือ "ก้าวที่เพิ่งทำ ดีกว่าหรือแย่กว่าก้าวทั่วไปจากสถานะนี้" — เพราะสถานะดีอยู่แล้วก็ได้ reward ดีโดยไม่เกี่ยว กับก้าวนั้นเลย · ตัวที่วัดสิ่งนี้คือ **advantage**: ค่าของสถานะเมื่อเลือกก้าวนี้ไว้ ลบด้วยค่าของ สถานะโดยเฉลี่ย

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

(8)

ค่าของ A	แปลว่า
> 0	ก้าวนี้ดีกว่าที่ policy ปัจจุบันจะทำโดยเฉลี่ยจากสถานะนี้ — ทำอีก
$= 0$	ก้าวธรรมดา ไม่ดีไม่แย่ไปกว่าปกติ
< 0	ก้าวนี้แย่กว่าปกติ — เลิกทำ

จุดที่ควรสังเกต — advantage เป็นค่าสัมพัทธ์ เกี่ยวกับ V ของสถานะเดียวกัน ไม่ใช่เกี่ยวกับ ศูนย์ · รุ่นที่ยื่นดีมาก (V สูง) แล้วก้าวได้ธรรมดา ๆ จะได้ $A \approx 0$ ไม่ใช่ A บวก แม้ reward จะสูง · นี่คือเหตุผลที่การเรียนรู้ไม่ได้ดู reward ดิบ (หมายเหตุในหัวข้อ 1.1)

ในตาราง: ที่ $t=0$ ค่าประมาณก้าวเดียวของ Q คือ 5.05 (หัวข้อ 1.6) และ $V = 5.0$ ดังนั้น $A \approx +0.05$ — ก้าวนี้ดีกว่าที่คาดนิดหน่อย · นี่เป็นค่าประมาณที่หยาบมาก (ดูขนาดแค่ก้าว เดียว) ตัวประมาณจริงที่ PPO ใช้อยู่ในหัวข้อ 4.5 และจะให้เลขคนละตัว — แต่ความหมายเดียวกัน

จบภาค 1 แล้วเรารู้อะไรบ้าง: โทษภัยคือสมการ (2) — หา policy ที่ทำให้ผลรวม reward สูงสุด; เรามีภาษาเรียกชิ้นส่วน (MDP 5 ช่อง); รู้ว่าเงื่อนไข Markov สำคัญแค่ไหนและ Tron1 ละเมิดมันตรงไหน; มีเครื่องมือประเมินสถานะ (V + Bellman) และประเมินก้าว (Q, A) สิ่งที่ยังไม่รู้เลยคือ "policy" ที่ตามหาอยู่นั้น หน้าตา เป็นยังไง — ภาค 2 ตอบเรื่องนี้

สรุปหัวข้อ — reward = แต้มที่เพิ่งได้ · return = แต้มรวมทั้งเกมที่จบแล้ว · value = การเดาแต้มรวม · advantage = ก้าวนี้ดีกว่าที่เดาไว้เท่าไร — ตัวสุดท้ายคือตัวที่ใช้เรียนรู้

ภาค 2 — คำตอบหน้าตาเป็นยังงี้: Policy

ภาค 1 บอกว่าเราตามหา π อยู่ แต่ยังไม่เคยบอกว่ามันคืออะไร ภาคนี้แหละให้เห็น ทุกชั้น ตั้งแต่ชนิดของวัตถุที่ชื่อ π ตัวเลขในโครงข่าย ไปจนถึงแรงบิดที่มอเตอร์ · ตัวเลขทุกตัวยังมาจากช่อง e0 t0 ของตาราง 2×4 (หัวข้อ 0.2): $\mu = 0.20, \sigma = 0.5, \epsilon = +0.40$

2.1 π_θ คืออะไรกันแน่ — ชนิดก่อน แล้วค่อยรูปร่าง

π (อ่านว่า "pi" ไม่เกี่ยวกับเลข 3.14159) ถูกพูดถึงลอย ๆ มาทั้งภาค 1 ว่าเป็น "กฎที่แปลงสถานะเป็น action" — ประโยคนั้นถูกครึ่งเดียว และครึ่งที่ผิดคือต้นเหตุของความสับสนเกือบทั้งหมดในโน้ตนี้ จึงขอเริ่มจากคำถามที่ต้องถามกับทุกสัญลักษณ์: มันเป็นของชนิดไหน

ประกาศชนิดก่อน

เวลาเขียนฟังก์ชันในคณิตศาสตร์ เราประกาศว่า "รับอะไร คืนอะไร" เช่น $f: \mathbb{R} \rightarrow \mathbb{R}$ อ่านว่ารับจำนวนจริง คืนจำนวนจริง · policy ของ Tron1 ประกาศแบบนี้:

$$\pi_\theta: \mathcal{S} \rightarrow \Delta(\mathcal{A})$$

ของ	ประกาศชนิด	อ่านว่า
ยกกำลังสอง	$f: \mathbb{R} \rightarrow \mathbb{R}$	รับจำนวนจริง คืนจำนวนจริง
Actor NN	$\mu_\theta: \mathbb{R}^{42} \rightarrow \mathbb{R}^8$	รับเวกเตอร์ 42 ช่อง คืนเวกเตอร์ 8 ช่อง
Policy	$\pi_\theta: \mathcal{S} \rightarrow \Delta(\mathcal{A})$	รับ state คืน ระดังหนึ่งใบ — ไม่ได้คืน action

$\Delta(\mathcal{A})$ (อ่านว่า "เดลตาของเอ") คือ **เซตของการแจกแจงทั้งหมดบน $\mathcal{A}$** — นึกภาพว่าเป็น "ระดังทุกใบที่วางบนแกน action ได้" policy รับ s แล้วหยิบระดังมาให้หนึ่งใบ ส่วนคำว่า action ที่จะสั่งจริงต้อง **สุ่ม** ออกมาจากระดัง ใบนั้นอีกที · นี่คือการแตกต่างระหว่าง "กฎที่ให้ action" กับ "กฎที่ให้การแจกแจงของ action" และ Tron1 ตอนแรกเป็นอย่างหลัง

สัญลักษณ์ที่หน้าตาคล้ายกันแต่เป็นคนละชนิด

ตารางนี้คือกุญแจของทั้งภาค — ทุกบรรทัดของโค้ดที่จะเห็นต่อจากนี้ตรงกับแถวใดแถวหนึ่งพอดี (โน้ต 06 หัวข้อ 3.5.0 ขยายตารางนี้เป็นเวอร์ชันเต็ม)

เขียนว่า	ชนิด	หมายถึง	ในโค้ด
π	ฟังก์ชัน	policy ทั่วไป ไม่ระบุเจ้าของใคร	—
π_θ	ฟังก์ชัน $\mathcal{S} \rightarrow \Delta(\mathcal{A})$	policy ตัวนี้ ที่มีพารามิเตอร์ θ	①+②+③ ในตารางถัดไป
θ	เวกเตอร์ยาว	ตัวเลขทุกตัวที่การเรียนรู้ไปเปลี่ยน = weight ของ NN บวก ลูกบิต ความกว้าง 8 ตัว	<code>actor_critic.parameters()</code> ส่วนของ actor + <code>logstd</code>
$\pi_\theta(\cdot s)$	การแจกแจง	ระดัง 1 ใบ ที่ได้จากการใส่ s — จุดกลมคือช่องที่ยังไม่เต็ม	<code>self.distribution(actor_critic.py:157)</code>
$a \sim \pi_\theta(\cdot s)$	โค้ด → เวกเตอร์ (8,)	$\tilde{a}$ อ่านว่า "ถูกสุ่มออกมาจาก" ระดังใบนั้น — เรียกซ้ำได้ค่าใหม่	<code>.sample() (:161)</code>
$\pi_\theta(a s)$	scalar	ความหนาแน่นของระดัง ณ จุด a — วิธีคำนวณอยู่หัวข้อ 2.3	<code>.log_prob(a).sum(-1) (:164)</code>
$\mu_\theta(s)$	เวกเตอร์ (8,)	output ของ NN = ค่าที่ actor คิดว่าดีที่สุด · นิพจน์ คำนวณใหม่ทุกครั้ง	<code>self.actor(obs) (:156)</code>
$\sigma \cdot \ell$	เวกเตอร์ (8,)	ความกว้างของระดัง · $\ell = \log \sigma$ คือ ตัวแปร ที่นอนอยู่ข้าง ๆ NN ไม่ใช่ข้างใน	<code>torch.exp(self.logstd) (:157) · self.logstd (:118)</code>

สองแถวที่สับสนบ่อยที่สุด: π_θ (ไม่มีวงเล็บ) เป็น **ฟังก์ชัน** ส่วน $\pi_\theta(a | s)$ (มีวงเล็บ ครบทั้ง a และ s) เป็น **ตัวเลขตัวเดียว** — ตัวหลัง รับ argument สองตัว ไม่ได้ "แปลง state เป็น action" แต่ "ตอบว่าคู่ (s, a) นี้ น่าจะเกิดแค่ไหน"

policy ประกอบจากสามชิ้น — และ NN เป็นแค่ชิ้นเดียว

ชิ้น	คืออะไร	ให้อะไรกับระดัง	ขนาด	เรียนใหม่	โค้ด
① NN (actor)	โครงข่าย $42 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 8$	ตำแหน่ง $\mu_\theta(s)$	187,272 weight	✓	<code>self.actor actor_critic.py:69-90</code>
② logstd	ตัวเลข 8 ตัว หนึ่งตัวต่อข้อต่อ	ความกว้าง $\sigma = e^\ell$	8 ตัว	✓	<code>self.logstd :118</code>

ชั้น	คืออะไร	ให้อะไรกับระบบ	ขนาด	เรียนใหม่	โค้ด
③ สูตร $\mathcal{N}$	บรรทัดโค้ดที่ประกอบ μ กับ σ เป็นระบบ	รูปทรง	0	X ตามตัว	<code>Normal(mean, ...)</code> :157

θ จึงมี $187,272 + 8 = 187,280$ ตัว — เวลาบอกว่า "ฝึก policy" จริง ๆ แล้ว หมายถึง "ปรับ θ " ทั้งสองส่วนนี้พร้อมกัน ส่วน ③ ไม่มีอะไรให้ปรับ ประกอบสามชั้นเข้าด้วยกันได้สมการแรกของภาคนี้:

$$\pi_{\theta}(\cdot \mid s) = \mathcal{N}(\mu_{\theta}(s), \sigma), \quad \sigma = e^{\ell}$$

(9)

ความหมาย: ใส่ s เข้า NN ได้จุดกลาง $\mu_{\theta}(s)$ · ยกกำลัง e ของลูกบิต ℓ ได้ความกว้าง σ · วางระบบ Gaussian (ระฆังคว่ำ) ที่จุดกลางนั้นด้วยความ กว้าง นั้น — ระบบใบนี้คือ "คำตอบ" ที่ policy ให้กับสถานะ s ตอนแทน action ที่สั่ง ออกไปคือการสุ่มจากระบบใบนี้ ไม่ใช่ค่า μ ตรง ๆ (ถ้าไม่ต้องการสุ่ม — หัวข้อ 2.4)

ธรรมเนียมการเขียน. ไม่ตบชุดนี้เขียน $\mathcal{N}(\mu, \sigma)$ โดยตัวหลังคือ ส่วนเบี่ยงเบนมาตรฐาน (standard deviation) ตรงกับ `Normal(loc, scale)` ของ PyTorch · ดำรัสส่วนใหญ่เขียน $\mathcal{N}(\mu, \sigma^2)$ โดยตัวหลังคือความแปรปรวน — ตัวเลขเดียวกัน แต่ถ้าอ่านสลับกันจะได้ความกว้างผิดไปหนึ่งกำลังสอง

Remark เรื่องสัญลักษณ์. สังเกตว่า $\mu_{\theta}(s)$ มี (s) ต่อท้าย แต่ σ ไม่มี — ตั้งใจเขียนต่างกัน เพราะในโค้ด `self.logstd = nn.Parameter(torch.zeros(8))` (`actor_critic.py:118`) คือ ℓ เป็นพารามิเตอร์เดี่ยว ๆ 8 ตัว ที่ไม่ขึ้นกับ state เลย (ฝึกได้ แต่ใช้ค่าเดียวกันหมดทุกสถานการณ์) ต่างจาก μ ที่คำนวณใหม่ ทุก step จาก s · ค่าเริ่มต้นของ Tron1 คือ $\ell = 0 \Rightarrow \sigma = e^0 = 1.0$ (`torch.zeros`; บรรทัด :117) ที่จะใช้ `init_noise_std` จาก `limx_rsl_rl_ppo_cfg.py:87` ถูกคอมเมนต์ทิ้งไว้ ค่า 1.0 ใน `cfg` จึงตรงกันโดยไม่ได้ถูก ใช้ · ใน toy เราตั้ง $\sigma = 0.5$ ไว้ตลอด นั่นคือ $\ell = \ln 0.5 = -0.6931$

ในช่วง e0 t0: ① คาย $\mu = 0.20$ · ② ให้ $\sigma = 0.5$ · ③ ประกอบเป็นระบบ $\mathcal{N}(0.20, 0.5)$ — ระบบใบนี้คือ $\pi_{\theta}(\cdot \mid s_0)$ ของ env 0 และจะถูกใช้ ต่อทุกหัวข้อในภาคนี้

stochastic กับ deterministic — และ NN เดี่ยว ๆ เป็น policy หรือไม่

แบบ	สัญกรณ์	คืออะไร	ใครใช้
stochastic (สุ่ม)	$a \sim \pi_{\theta}(\cdot \mid s)$	การแจกแจง แล้วสุ่ม	Tron1 ตอนแทน · PPO, SAC (soft actor-critic)
deterministic (ไม่สุ่ม)	$a = \mu_{\theta}(s)$	action ตรง ๆ	Tron1 ตอน deploy · DDPG (deep deterministic policy gradient), TD3 (twin delayed DDPG)

ตอนแทน NN เดี่ยว ๆ ไม่ใช่ policy — `self.actor(obs)` คืนเวกเตอร์ ไม่ใช่ระบบ ต้องมี ② กับ ③ ถึงจะครบชนิด $\mathcal{S} \rightarrow \Delta(\mathcal{A})$ · ตอน deploy (หัวข้อ 2.6) ใช้ ① อย่างเดียว ส่ง μ ไปเป็น action ตรง ๆ ชนิดกลายเป็น $\mathcal{S} \rightarrow \mathcal{A}$ — ตอนนั้น NN เป็น policy ทั้งตัว นี่คือเหตุผลที่โปรแกรมเมอร์กับ ดำรัสใช้คำว่า "policy" ไม่ตรงกันโดยที่ไม่มีใครผิด · รูป E [diagrams/tron1-policy-objects.html](#) วาดสายส่งค่าจาก s ถึง a พร้อมบอกว่าแต่ละช่วง เป็นวัตถุชนิดไหน

Tron1 มี neural network สองก้อน ไม่ใช่ก้อนเดียว

ระหว่างการฝึก Tron1 ไม่ได้ฝึก neural network แยกกันเดียว แต่ฝึก สองก้อน คู่กัน ที่มีหน้าที่ต่างกันสิ้นเชิง — ต้องรู้จักตรงนี้ก่อน เพราะหัวข้อต่อ ๆ ไป จะอ้างถึงทั้งคู่ (เหตุผลเชิงอัลกอริทึมว่าทำไมต้องมีสองก้อน อยู่ในหัวข้อ 4.2)

	Actor (π_{θ})	Critic (V_{ϕ})
หน้าที่	ตัดสินใจว่าจะส่งมอบข้อต่อเท่าไร — คือ policy ที่เราคุยกันมาตลอด	ให้คะแนนว่า "สถานการณ์ตอนนี้ดีแค่ไหน" ใช้เป็นครูสอน actor
output	8 ตัวเลข (μ — จุดกลางของระบบ)	1 ตัวเลข (ค่าประมาณของ $V^{\pi}(s)$ จากหัวข้อ 1.6)
เห็น input อะไรบ้าง	42 มิติ — obs 36 ที่เซนเซอร์วัดได้ + command 3 + latent 3 จาก encoder (<code>ppo.py:137-139</code>)	230 มิติ (privileged 227 + command 3, <code>ppo.py:134</code>) — ทุกอย่างที่ actor เห็น ยก ข้อมูลลับที่มีแค่ในซิม
พารามิเตอร์	θ — 187,280 ตัว	ϕ — 282,625 ตัว (<code>actor_critic.py:92-111</code>)
ถูก deploy ลงหุ่นยนต์ไหม	ใช่ — export เป็น <code>.onnx</code> ไปรันบนหุ่นยนต์จริง	ไม่ — ถูกทิ้งกันที่ฝึกเสร็จ

คำว่า "เห็น" ในตารางนี้ไม่ได้หมายถึงการมองด้วยกล้อง — หมายถึง "เป็น ตัวเลข input ที่ network ก้อนนั้นรับเข้าไปคำนวณได้" เท่านั้น

ใน 227 มิติของ critic (`CriticCfg limx_base_env_cfg.py:179-219`) มี 36 มิติที่เป็น ชุดเดียวกับ actor (แต่ไม่ใช่ noise) ส่วนที่เหลือคือข้อมูลลับที่ actor ไม่เห็น: `base_lin_vel` (ความเร็วลำตัวจริง — ตัวที่หายไปจนทำให้ไม่ Markov ในหัวข้อ 1.3 นั้นเอง, :183), แรงบิดและความเร่งของข้อต่อ, ความเร็วและ แรงสัมผัสของเท้า, ตำแหน่งและความเร็วของลำตัว, และค่าพารามิเตอร์ที่ระบบสุ่มมาในรอบนั้น (`robot_mass, robot_inertia,`

`robot_joint_stiffness, robot_joint_damping, robot_material_properties`) · ส่วน `heights` (ระยะพื้นรอบตัว) มีอยู่ในนิยาม `CriticCfg (:201)` แต่ถูกปิดใช้งาน `Blind-Flat` (`limx_solefoot_env_cfg.py:76-78`) จึงไม่อยู่ใน 227 มิติ — คำว่า *Blind* ในชื่อ `task` หมายถึงตรงนี้

ทำไมให้ critic โหลดเลยได้: เพราะ critic ไม่เคยต้องไปทำงานบนหุ่นยนต์ จริง มันมีชีวิตอยู่แค่ในซิมูเลเตอร์ระหว่างการฝึกเท่านั้น ยิ่งมันประเมินสถานการณ์ได้แม่นยำ (จากข้อมูลที่ครบกว่า) ก็ยิ่งสอน actor ได้ดี ส่วน actor ต้องถูกบังคับให้เก่งจากข้อมูลจำกัด เพราะมันคือตัวที่จะไปเจอโลกจริงที่ไม่มีเลยให้ดู — เทคนิคนี้เรียกว่า **asymmetric actor-critic** (รูป **A** [diagrams/tron1-actor-critic.html](#))

ในตาราง 2 × 4: แถว μ คือสิ่งที่ actor คาย และแถว V คือสิ่งที่ critic คาย ในแต่ละช่อง — สองก้อนนี้ทำงานทุก step คู่กัน แต่ค่าของมันไปคนละทาง: μ ไปเป็น ระยะแล้วสุ่ม action ส่วน V เก็บไว้ให้คะแนน action ที่หลัง (ภาค 4)

สรุปหัวข้อ — π_θ เป็นฟังก์ชันที่รับ s คืนระยะหนึ่งใน ประกอบด้วย NN + `logstd` + สูตร $\mathcal{N}$ (สมการ (9)); NN เดี่ยว ๆ เป็น policy เฉพาะตอน `deploy` · Tron1 ฝึก NN สองก้อน actor กับ critic โดย critic เห็นมากกว่า

2.2 ข้างใน neural network — แยกให้เห็นทีละชั้น

หัวข้อก่อนบอกว่าชั้น ① "คือก้อน neural network" ลอย ๆ — ที่นี้เกาะดูว่าข้างใน ก้อนนั้นมีอะไรจริง ๆ และ "การเรียนรู้" ไปแตะตัวเลขไหน

หนึ่ง node คืออะไร

Node (เรียกอีกชื่อว่า *neuron*) หนึ่งตัว ไม่ได้ทำอะไรซับซ้อน — มันรับตัวเลข หลายตัวเข้ามา คูณแต่ละตัวด้วย "น้ำหนัก" (*weight*) ของตัวเอง บวกกันทั้งหมด บวกค่าคงที่อีกตัว (*bias*) แล้วส่งผลรวมนั้นผ่านฟังก์ชันไม่เชิงเส้นตัวหนึ่ง (*activation function*) ก่อนส่งออกไป

$$\text{node output} = f\left(\underbrace{w_1x_1 + w_2x_2 + \dots + w_nx_n + b}_{\text{ผลรวมถ่วงน้ำหนัก}}\right)$$

(10)

สัญลักษณ์	ชนิด	ความหมาย
$x_1, x_2, \dots, x_n$	scalar n ตัว	ตัวเลข input ที่ node นี้รับเข้ามา (มาจาก node ชั้นก่อนหน้า หรือมาจาก observation ตรง ๆ ถ้าเป็นชั้นแรก)
$w_1, w_2, \dots, w_n$	scalar n ตัว — parameter	weight — ตัวเลขที่บอกว่า input แต่ละตัว "สำคัญแค่ไหน" ต่อ node นี้ นี่คือสิ่งที่ถูกปรับระหว่างฝึก
b	scalar — parameter	bias — ค่าคงที่พิเศษ ปรับ threshold ว่า node นี้จะ "ตื่น" ง่ายหรือยาก
$f(\cdot)$	ฟังก์ชัน $\mathbb{R} \rightarrow \mathbb{R}$	activation function — ฟังก์ชันไม่เชิงเส้น (Tron1 ใช้ ELU — บล็อกถัดไป) ที่บิดผลรวมให้ไม่เป็นเส้นตรง — ถ้าไม่มีขั้นนี้ ต่อให้ชั้นที่ขึ้น เครือข่ายก็กองก็ยังไม่ทำงานเชิงเส้นเท่าไร

หนึ่ง**ชั้น** (*layer*) คือกลุ่มของ node หลายตัวที่รับ input ชุดเดียวกัน แต่ แต่ละตัวมี weight ของตัวเองไม่เหมือนกัน คำว่า "512" ใน `42 → 512 → 256 → 128 → 8` แปลว่าชั้นนั้นมี node 512 ตัว โดยแต่ละตัวรับ input ทั้ง 42 ค่าเข้ามา พร้อมกัน (คนละชุด weight) แล้วคาย output ออกมาคนละ 1 ค่า รวมเป็น 512 ค่า ส่งต่อให้ชั้นถัดไป · ในโค้ดหนึ่งชั้นคือ `nn.Linear(in, out)` หนึ่งบรรทัด ตามด้วย activation หนึ่งบรรทัด (`actor_critic.py:71-88`) และตัวเลข 512/256/128 มาจาก `actor_hidden_dims` ที่ `limx_rsl_rl_ppo_cfg.py:88`

Remark. "เครือข่าย" (*network*) คือการเอา node หลายชั้นมาต่อกันเป็นสาย — output ของชั้นหนึ่งเป็น input ของชั้นถัดไป ไล่ไปเรื่อย ๆ นี่คือการมาของคำว่า "deep" ใน *deep learning*: deep แปลว่ามีหลายชั้นซ้อนกัน ไม่ได้แปลว่าฉลาดลึกซึ้ง · โครงข่ายแบบนี้ (ชั้นเชิงเส้นสลับ activation ต่อกันเป็นสายเดียว) มีชื่อเรียกว่า **multilayer perceptron (MLP)** — คำที่ `actor_critic.py:113` พิมพ์ออกมาตอนสร้าง

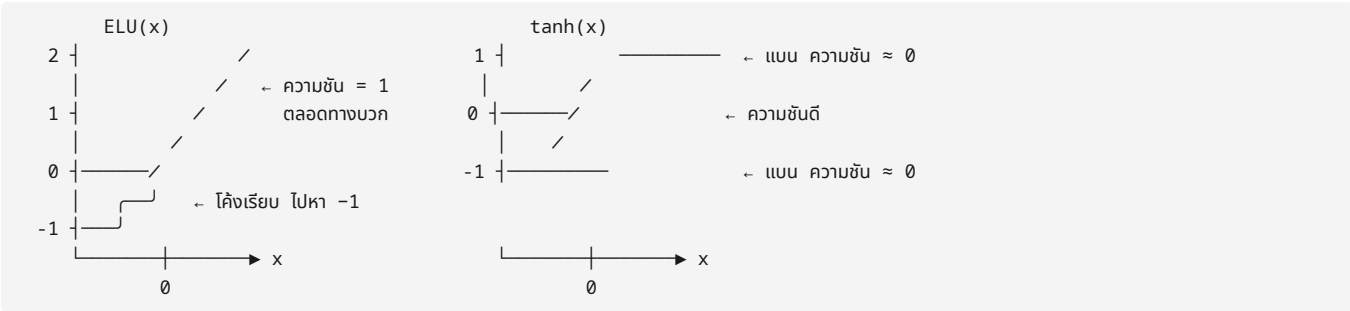
ฟังก์ชัน f ที่ Tron1 ใช้ — ELU

f ในสมการ (10) ของ Tron1 คือ **ELU** (*exponential linear unit*) ตั้งใน `cfg` เป็น `activation="elu"` (`limx_rsl_rl_ppo_cfg.py:90`) และแปลงเป็น `nn.ELU()` ที่ `actor_critic.py:175-177`:

$$\text{ELU}(x) = \begin{cases} x & x > 0 \\ e^x - 1 & x \leq 0 \end{cases}$$

(11)

ความหมาย: ทางบวกปล่อยผ่านตรง ๆ (ความชัน 1 เสมอ) ทางลบค่อย ๆ โค้งลงไปหา -1 แบบเรียบ ๆ (ความชัน e^x ซึ่งเล็กแต่ไม่เป็นศูนย์) — เทียบกับ `tanh` ที่จะเจอใน บล็อกถัดไป:



ฟังก์ชัน	ทางบวก	ทางลบ	ผลต่อการเรียนรู้
ELU	ไม่อั้นตัว ความชัน 1	โค้งเรียบ ความชัน $e^x > 0$	gradient ไม่ตายทั้งสองข้าง
tanh	อั้นตัวที่ 1 ความชัน $\rightarrow 0$	อั้นตัวที่ -1 ความชัน $\rightarrow 0$	gradient ตายเมื่อ $ x $ ใหญ่
ReLU ($\max(0, x)$)	ความชัน 1	ศูนย์พอดี ความชัน 0	node ที่ตกทางลบ "ตาย" ถาวร

ELU จึงเป็นทางสายกลาง: ได้ความไม่เชิงเส้นที่ต้องการ โดย gradient ยังไหลผ่านได้ ทุกช่วง — เหตุผลที่ต้องแคร์เรื่อง gradient ไหลผ่านจะขัดในบล็อกร gradient descent ทำยหวัข้อนี้

ทำไมขั้นสุดท้ายถึงไม่มี activation — และมันควรมีไหม

ในโค้ดจริง ขั้นสุดท้ายของ actor คือ `nn.Linear(128, 8)` (`actor_critic.py:77`) ที่ไม่มี activation ต่อท้าย ต่างจากชั้นกลางที่มี ELU ทุกชั้น — ตั้งใจ ไม่ใช่ลืมนะ

เริ่มจากแยกให้ออกว่า activation มี สองหน้าที่:

หน้าที่	คำอธิบาย	จำเป็นตรงไหน
(ก) สร้างความไม่เชิงเส้น	ทำให้การซ้อนหลายชั้นมีความหมาย	ชั้นกลางเท่านั้น
(ข) จำกัดช่วงของ output	เช่น tanh บีบเป็น $[-1, 1]$, sigmoid บีบเป็น $[0, 1]$	ขึ้นกับว่าอยากได้ไหม

หน้าที่ (ก) สำคัญมากที่ชั้นกลาง — ถ้าถอด ELU ออกจากชั้นกลางทั้งหมด เครือข่าย 42-512-256-128-8 จะยุบเหลือการคูณเมทริกซ์ต่อกัน ซึ่งเท่ากับเมทริกซ์เดียว = **ชั้นเดียว** ความลึกหายไปหมด (เพราะการคูณเมทริกซ์ต่อกันยังคงเป็นเมทริกซ์เดียว เสมอ)

แต่ ที่ขั้นสุดท้าย หน้าที่ (ก) **ไม่มีความหมายเลย** เพราะไม่มีชั้นไหนต่อจากมัน อีกแล้ว — จะไม่เชิงเส้นเทียบกับอะไร? เหตุผลเดียวที่จะใส่ activation ตรงนี้ จึงมี แค่หน้าที่ (ข) คืออยากบีบช่วงค่า ซึ่งเราไม่อยากจะสามเหตุผล:

1. μ คือจุดกลางของ Gaussian ซึ่งทางคณิตศาสตร์เป็นจำนวนจริงอะไรก็ได้ — ถ้าใส่ tanh ปิดท้าย μ จะถูกล็อกใน $[-1, 1]$ ตลอดกาล ต่อให้ค่าเดินที่ที่ดีที่สุด ต้องการ action = 1.5 policy ก็ไปไม่ถึง เป็นการตัดความสามารถทิ้งโดยไม่จำเป็น

2. tanh ทำให้ gradient ตาย (สำคัญกว่าข้อแรก) — ดูกราฟขวาในบล็อก ELU: ที่ปลาย ทั้งสองข้างของ tanh กราฟแบนราบ ความชันเข้าใกล้ 0 ถ้า action ที่ดีที่สุดคือ 0.99 ต้องมี $x \approx 2.6$ ถ้าอยากได้ 0.999 ต้องมี $x \approx 3.8$ — ยิ่งเข้าใกล้ขอบยิ่ง ต้องดัน x ไปไกลแบบทวีคูณ ในขณะที่ gradient ที่ไหลย้อนกลับ (สมการ (12) ทำยหวัข้อ) ยิ่งเล็กลงเรื่อย ๆ $\rightarrow$ weight แทบไม่ถูกปรับ ถ้าคำตอบที่ต้องอยู่ใกล้ขอบ tanh จะกัน **ไม่ให้ไปถึง**

3. มีตัวกันพ่วงอยู่แล้วที่อื่น ไม่ต้องพ่วง activation:

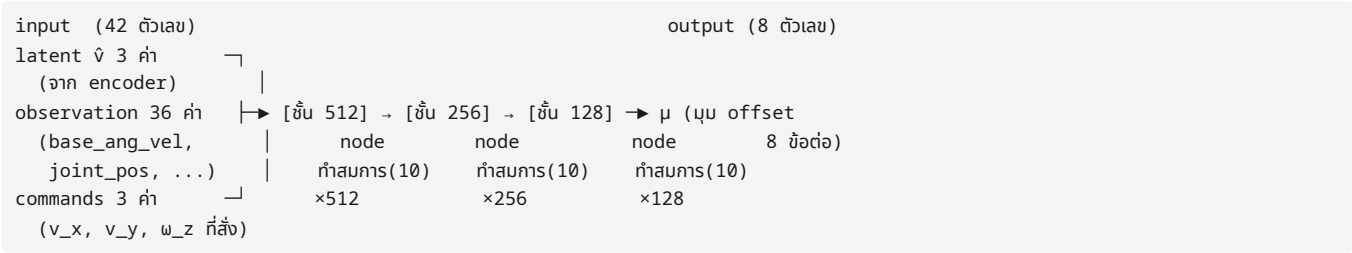
ด้านกัน	อยู่ตรงไหน	ค่า	ที่
scale	ตอนแปลงเป็นเรเดียน (หวัข้อ 2.5)	0.25 (บีบ $\pm 1 \rightarrow \pm 0.25$ rad)	<code>limx_base_env_cfg.py:119</code>
effort_limit	ที่ actuator	80 N·m	<code>solefoot_cfg.py:48, :59</code>
joint limits	ที่ตัวหุ่นยนต์	<code>soft_joint_pos_limit_factor=0.9</code>	<code>solefoot_cfg.py:37</code>
pen_joint_pos_limits	ที่ reward	น้ำหนัก -2.0 ลงโทษเมื่อเข้าใกล้ลิมิต	<code>limx_base_env_cfg.py:396</code>

สรุป: ไม่ควรมี และการที่ไม่มีคือการออกแบบที่ถูกต้อง เป็นมาตรฐานของ policy gradient สำหรับ continuous control ทุกตัว สังเกตว่า critic ก็ไม่มี activation ที่ขั้นสุดท้ายเหมือนกัน (`nn.Linear(128, 1)`, `actor_critic.py:98`) ด้วยเหตุผลเดียวกัน — $V(s)$ อาจเป็น 650 หรือ -200 ก็ได้ จะบีบให้อยู่ $[-1, 1]$ ทำไ้

ข้อยกเว้นที่มีอยู่จริง. อัลกอริทึม SAC ใช้ tanh บีบ output จริง แต่ ต้องเพิ่มพจน์แก้ค่า log-probability (Jacobian correction) เข้าไปในสมการ เพื่อ ชดเชยการบิดเบือนของ distribution — เป็นการออกแบบที่ตั้งใจและมีคณิตศาสตร์รองรับ ไม่ใช่แค่ "ปะ tanh เลย ะ" ซึ่ง PPO ใน `rs1_rl` ไม่ได้ทำแบบนั้น

Input / Output ของชั้น ① ทั้งก่อนคืออะไรกันแน่

ไม่ใช่แค่ node เดียว แต่ทั้งเครือข่าย 42 → 512 → 256 → 128 → 8 รวมกัน ทำหน้าที่เป็นฟังก์ชันเดียว $\mu_\theta : \mathbb{R}^{42} \rightarrow \mathbb{R}^8$: รับเวกเตอร์ 42 ตัวเลขเข้า
กายเวกเตอร์ 8 ตัวเลขออก



ตัวเลข 42 และ 8 **ไม่ใช่ทางเลือก** — 42 = 3 + 36 + 3 คือ latent จาก encoder (**encoder คืออะไร อธิบายเต็ม ๆ ในหัวข้อ 4.4** — ตอนนี้อยู่แค่ว่ามันคือ NN ก่อนที่สาม ที่เอาความเร็วลำตัวจากประวัติ observation แล้วส่งค่าเอา 3 ตัวชื่อ $\hat{v}$ มาให้) + observation ที่เซนเซอร์วัดได้ + เป้าหมายความเร็วที่ผู้ใช้สั่ง ทั้งหมดถูกบังคับโดยโจทย — ประเด็นคือ **input/output ถูกกำหนดโดยโจทย ไม่ใช่ถูกเลือกเอง** ส่วน 8 คือจำนวน บิตต่อจริงของ SF (2 ขา × 4 บิตต่อ) เปลี่ยนไม่ได้เพราะเป็นจำนวนบิตของจริงบน หุ่นยนต์

ที่มาของเลข 42 (ตรวจสอบจากโค้ดแล้ว). โน้ต 03 นั้น observation ได้ 36 เพราะ มันเฉพาะ group policy — แต่ actor รับ input สามก้อน ต่อกัน (ppo.py:137-139): cat(encoder_out, obs, commands) = 3 + 36 + 3 = 42 โดย commands คือ เป้าหมายความเร็ว (v_x, v_y, w_z) ที่ผู้ใช้สั่ง เป็น observation group แยกชื่อ "commands" ไม่ได้อยู่ใน 36 — critic ก็ได้ commands ต่อกันเช่นกัน จึงเป็น 227 + 3 = 230 (ppo.py:134) รายละเอียดเต็มอยู่ในโน้ต 06 หัวข้อ 1

Weight กับขนาด network ต่างกันยังไง — และทำไมขนาดถึง "ไม่ถูกปรับ" ระหว่างฝึก

ตัวเลข 512, 256, 128 (ชั้นตรงกลาง) ต่างจาก 42/8 ตรงที่ **เลือกเองได้** — ตรงนี้ตอบคำถาม "ทำไมขนาดเท่านี้" ได้ตรง ๆ: **ไม่มีสูตรคณิตศาสตร์ที่คำนวณ ขนาดที่ "ถูกต้อง" ออกมาได้** ขนาดชั้นกลางเป็นตัวเลขที่ผู้ออกแบบ (LimX) เลือก จากประสบการณ์และการทดลอง (trial and error) — ไม่ใช่ค่าที่ derive มาจากทฤษฎี เครือข่ายใหญ่กว่านี้ (เช่น 1024 → 512 → 256) ก็มีความสามารถจะเรียนรู้ได้ เหมือนกัน อาจจะดีกว่าเล็กน้อยหรือแย่กว่า (overfit, ฝึกช้าลง)

จุดที่ต้องแยกให้ออกคือ **สองสิ่งที่ถูกเปลี่ยนแต่เป็นคนละประเภท:**

	Weight (θ)	ขนาด/สถาปัตยกรรมของ network
ใครเป็นคนกำหนด	gradient descent ปรับให้เอง ระหว่างฝึก	มนุษย์กำหนดไว้ ก่อน เริ่มฝึก แล้วคงที่ตลอด
เปลี่ยนแปลงได้ไหม ระหว่างฝึก	เปลี่ยนทุก iteration (คือ "การเรียนรู้" ทั้งหมด)	ไม่เปลี่ยนเลยตลอดการฝึกหนึ่งรอบ
เรียกว่าอะไรในศัพท์ ML	parameter	hyperparameter
ตัวอย่างจำนวน	actor มี weight 187,272 ตัว (42×512 + 512×256 + 256×128 + 128×8 = 186,368 บวก bias 512 + 256 + 128 + 8 = 904)	แค่ 3 ตัวเลข: 512, 256, 128 (limx_rsl_rl_ppo_cfg.py:88-90)

นี่คือคำตอบของคำถาม "ขนาด network มีผลไหม เกี่ยวกับการปรับ weight" — จริง ๆ แล้วขนาด network **มีผล** ต่อคุณภาพของ policy สุดท้ายเหมือนกัน (network เล็ก เกินไปอาจเรียนรู้รู้เท่าเดินซับซ้อนไม่ได้, ใหญ่เกินไปอาจฝึกช้าและ overfit) เพียงแต่ **ผลของมันถูกทดสอบด้วยวิธีคนละแบบ:** weight ถูกปรับ "ภายใน" การฝึกหนึ่งรอบด้วย gradient descent อัตโนมัติ ส่วนขนาด network ถูกทดสอบด้วยการฝึกซ้ำหลายรอบด้วย **ขนาดต่างกันแล้วเทียบผลลัพธ์** (เรียกว่า hyperparameter search) — เป็นการทดลอง นอกกรอบการฝึก ไม่ใช่ในลูป

ตารางใน notes/04-file-structure-vs-ros2.md ระบุว่า "network architecture beyond hidden sizes" แก่ได้ที่ rsl_rl/modules/actor_critic.py ส่วนขนาดชั้น แก่ได้ที่ agents/limx_rsl_rl_ppo_cfg.py — เป็นค่าที่แก้ไขได้อิสระ ต่างจาก 42/8 ที่แก้ไขไม่ได้ เพราะผูกกับโจทยจริง

ตอนปรับ weight รู้ได้ไงว่าต้องปรับตัวไหน — หลักการของ gradient descent

คำถามนี้คือหัวใจของการฝึก neural network ทั้งหมด และคำตอบสั้น ๆ คือ: **ไม่มีใครไป "เดา" ว่า weight ตัวไหนควรปรับ** — มันคำนวณออกมาได้ **แม่นยำด้วย แคลคูลัส (calculus) ที่ละตัวพร้อมกันทั้งหมด**

ลองนึกภาพง่าย ๆ ก่อน (ยังไม่มี neural network เลย): สมมติเราอยู่บนภูเขา หมอกลงจัดมองไม่เห็นอะไรเลย อยากเดินลงไปยังจุดต่ำสุด (หุบเขา) วิธีที่ทำได้ คือ **คลำพื้นรอบเท้าดูว่าทางไหนลาดลงชันที่สุด แล้วก้าวไปทางนั้นทีละก้าว** ทำซ้ำไปเรื่อย ๆ จนถึงจุดต่ำสุด — นี่คือ **gradient descent** ทั้งหมดในหนึ่ง ประโยค

ทีนี้แทนที่ "ความสูงของภูเขา" ด้วย **loss** (ในกรณี RL คือค่าลบของ objective ในสมการ (2) — ยิ่ง policy แ่ ยิ่งค่าสูง) และแทนที่ "ตำแหน่งที่ยืนอยู่บนภูเขา" ด้วย **ค่า weight ทุกตัวรวมกัน (θ)** เราจะได้ภาพเดียวกันเป๊ะ: **ภูเขานี้ไม่ใช่ 2 มิติ แต่มีจำนวนมิติเท่ากับจำนวน weight ทั้งหมด** (187,280 มิติ สำหรับ policy ของ Tron1) และเราต้องหาว่า จะ "ก้าว" ไปทิศทางไหนในพื้นที่ หลักแสนมิตินั้น เพื่อให้ loss ลดลงเร็วที่สุด

Gradient คือคำตอบของคำถามนั้น — มันคือเวกเตอร์ที่บอก **สำหรับ weight แต่ละตัว ถ้าเพิ่มมันขึ้นนิดหนึ่ง loss จะเปลี่ยนไปเท่าไร** (คือ อนุพันธ์ย่อย, partial derivative ของ loss เทียบกับ weight ตัวนั้น) — ค่ามันได้สำหรับ **weightทุกตัวพร้อมกันในครั้งเดียว** ด้วยเทคนิคที่เรียกว่า **backpropagation** (ไล่คำนวณ derivative ย้อนจาก output กลับไปยัง input ทีละชั้น โดยอาศัย chain rule ของแคลคูลัส)

สัญลักษณ์	ชนิด	ความหมาย	ค่า Tron1
$L(\theta)$	ฟังก์ชัน $\mathbb{R}^{187,280} \rightarrow \mathbb{R}$	loss — ตัวเลขเดียวที่บอกว่า policy (ที่มี weight ปัจจุบัน θ) แย่แค่ไหน ยิ่งน้อยยิ่งดี · ตัว L ใหญ่ คนละตัวกับ ℓ ลูกบิตความกว้าง	หน้าตาจริงของมันคือทั้งภาค 4
$\nabla_{\theta} L$	เวกเตอร์ยาว 187,280	gradient — บอกทิศทางที่ทำให้ L เพิ่มขึ้น เร็วที่สุด	<code>loss.backward()</code>
α	ค่าคงที่	learning rate (lr) — ก้าวยาวแค่ไหนต่อครั้ง	<code>learning_rate=1.0e-3</code> (<code>limx_rsl_rl_ppo_cfg.py:100</code>)

อัปเดต weight ทุกตัวพร้อมกันด้วยกฎเดียว:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L(\theta)$$

(12)

ความหมาย: เดินสวนทางกับ gradient (เพราะ gradient ชี้ทางที่ loss *เพิ่ม* เราจึงเดินย้อนกลับเพื่อให้ loss *ลด*) ระยะก้าวถูกคุมด้วย α — weight ตัวที่ "มีผลต่อความแย่ของ policy มาก" จะมีค่า derivative (ในเวกเตอร์ $\nabla_{\theta} L$) ที่ขนาดใหญ่ จึงถูกปรับมากกว่า ส่วน weight ที่แทบไม่มีผล จะมีค่า derivative ใกล้ศูนย์ แทบไม่ถูกแตะเลย — **นี่คือคำตอบของ "รู้ได้ไงว่า ต้องปรับตัวไหน": ไม่ใช่การเลือก แต่เป็นตัวเลขที่คำนวณออกมาได้ตรง ๆ ว่า weight ไหนควรปรับมากปรับน้อยแค่ไหน ในทุกรอบ** · และนี่คือเหตุผลที่ ELU ถูกเลือก: ถ้า activation ทำให้ derivative เป็นศูนย์ weight ตัวนั้นจะไม่ถูกแตะเลยแม้จะควรปรับ

Remark. ในทางปฏิบัติ PyTorch (ที่ Tron1 ใช้) คำนวณ backpropagation ให้อัตโนมัติทั้งหมด — โค้ดใน `algorithm/ppo.py` มีสับสกริปต์หลัก (`ppo.py:287-290`): `optimizer.zero_grad()` (ล้างค่า gradient เก่า), `loss.backward()` (คำนวณ $\nabla_{\theta} L$ ทุกตัวด้วย backprop), `clip_grad_norm(..., max_grad_norm)` (ถ้าเวกเตอร์ gradient ยาวเกิน 1.0 ให้ย่อลง ทั้งเวกเตอร์ — `max_grad_norm=1.0` ที่ `limx_rsl_rl_ppo_cfg.py:105`) และ `optimizer.step()` (ทำสมการ (12) จริง ๆ) ไม่มีใครตั้งคำนวณ derivative ด้วยมือ · optimizer ที่ใช้คือ **Adam** (`optim.Adam ppo.py:85`) ซึ่งเป็นสมการ (12) ที่ปรับความยาวก้าวให้แต่ละ weight แยกกัน — กลไกข้างในอยู่ใต้ 06 หัวข้อ 3.4

สรุปหัวข้อ — NN คือ node ที่ทำสมการ (10) ซ้อนกันเป็นชั้น คั่นด้วย ELU (11) ยกเว้นชั้นสุดท้าย; 42 กับ 8 ถูกบังคับโดยโจทย์ ส่วน 512/256/128 เป็น hyperparameter; weight ทุกตัวถูกปรับ พร้อมกันด้วยสมการ (12) โดยทิศทางมาจาก gradient ไม่ใช่การเดา

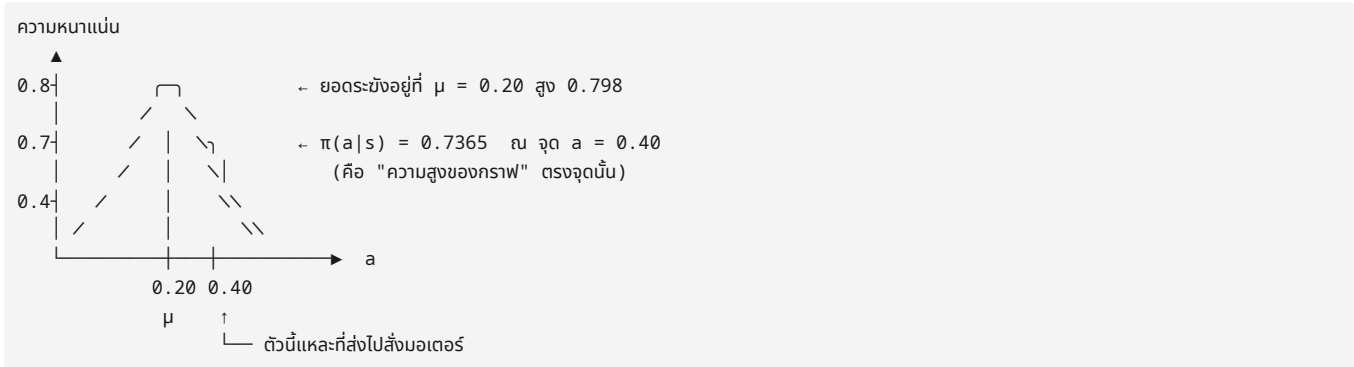
2.3 $\mu, a, \pi_{\theta}(a \mid s)$ — policy คายอะไรออกมากันแน่

สมการ (9) อ่านแล้วมักตีความผิดเป็น *"network คายความน่าจะเป็นออกมา"* — ไม่ใช่ network คาย μ เท่านั้น ส่วน $\pi_{\theta}(a \mid s)$ ต้อง **คำนวณเพิ่ม** จาก μ กับ a อีกทีหนึ่ง

สามสิ่งที่ต่างกัน แต่ชื่อคล้ายจนสับสน

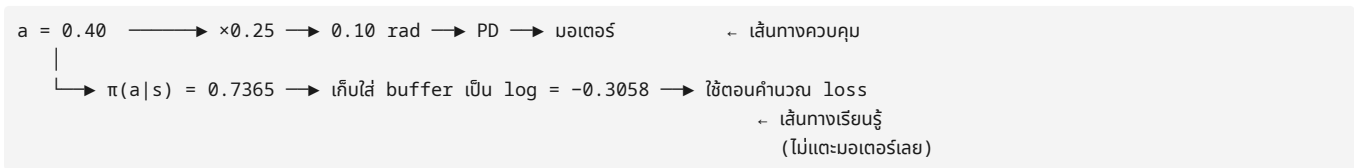
	คืออะไร	ชนิด	ใครสร้าง	เอาไปทำอะไร	e0 t0
μ	จุดกลางที่ network คายออกมา	เวกเตอร์ 8 ตัวเลข	<code>self.actor(obs)</code>	เป็นศูนย์กลางของระฆังคว่ำ	0.20
a	action ที่สุ่มได้	เวกเตอร์ 8 ตัวเลข	<code>distribution.sample()</code>	ส่งไปสั่งมอเตอร์	0.40
$\pi_{\theta}(a \mid s)$	ความหนาแน่นความน่าจะเป็นของ a นั้น	1 ตัวเลข	<code>get_actions_log_prob(a)</code>	ส่งให้อัลกอริทึม PPO เท่านั้น	0.7365

ดูที่เข้าซ้ายข้อเดียว (toy อ่าน action 1 มิติเป็นเข้าซ้าย) ช่อง e0 t0: $\mu = 0.20, \sigma = 0.5$:



คำถามที่เกิดขึ้นตามธรรมชาติ: ถ้า $\pi_\theta(a | s)$ แปลว่า "ความน่าจะเป็นที่ จะสั่งมอเตอร์ไปตำแหน่ง a " แล้วมันเอาไปทำอะไรกับมอเตอร์?

คำตอบ: ไม่ทำอะไรเลย — ตัวเลขนี้ไม่เคยถูกส่งไปสั่งมอเตอร์ มันถูกคำนวณขึ้นมา จาก a ตัวที่เพิ่งสุ่มได้ เพื่อให้อัลกอริทึมเรียนรู้ใช้อย่างเดียว จากจุดนั้นสองค่า แยกกันไปคนละทาง:



นี่คือเหตุผลที่ตอน deploy (หัวข้อ 2.6) ถึง $\pi_\theta(a | s)$ ได้เลยโดยไม่มีผลอะไร — บนหุ่นจริงไม่ต้องเรียนรู้แล้ว

ความหนาแน่นเกิน 1 ได้ — เพราะสิ่งที่ต้องเท่ากับ 1 คือ พื้นที่ ไม่ใช่ ความสูง

ที่ $\sigma = 0.5$ ความสูงยอดระฆังคือ 0.798 ยังไม่เกิน 1 — แต่ถ้า σ เล็กลง (ซึ่งเกิดขึ้นจริงระหว่างเทรน) ยอดจะสูงเกิน 1 ได้ ความน่าจะเป็นจริง ๆ เกิน 1 ไม่ได้ แต่ตัวเลขนี้ไม่ใช่ความน่าจะเป็น มันคือ **ความหนาแน่นความน่าจะเป็น (probability density)** กฎที่แท้จริงคือ:

$$\int_{-\infty}^{\infty} p(a) da = 1 \quad (\text{พื้นที่ใต้ กราฟทั้งหมด} = 1)$$

ไม่มีกฎไหนบอกว่าความสูงต้อง ≤ 1 ความสูงยอดระฆังคือ $\frac{1}{\sigma\sqrt{2\pi}}$ ซึ่งขึ้นกับ σ อย่างเดียว:

σ	ความสูงยอด	รูปร่าง
1.0	0.399	เตี้ย กว้าง ← ค่าเริ่มต้นของ Tron1
0.5	0.798	← toy ของเรา
0.3	1.330	สูงขึ้น แคบลง
0.25	1.596	เกิน 1 แล้ว — ความหนาแน่นที่ยอดสูงกว่า 1
0.1	3.989	สูงมาก แคบมาก

ยิ่งแคบยิ่งต้องสูง เพื่อรักษาพื้นที่ให้เท่ากับ 1 เสมอ

เปรียบเทียบกับความหนาแน่นมวล. น้ำมีความหนาแน่น 1000 kg/m^3 — เลข 1000 ไม่ได้แปลว่า "หนัก 1000 kg" ต้องเอาไปคูณปริมาตรก่อนถึงได้มวลจริง probability density ก็เหมือนกันเป๊ะ:

ความน่าจะเป็น = ความหนาแน่น $\times$ ความกว้างของช่วง

คำถาม	คำตอบ
$P(a = 0.40 \text{ เป๊ะ ๆ})$	= 0 (ความกว้าง 0 → พื้นที่ 0)
$P(0.39 < a < 0.41)$	$\approx 0.7365 \times 0.02 = 0.0147$ คือ ~1.5%

เลข 0.7365 จึงไม่ใช่ "โอกาส 73.65%" แต่เป็นความหนาแน่นต่อหนึ่งหน่วยของ a — และเลข 1.596 ที่ $\sigma = 0.25$ ก็ไม่ใช่ "โอกาส 159.6%" ด้วยเหตุผลเดียวกัน

คำนวณ $\pi_\theta(a | s)$ ได้ยังไง — สูตร Gaussian PDF

$$p(a) = \underbrace{\frac{1}{\sigma\sqrt{2\pi}}}_{\text{ความสูงยอด}} \times \underbrace{\exp\left(-\frac{(a-\mu)^2}{2\sigma^2}\right)}_{\text{ตัวลดทอน } 0 \rightarrow 1} \quad (13)$$

แยกเป็นสองส่วนแล้วเข้าใจง่ายขึ้นมาก:

ส่วน	ขึ้นกับอะไร	ทำหน้าที่	ช่วงค่า
$\frac{1}{\sigma\sqrt{2\pi}}$	σ อย่างเดียว	กำหนดความสูงยอดระยะขั้ว	เท่าไรก็ได้ (จึงเกิน 1 ได้)
$\exp\left(-\frac{(a-\mu)^2}{2\sigma^2}\right)$	ระยะห่างจาก a ถึง μ	ลดทอนตามระยะห่าง	0 ถึง 1 เสมอ

ความหนาแน่น = ความสูงยอด $\times$ เปอร์เซนต์ที่เหลือเมื่อห่างจากยอด

ไล่คำนวณด้วยตัวเลขจริงของช่อง e0 t0 ($\mu = 0.20, \sigma = 0.5, a = 0.40$):

ขั้น	คำนวณอะไร	ผลลัพธ์	เหตุผล
1	$\sigma\sqrt{2\pi} = 0.5 \times 2.5066$	1.25331	ตัวหารของส่วนหน้า
2	$1/1.25331$	0.79788	ความสูงยอดระยะขั้ว
3	$a - \mu = 0.40 - 0.20$	0.20	ห่างจากยอดเท่าไร
4	$(a - \mu)^2 = 0.20^2$	0.04	ยกกำลังสอง (ย้ายขวาท่กััน)
5	$2\sigma^2 = 2(0.25)$	0.5	ตัวหารที่ปรับตามความกว้างระยะขั้ว
6	$-0.04/0.5$	-0.08	เลขชี้กำลัง
7	$\exp(-0.08)$	0.92312	ตัวลดทอน — เหลือ 92.3% ของยอด
8	0.79788×0.92312	0.73654	คำตอบ

ตัวลดทอนลดเร็วแค่ไหน — วัดระยะเป็น "กี่เท่าของ σ " เรียกว่า **ระยะมาตรฐาน** $z = \frac{a-\mu}{\sigma}$ (ของ e0 t0 คือ $0.20/0.5 = 0.40$) จะได้ ตารางที่ใช้ได้กับทุก Gaussian:

ห่างจากยอด	z	$\exp(-z^2/2)$	เหลือกี่ % ของยอด
ตรงยอดพอดี	0	1.000	100% ← สูงสุด
กรณี e0 t0 ($0.20 = 0.4\sigma$)	0.40	0.923	92.3%
1σ	1	0.607	60.7%
2σ	2	0.135	13.5%
3σ	3	0.011	1.1%

ในโค้ดคำนวณใน log space

`distribution.log_prob(actions)` ของ PyTorch (`torch/distributions/normal.py`) เอา log ของสมการ (13) ทั้งหมด — การคูณกลายเป็นการบวก และ **exp** หายไป:

$$\log p(a) = -\frac{(a-\mu)^2}{2\sigma^2} - \log \sigma - \log \sqrt{2\pi} \quad (14)$$

ความหมาย: สามพจน์ = ตัวลดทอน (ขึ้นกับ a) + ค่าคงที่สองตัวที่ขึ้นกับ σ อย่างเดียว $\cdot$ เหลือแค่บวกลบกับยกกำลังสอง เร็วและเสถียรมากกว่ามาก

ตรวจกับ e0 t0: พจน์แรก = -0.08 (ขั้น 6 ข้างบน), $-\log 0.5 = +0.6931$, $-\log \sqrt{2\pi} = -0.9189$ → สองพจน์หลังรวมเป็น -0.2258 (ค่าคงที่ที่ใช้ซ้ำทุกช่อง ตารางใดที่ $\sigma = 0.5$) → $\log p = -0.08 - 0.2258 = -0.3058$ และ $\ln(0.73654) = -0.3058$ ตรงกัน

log p ติดลบตรงนี้เพราะความหนาแน่น 0.7365 ต่ำกว่า 1 — แต่ถ้า $\sigma = 0.25$ ที่ยอด 1.596 จะได้ **log p = +0.467** เป็นบวก ไม่ใช่ความผิดปกติอะไร ทั้งสองกรณี คือ log ของความหนาแน่น ไม่ใช่ log ของความน่าจะเป็น

ลองเล่นเองได้ ลองเปลี่ยน σ ดูจะเห็นว่ายิ่งเล็กลงยิ่งสูง ($\sigma=0.1 \rightarrow$ ยอด 3.99) ซึ่งเป็นสิ่งที่เกิดขึ้นจริงระหว่างเทรนเมื่อ Mean action noise std ค่อย ๆ ลดลง:

```
import math
def pdf(a, mu, sigma):
    return (1/(sigma*math.sqrt(2*math.pi))) * math.exp(-((a-mu)**2)/(2*sigma**2))

print(pdf(0.40, 0.20, 0.5))          # 0.73654
print(math.log(pdf(0.40, 0.20, 0.5))) # -0.3058
```

จริง ๆ แล้ว $\pi_\theta(\mathbf{a} \mid \mathbf{s})$ เป็นผลคูณของ 8 ตัว

actor_critic.py:163-164:

```
def get_actions_log_prob(self, actions):
    return self.distribution.log_prob(actions).sum(dim=-1)
        ↳ รวม 8 ข้อต่อ
```

`.sum(dim=-1)` ในปริภูมิ log = การคูณ ในปริภูมิปกติ:

$$\pi_\theta(\mathbf{a} \mid \mathbf{s}) = \prod_{i=1}^8 p_i(a_i \mid \mathbf{s}) \iff \log \pi_\theta(\mathbf{a} \mid \mathbf{s}) = \sum_{i=1}^8 \log p_i(a_i \mid \mathbf{s}) \quad (15)$$

ความหมาย: สมการ (13)–(14) คือของข้อต่อเดียว (ตัวห้อย $\mathbf{x}$) · ความหนาแน่นของ "ท่าทั้งตัว" 8 ข้อต่อคือผลคูณ — ในโค้ดจึงเป็นผลบวกของ log แปลงตัว · toy มีมิติเดียว ผลบวกจึงมีพจน์เดียว = **-0.3058** ส่วนเวอร์ชัน 8 มิติจริงอยู่หน้า 06 หัวข้อ 3.5.5

แปลว่าโมเดลถือว่า **ข้อต่อทั้ง 8 สุ่มอิสระจากกัน** — เป็นข้อสมมติที่ทำให้คำนวณง่ายมาก แลกกับการที่ policy แสดง "ความสัมพันธ์ระหว่างข้อต่อ" ในการสุ่มไม่ได้ (เช่น บอกไม่ได้ว่า "ถ้าเข้าซ้ายเบี่ยงขึ้น ควรให้สะโพกซ้ายเบี่ยงลงด้วย")

"Policy คือ NN และฟังก์ชัน π ไซโหม" — สรุปด้วยตารางชนิดของหัวข้อ 2.1

```
policy (ทั้งก้อน) = ① actor NN + ② logstd + ③ กติกาการสุ่ม      ↳  $\pi_\theta$  ชนิด: ฟังก์ชัน
                  ↳ "อยู่ที่  $\mathbf{s}$  ควรทำอะไร" → ตอบด้วย  $\mu$  แล้วสุ่ม → ได้  $\mathbf{a}$ 

 $\pi_\theta(\mathbf{a} \mid \mathbf{s})$  (ตัวเลข) = "คู่  $(\mathbf{s}, \mathbf{a})$  นี้ น่าจะเกิดแค่ไหน" → ตอบเป็นตัวเลขเดียว ↳ ชนิด: scalar
```

อันแรกใช้ควบคุมหุ่นยนต์ อันที่สองใช้เรียนรู้เท่านั้น — และทั้งคู่มารจาก network ก้อนเดียวกัน ความหลวมของคำว่า "policy" ในวงการ RL อยู่ตรงที่ใช้คำเดียว เรียกทั้งสองแถวนี้ ไม่ใช่ความเข้าใจผิดของผู้อ่าน

สรุปหัวข้อ — NN คาย μ ; $\mathbf{a}$ ถูกสุ่มรอบ μ ; $\pi_\theta(\mathbf{a} \mid \mathbf{s})$ คือความสูงของระฆัง ณ $\mathbf{a}$ คำนวณจากสมการ (13) (ในโค้ดใช้รูป log (14) รวม 8 ข้อต่อด้วย (15)) · ความหนาแน่นเกิน 1 ได้ เพราะพื้นที่ต่างหากที่ต้องเป็น 1 · ค่านี้ไปที่ buffer ไม่ไปที่มอเตอร์

2.4 ทำไมต้องสุ่ม — และ explore/exploit ทำงานยังไง

ถ้าไม่สุ่ม จะไม่มี "การลองผิดลองถูก" เลย

สมมติใช้ μ ตรง ๆ ไม่สุ่ม — หุ่นยนต์อยู่ในสถานะ $\mathbf{s}$ หนึ่ง network คาย $\mu = 0.20$ ให้เข้าซ้าย ก็จะสั่ง 0.20 **ทุกครั้งที่เจอสถานะนี้ ตลอดไป**

แล้วมันจะรู้ได้ยังไงว่า 0.40 อาจดีกว่า? **ไม่มีทางรู้ เพราะไม่เคยลอง** ข้อมูล ที่เก็บมาจะมีแต่ "ทำ 0.20 แล้วได้ reward เท่านี้" ซ้ำ ๆ ไม่มีอะไรให้เปรียบเทียบ gradient ก็ไม่รู้ว่าจะควรดัน μ ขึ้นหรือลง

พอใส่ความสุ่มเข้าไป ข้อมูลที่ได้กลายเป็นแบบนี้ (การทดลองความคิด: เจอสถานะเดียวกันสักครั้ง reward เป็นค่าสมมติ — แถวแรกคือช่อง e0 t0 จริง):

ครั้งที่	μ (network คาย)	สุ่มได้ $\mathbf{a}$	reward (สมมติ)
1	0.20	0.40	0.85 ← ดี
2	0.20	-0.10	0.40 ← แย่
3	0.20	0.60	0.91 ← ดีมาก
4	0.20	-0.35	0.22 ← แย่มาก

ตอนนี้มีข้อมูลให้เปรียบเทียบแล้ว — เห็นชัดว่ายิ่งค่าสูงยิ่งดี gradient จึง คำนวณได้ว่าควรดัน μ จาก 0.20 ขึ้นไปทาง 0.5 นี่คือการ "การลองผิดลองถูก" ที่หยุดถึงตั้งแต่หัวข้อ 1.1 — **ความสุ่มคือกลไกที่ทำให้เกิดการลองจริง ๆ**

Remark. ยังมีเหตุผลสองที่มาจากคณิตศาสตร์ของ PPO เอง — สูตรของมันต้องการ ความหนาแน่นของ policy สองตัวกัน ซึ่งทำได้ก็ต่อเมื่อ policy ให้ระบั้งจริง ๆ ไม่ใช่ค่าเดียว เหตุผลนี้จะอธิบายในภาค 3 หลังจากรู้จักคำว่า on-policy แล้ว

การสุ่มทำยังงในโค้ด

actor_critic.py:155-161:

```
def update_distribution(self, observations):
    mean = self.actor(observations) # ← μ จาก network (①)
    self.distribution = Normal(mean, mean * 0.0 + torch.exp(self.logstd)) # ← ③ ประกอบระบั้งจาก ① กับ ②

def act(self, observations, **kwargs):
    self.update_distribution(observations)
    return self.distribution.sample() # ← สุ่มตรงนี้
```

mean * 0.0 + เป็นแค่ trick ให้ σ ขยายขนาดตรงกับ batch เลย ๆ ไม่มี ความหมายทางคณิตศาสตร์ (mean * 0.0 ได้ศูนย์ยั้งทั้งหมด แล้วบวก σ เข้าไป)

ข้างใน .sample() ทำอะไร — กลไกเรียบง่ายมาก: สุ่มจากระบั้งมาตรฐานก่อน แล้ว ยึดและเลื่อน

$$a = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, 1)$$

(16)

สัญลักษณ์	ชนิด	ความหมาย
ϵ (varepsilon)	เวกเตอร์ (8,) — สุ่มใหม่ทุกครั้ง	noise มาตรฐาน — เลขสุ่มจาก $\mathcal{N}(0, 1)$ ที่ทำให้ $a \neq \mu$ · ตัวนี้เท่านั้น ที่ไ้แต่เรียกว่า ϵ (ตัว ϵ อีกแบบเก็บไว้ให้ค่า clip ของ PPO)
$\odot$	ตัวดำเนินการ	คูณทีละตัวตรงตำแหน่ง (Hadamard) — ในโค้ดคือ * · ใน toy 1 มิติก็คือคูณธรรมดา

ขั้น	ทำอะไร	e0 t0
1	สุ่ม ϵ จากระบั้งมาตรฐาน ($\mu = 0, \sigma = 1$) ด้วย RNG (random number generator)	$\epsilon = +0.40$
2	คูณด้วย σ เพื่อยึดความกว้าง	$0.5 \times 0.40 = 0.20$
3	บวก μ เพื่อเลื่อนไปทีศูนย์กลางที่ถูกต้อง	$0.20 + 0.20 = 0.40$

ระบั้งมาตรฐาน $\mathcal{N}(0, 1)$ เป็นตัวเดียวที่ต้องสุ่มจริง ๆ ที่เหลือเป็นแค่ การคูณและบวก — และการสุ่มจาก $\mathcal{N}(0, 1)$ เป็นอัลกอริทึมมาตรฐานที่ RNG ทำได้ (Box-Muller transform หรือคล้ายกัน) · สังเกตว่า $\epsilon = 0.40$ กับ ระบั้งมาตรฐาน $z = 0.40$ ของหัวข้อ 2.3 เป็นเลขเดียวกัน — ไม่บังเอิญ: $z = (a - \mu)/\sigma$ คือการย้อนสมการ (16) กลับไปหา ϵ นั่นเอง

กำหนดนี้ **8 ครั้งพร้อมกัน** (ทีละข้อต่อ ด้วย ϵ คนละตัว) และทำ **ทุก env พร้อมกัน** (2,048 ตัวใน exp_001) เป็น tensor operation ครั้งเดียวนบน GPU

.sample() ทำงานภายใต้ torch.no_grad() (torch/distributions/normal.py:70-73) — ตัวการสุ่มไม่มี gradient ไหลผ่าน gradient ไปเข้าที่ μ ผ่านทาง $\log \pi_\theta(a | s)$ ในสมการ loss แทน ซึ่งเป็นเรื่องของภาค 4

Explore กับ Exploit ไม่ใช่ "สองโหมด" ที่สลับกัน

ตำรา RL ส่วนใหญ่อธิบาย explore/exploit ด้วย **ϵ -greedy** (ของ Q-learning) ซึ่ง เป็นการสลับโหมดจริง ๆ:

ϵ -greedy — สองโหมดแยกกันชัดเจน:
สุ่มเลข r
ถ้า $r < 0.1$ → explore: เลือก action มั่ว ๆ ใดๆ ไม่เลย ← โหมด A
ถ้า $r \geq 0.1$ → exploit: เลือก action ที่ดีที่สุดที่รู้ ← โหมด B

แต่ PPO ที่ใช้ Gaussian policy ไม่ได้ทำแบบนั้นเลย — มันผสมทั้งสองอย่างเข้า ด้วยกัน ในทุก ๆ action ตามสมการ (16):

$$a = \underbrace{\mu}_{\text{exploit: ความรู้ที่ 'ดี' ที่ 'สุดตอนนี้'}} + \underbrace{\sigma \epsilon}_{\text{explore: ลองเบี่ยงจากที่ 'รู้' มี ดหน่อย}}$$

	ϵ -greedy	PPO Gaussian
explore คือ	เลือก action มั่วทั้งตัว	เบี่ยงจาก μ เล็กน้อย
exploit คือ	เลือก action ที่ดีที่สุด	ค่า μ ที่เป็นแกนกลาง
สลับโหมดไหม	สลับ (โหมด A หรือ B)	ไม่สลับ — ทำพร้อมกันทุก step
ปรับความ "กล้าลอง" ยังไง	ลด ϵ ลงเรื่อย ๆ ตามตาราง	σ ค่อย ๆ เล็กลงเอง จากการฝึก

ไม่มีวันที่ไหนที่หุ่นยนต์ "ทำมั่ว ๆ ล้วน ๆ" — มันทำสิ่งที่ตัวเองคิดว่าดี ที่สุด (μ) แล้วเบี่ยงออกไปนิดหน่อยเพื่อดูว่าเบี่ยงแล้วดีขึ้นหรือแย่ลง

ผลที่ตามมา: เห็นหุ่นค่อย ๆ ดีได้ระหว่างเทรน ไม่ต้องรอจบ

คำถามที่เกิดขึ้นตามธรรมชาติคือ "ถ้าตอนเทรนสุ่มตลอด แล้วทำไมถึงเห็นหุ่นค่อย ๆ เดินได้ ไม่ใช่มั่วไปจนจบ?"

คำตอบตามมาจากการสมการ (16) โดยตรง: เพราะ $a = \mu + \sigma\epsilon$ และ μ เก่งขึ้นเรื่อย ๆ ตลอดเวลา — noise ไม่ได้ทำให้พฤติกรรมมั่ว มันแค่ทำให้ "เก่งแบบสั้น ๆ" รอบ ๆ ความเก่งปัจจุบัน

ช่วงการเทรน	μ เป็นยังไง	σ	สิ่งที่เห็นในจอ
iteration 0	weight สุ่ม → คายค่ามั่ว	1.0 ($\ell = 0$)	ล้มทันทีทุกตัว กระตุกไปมา
~100	เริ่มรู้ว่าต้องยืนพื้น	↓	ยืนได้แป๊บนึงแล้วล้ม
~1,000	ยืนได้ เริ่มก้าว	↓	เดินโซเซ ล้มบ้าง
~5,000	เดินได้	↓	เดินค่อนข้างนิ่ง สั้นเล็กน้อย
~15,000	เดินดี	ต่ำ	เดินนิ่ง

สองอย่างดีขึ้นพร้อมกัน: (1) μ ดีขึ้นจาก gradient descent ทุก iteration — ตัวหลัก และ (2) σ เล็กลง เพราะ ℓ ก็เป็นส่วนหนึ่งของ θ ในสมการ (12) เหมือนกัน — พอ policy เก่งแล้ว การสุ่มกว้าง ๆ ทำให้ reward ต่ำลง gradient จึงบีบ ℓ ลง (มี `entropy_coef=0.01` ที่ `limx_rsl_rl_ppo_cfg.py:97` คอยดันไม่ให้ ลดเร็วเกินไป — หัวข้อ 4.6)

ดูของจริงได้ตอนเทรน. σ ไม่ใช่เรื่องนามธรรม — `train.py` พิมพ์ค่านี้ ทุก iteration ในชื่อ Mean action noise std (`on_policy_runner.py:324`) และมันก็กลาง tensorboard ที่ `Policy/mean_noise_std` (`on_policy_runner.py:284`) เปิดกราฟนี้ตอนรับ `exp_001` จะเห็นมันเริ่มที่ ~1.0 แล้วค่อย ๆ ลดลง คู่ไปกับ กราฟ reward ที่สูงขึ้น — เป็นหลักฐานตรง ๆ ว่า explore ค่อย ๆ ลดลงระหว่างที่ exploit ค่อย ๆ ดีขึ้น โดยไม่มีการสลับโหมดที่จุดไหนเลย

สรุปหัวข้อ — สุ่มเพราะไม่สุ่มก็ไม่มีข้อมูลให้เปรียบเทียบ; วิธีสุ่มคือสมการ (16) $a = \mu + \sigma\epsilon$ (e0 t0: $0.20 + 0.5 \times 0.40 = 0.40$); explore กับ exploit อยู่ในสมการเดียวกันทุก step ไม่ใช่สองโหมด; σ เล็กลงเองระหว่างฝึกเพราะ ℓ อยู่ใน θ

2.5 จาก μ ไปถึงมอเตอร์ — ท่อแปลงค่าทั้ง 5 ชั้น

จนถึงตอนนี้เรารู้แล้วว่า network คาย μ ออกมา 8 ตัว แล้วสุ่มได้ a — แต่ a เป็นแค่ตัวเลขไร้หน่วย มันกลายเป็นคำสั่งที่มอเตอร์เข้าใจได้อย่างไร

[1] network คาย μ (8 floats)

↓

[2] สุ่มจาก $N(\mu, \sigma)$ สมการ (16)

↓

[3] คูณ `scale = 0.25`

↓

[4] บวก `default joint position`

↓

[5] PD (proportional-derivative) controller ใน PhysX

↓

มอเตอร์รับขับ

→ เช่น เข้าซ้าย $\mu = 0.20$ ← ไม่มีหน่วย (e0 t0)

→ $a = 0.40$ ← ยังไม่มีหน่วย
(ตอน deploy ข้ามขั้นนี้ ใช้ $a = \mu$)

→ $0.40 \times 0.25 = 0.10$

→ $0.10 + 0.0 = 0.10$ ← เป็นเรเดียนแล้ว ($\approx 5.7^\circ$)

→ $\tau = 45 \cdot (0.10 - q) - 1.5 \cdot \dot{q}$

ค่าจริงทั้งหมดมาจาก `limx_base_env_cfg.py:116-121`:

```
joint_pos = mdp.JointPositionActionCfg(
    asset_name="robot",
    joint_names=[".*"],
    scale=0.25, # ← ชั้น [3] (:119)
    use_default_offset=True, # ← ชั้น [4] (:120)
)
```

และค่า PD gain จาก `solefoot_cfg.py:38-64`:

ข้อต่อ	stiffness (k_p)	damping (k_d)	effort_limit	ที่
legs (abad, hip, knee)	45.0	1.5	80.0 N·m	:48-51
ankles	45.0	0.8	80.0 N·m	:59-62

ชั้น [3] กับ [4] รวมกันเป็นสมการเดียว:

$$q_{target} = 0.25 a + q_{default}$$

(17)

ความหมาย: q_{target} คือ **มุมเป้าหมาย** ของข้อต่อ — action ไร้หน่วย คูณ `scale` แล้วบวกค่าเริ่มต้น · ในโค้ดคือบรรทัดเดียว `raw * scale + offset` (`joint_actions.py:134`) โดย `offset` ถูกตั้งเป็น `default_joint_pos` ที่ :155-156 IWS: `use_default_offset=True`

หมายเหตุ. ค่า default joint position ของ Tron1 SF คือ **0.0 ทุกข้อ** (`joint_pos={"._Joint": 0.0}`, `solefoot_cfg.py:32-34`) ทำให้ชั้น [4] ในตัวอย่างนี้ ดูเหมือนไม่ทำอะไร — แต่ในหุ่นยนต์ที่ default pose ไม่ใช่ศูนย์ ชั้นนี้สำคัญมาก

ทำไมต้องคูณ `scale = 0.25`

network คายค่าออกมาในย่านราว ๆ ± 1 ถึง ± 2 (เพราะ weight เริ่มต้นสุ่ม เล็ก ๆ และ σ เริ่มต้นที่ 1.0) แต่มุมข้อต่อที่ควรรับได้ใน **หนึ่ง step (0.02 วินาที)** อยู่ที่ราว ๆ ไม่กี่องศาเท่านั้น:

	ไม่คูณ scale	คูณ scale 0.25
$a = 1.0$	1.0 rad = 57 องศา ใน 0.02 วินาที	0.25 rad = 14 องศา
$a = 2.0$	2.0 rad = 115 องศา ใน 0.02 วินาที	0.5 rad = 29 องศา

57 องศาใน 1/50 วินาทีคือการกระตุกรุนแรงมาก — PD จะสร้างแรงบิดมหาศาลจนชนลิมิต 80 N·m ทันที หุ่นยนต์กระเด็นล้มทุกครั้ง และ **ฝึกไม่ได้เลย เพราะทุก action พัง แท้ทั้งหมด** ไม่มีสัญญาณให้เรียนรู้

มองอีกมุมคือ **การแบ่งงาน**: ถ้าไม่มี scale network ต้องเรียนรู้เองว่า "ต้องคาย เลขเล็ก ๆ ประมาณ 0.1 นะ" ซึ่งทำได้ แต่เสียเวลาฝึกเพิ่มเพื่อบีบ weight ให้เล็กลง การมี scale เท่ากับบอก network ล่วงหน้าว่า "คายเลขในย่าน ± 1 ตามธรรมชาติได้เลย เดี่ยวแปลงหน่วยให้เอง" — network จึงโฟกัสที่การเรียนรู้ *รูปแบบ* การเดิน ไม่ต้อง เสียเวลาเรียนรู้ *สเกลตัวเลข* (0.25 เป็น hyperparameter เหมือน 512/256/128 และเป็นค่ามาตรฐานที่ใช้กันแพร่หลายในงาน legged locomotion)

ตัวเลขไร้หน่วย "กลายเป็นเรเดียน" ได้ยังไง

คำตอบที่อาจฟังดูแปลก: **ไม่มีการแปลงหน่วยเกิดขึ้นเลย** — มันเป็นเรื่องของการ ตีความล้วน ๆ

เปรียบเทียบกับกรอ่านเซนเซอร์: ADC (analog-to-digital converter) ให้ตัวเลข 3.2 ออกมา ตัวเลขนี้ไม่รู้ว่าเป็นอะไร แล้วเราเขียนโค้ด `float temp_celsius = adc_value * 25.0;` **การคูณ 25 ไม่ได้แปลงอะไรในเชิงกายภาพ** — มันเป็นเรื่องที่ตัดสินใจว่า "เลขจาก ADC คูณ 25 แล้วจะเรียกว่า องศาเซลเซียส" แล้วเอาไปใส่ตัวแปรที่ชื่อแบบนั้น **80.0 กลายเป็นองศาเซลเซียสเพราะปลายทางที่มันถูกส่งไป ไม่ใช่เพราะการคูณ**

ใน Tron1 ก็เหมือนกันเป๊ะ ใส่ตามโค้ดจริง:

บรรทัด	โค้ด	ค่า	สถานะของตัวเลข
1	<code>actions = policy(obs)</code>	0.40	เลขเปล่า ไม่มีหน่วย
2	<code>processed = raw * scale + offset</code> (<code>joint_actions.py:134</code>) — สมการ (17)	0.10	ยังเป็นเลขเปล่า — Python ไม่รู้จักคำว่าเรเดียน
3	<code>set_joint_position_target(processed)</code> (<code>joint_actions.py:160</code>)	0.10	← ตรงนี้กลายเป็นเรเดียน
4	PhysX: $\tau = 45(0.10 - q) - 1.5\dot{q}$	—	ฟิสิกส์ใช้งานจริง

ฟังก์ชัน `set_joint_position_target()` ถูกนิยามโดย Isaac Lab/PhysX ว่า "ช่อง นี้รับค่าเป็นเรเดียน" พอส่ง 0.10 เข้าไปในช่องนั้น มันก็เป็น 0.10 เรเดียน (≈ 5.7 องศา) โดยปริยาย — เลขกลายเป็นเรเดียนเพราะ "ช่อง" ที่มันถูก เสียบเข้าไป ไม่ใช่เพราะมีสูตรแปลงพิเศษ

จุดที่ลึกที่สุด: **network ไม่เคยรู้จักคำว่า "เรเดียน" เลย**. ลองคิดดู — ถ้าแก้ `scale` จาก 0.25 เป็น 0.5 แล้วฝึกใหม่ จะเกิดอะไรขึ้น? คำตอบคือ **network จะเรียนรู้ที่จะคายเลขเล็กลงครั้งหนึ่งเอง** (เดิมคาย 0.20 จะกลายเป็น ~0.10) เพราะสุดท้ายมันจะทำให้เดินได้ดีคือค่าเดิม

network ไม่ได้เรียนรู้ว่า "เรเดียนคืออะไร" — มันเรียนรู้แค่ว่า "คายเลขแบบไหน แล้ว reward สูง" ท่อแปลงค่าทั้งหมดเป็นสิ่งที่**คงที่ตลอดการฝึก** network จึง ค่อย ๆ ปรับตัวให้เข้ากับท่อนี้เองผ่าน gradient หน่วยเป็น**เรื่องของมนุษย์ที่ ออกแบบระบบ ไม่ใช่เรื่องของ network**

จุดสำคัญ: **policy ไม่เคยสั่ง torque โดยตรง**

ชั้น [5] คือ **PD controller** ที่ทำงานอยู่ **นอก** neural network:

$$\tau = k_p (q_{target} - q_{current}) - k_d \dot{q}_{current}$$

(18)

สัญลักษณ์	ชนิด	ความหมาย	ค่า Tron1
τ	scalar ต่อข้อต่อ	แรงบิดที่มอเตอร์ออก (N·m)	ถูกตัดที่ ±80 (effort_limit)
q_{target}	scalar ต่อข้อต่อ	มุมเป้าหมายจากสมการ (17)	0.10 rad ในตัวอย่าง
$q, \dot{q}$	scalar ต่อข้อต่อ	มุมและความเร็วเชิงมุมที่วัดได้ตอนนี้	จากฮิม/เซนเซอร์
k_p, k_d	ค่าคงที่	stiffness / damping	45.0 / 1.5 (ขา), 45.0 / 0.8 (ข้อเท้า)

ความหมาย: ยิ่งห่างจากมุมเป้าหมายมาก ยิ่งออกแรงมาก (พจน์ k_p) แต่ถ้ากำลัง ขยับเร็วอยู่ ให้หน่วงไว้ (พจน์ k_d) — เป็นตัวควบคุมคลาสสิกที่ไม่มีอะไรเกี่ยวกับ RL เลย

	PD ตัวนี้รันอยู่ที่ไหน
ในฮิม	ImplicitActuatorCfg ข้างใน PhysX ที่ 200 Hz (4 รอบย่อยต่อ policy step ตาม <code>decimation=4</code> , <code>limx_base_env_cfg.py:497</code>)
บนหุ่นยนต์จริง	motor driver board ที่ความถี่สูง — policy แคล่ง q_{target} ไปให้ผ่าน SDK (software development kit ของ LimX)

นี่คือเหตุผลที่ 03-sf-task-anatomy.md เขียนว่า *"The policy never emits torque directly, which is what makes the ONNX portable to hardware"* — ถ้า policy สั่ง torque ตรง ๆ มันจะต้องรันที่ 200 Hz+ และต้องรู้พลศาสตร์ของมอเตอร์แต่ละตัว แต่การ **สั่งเป็นตำแหน่งเป้าหมาย**ทำให้ policy รันแค่ 50 Hz ก็พอ แล้วปล่อยให้ PD ความถี่สูงบนบอร์ดจัดการรายละเอียดแทน

สรุปหัวข้อ — a ไร้หน่วย → สมการ (17) ให้มุมเป้าหมาย q_{target} (e0 t0: 0.40 → 0.10 rad) → สมการ (18) ให้แรงบิด; scale 0.25 กันการกระตุก; "เรเดียน" เกิดจากช่องที่เสียบ ไม่ใช่การคูณ; policy ไม่เคยสั่ง torque

2.6 `act` กับ `act_inference` — ไม่มี "transition" ระหว่างสองโหมด

ชั้น [2] ในท่อข้างบนบอกว่า "ตอน deploy ข้ามขั้นนี้" — คำถามที่ตามมาคือ **แล้ว มันสลับตอนไหน?** คำตอบคือ **ไม่มีการสลับเลย เพราะมันคือคนละโปรแกรมที่รันคนละ เวลา**

09:00 \$ python train.py --task Isaac-Limx-SF-Blind-Flat-v0

โปรแกรมที่ 1: train.py

วนลูป 15,000 iterations

ใช้ act() ← สุ่มทุกครั้ง ตลอดหลายชั่วโมง

ปรับ weight ทุก iteration

15:00 บันทึก logs/.../model_15000.pt แล้วโปรแกรมจบ

← มีแค่ "ไฟล์ weight" ที่ข้ามช่วงนี้ไป

ไม่มีโปรแกรมไหนรับอยู่เลย

15:30 \$ python play.py --task Isaac-Limx-SF-Blind-Flat-Play-v0

โปรแกรมที่ 2: play.py

โหลด model_15000.pt เข้ามา

ใช้ act_inference() ← ไม่สุ่ม

ไม่ปรับ weight เลย (แค่เอามาใช้)

ดูจากโค้ดจะชัดที่สุด — สองฟังก์ชันนี้อยู่ในคลาสเดียวกัน ใช้ `self.actor` **ก้อนเดียวกัน weight ชุดเดียวกัน** ([actor_critic.py:159-168](#)):

```
def act(self, observations, **kwargs):          # train.py เรียกใช้
    self.update_distribution(observations)      # สร้าง N(μ, σ) โดย μ = self.actor(obs)
    return self.distribution.sample()           # สุ่มออกมา

def act_inference(self, observations):          # play.py เรียกใช้
    actions_mean = self.actor(observations)     # เรียก self.actor(obs) ตรง ๆ
    return actions_mean                        # คืนค่าเลย ไม่สุ่ม
```

$$\text{act}(s) = \underbrace{\text{actor}(s)}_{\mu} + \sigma \epsilon$$

$$\text{act_inference}(s) = \underbrace{\text{actor}(s)}_{\mu}$$

act_inference ก็คือ **act** ที่ตัดขั้นสุ่มออก — ไม่มีการ "เปลี่ยนโหมด" ไม่มี flag ที่ต้องตั้ง แค่คนละโปรแกรมเรียกคนละฟังก์ชัน · ในภาษาของหัวข้อ 2.1: ตอนเทรน ใช้ ①+②+③ ตอน deploy ใช้ ① อย่างเดียว

เทียบกับนักดนตรี:

	ซ้อม (train.py → act)	แสดงจริง (play.py → act_inference)
ใช้ทักษะชุดไหน	ทักษะที่มี ณ ตอนนั้น	ทักษะชุดเดียวกัน
ทำอะไรเพิ่ม	ลองเล่นเพี้ยนไปนิดหน่อย เพื่อดูว่าแบบไหนเพราะกว่า	เล่นเวอร์ชันที่ดีที่สุดที่รู้ ไม่ทดลอง
ทักษะเปลี่ยนไหม	เปลี่ยน (เก่งขึ้นเรื่อย ๆ)	ไม่เปลี่ยน (ใช้อย่างเดียว)

วงจรชีวิตทั้งหมด รวมถึงหุ่นยนต์จริง

```
[1] train.py      ใช้ act() (สุ่ม)          → ได้ model_15000.pt
                                     |
[2] play.py      ใช้ act_inference() (ไม่สุ่ม) ← โหลดไฟล์นี้
                                     |
                                     |   ดูผลในซิม + export
                                     |   ได้ policy.onnx
                                     |
[3] tron1-rl-deploy รันบนหุ่นยนต์จริง ไม่สุ่มเช่นกัน ← โหลดไฟล์นี้
```

ไฟล์ `policy.onnx` ที่ export ไปลงหุ่นยนต์จริง **ussชุด actor network เท่านั้น** — ไม่มี σ ไม่มี critic ไม่มีตัวสุ่ม เพราะบนหุ่นยนต์จริงไม่ต้องเรียนรู้อะไร · แต่หุ่นยังต้องโหลด `encoder.onnx` อีกไฟล์ (`play.py:117-122`) เพราะ 3 ใน 42 ช่อง input ของ actor คือ $\hat{v}$ (หัวข้อ 4.4) — ตรงกับกล่อง

ENCODER ในรูป **D** อีกแล้ว มีแต่การเอา μ ไปใช้ · รูป **D** [diagrams/tron1-deploy.html](#) วาดสายส่งค่าตอน deploy ทั้งเส้น ตั้งแต่เซนเซอร์จนถึง PD บบนบอร์ด

Remark. ตรงนี้เชื่อมกับที่ 03-sf-task-anatomy.md บันทึกว่า critic ถูก ทั้งตอน deploy — σ ก็ถูกทั้งด้วยเหตุผลเดียวกัน ทั้งคู่เป็น "เครื่องมือสำหรับการเรียนรู้" ไม่ใช่ "ส่วนหนึ่งของการควบคุม"

ผลข้างเคียงที่สังเกตได้: หุ่นใน play.py จะเดินนิ่งกว่าที่เห็นตอนเทรนเสมอ เพราะตัดความสั่นจากการสุ่มออกไปแล้ว

สรุปหัวข้อ — `act` = actor + สุ่ม, `act_inference` = actor อย่างเดียว; ไม่มีโหมดให้สลับ มีแต่ สองโปรแกรมที่รันคนละเวลาโดยส่งไฟล์ `weight` ให้กัน; `policy.onnx` มีแต่ชิ้น ① (encoder เป็นอีกไฟล์)

จบภาค 2 แล้วเรารู้อะไรบ้าง: policy π_θ เป็นฟังก์ชันที่รับ s ค็นะขังหนึ่งใบ (สมการ (9)) ประกอบจาก NN ที่คาย μ 8 ตัว + `logstd` + สูตร $\mathcal{N}$; action จริง คือการสุ่มรอบ ๆ μ ด้วยสมการ (16); ความหนาแน่นของ action ที่สุ่มได้คำนวณจาก (13)–(15) และไปที่ buffer ไม่ไปทิ่มมอเตอร์; ตัวเลขนั้นเดินทางผ่านท่อ 5 ชั้น (17)(18) จนกลายเป็นแรงบิด; และ `weight` ทุกตัวถูกปรับด้วยสมการ (12) **สิ่งที่ยังไม่รู้คือ loss L ในสมการ (12) หน้าตาเป็นยังไง** — ต้องรู้ก่อนว่าทำไมวิธีตรงไปตรงมาใช้ไม่ได้ (ภาค 3) แล้วค่อยประกอบ PPO ขึ้นมา (ภาค 4)

ภาค 3 — ทางตัน: ทำไมวิธี RL คลาสสิกใช้กับ Tron1 ไม่ได้

ทุกวิธีในภาคนี้พยายามแก้สมการ (2) เหมือนกันหมด — แต่ละอันล้มเหลวด้วยเหตุผล คนละแบบ และความล้มเหลวเหล่านี้แหละที่ชี้ทางไปสู่ PPO ในภาค 4 · สามหัวข้อคือสาม คำถาม: เก็บความรู้ไว้ที่ไหน (3.1) · เรียนจากผลจริงหรือจากคำทำนาย (3.2) · ข้อมูลที่ เก็บมาแล้วใช้ได้นานแค่ไหน (3.3)

3.1 tabular methods (Q-learning ตั้งแต่เริ่ม, Dynamic Programming)

วิธี RL ยุคแรก ๆ แก้สมการ Bellman (4) ได้ตรง ๆ โดยสร้าง ตาราง เก็บค่า $V(s)$ หรือ $Q(s, a)$ ไว้ทุกคู่ state-action แล้วอัปเดตค่าซ้ำ ๆ จนกว่าตัวเลขจะนิ่ง (converge) — ตรงนี้เรียกว่า **tabular methods** (วิธีแบบตาราง) และเมื่อรู้แบบจำลอง ของโลก $P(s' | s, a)$ ครบ ก็ทำได้ด้วย **dynamic programming (DP)**

ปัญหาคือ ตารางต้องมี **แถวจำกัด** — ใช้ได้ดีเมื่อ state นับจำนวนได้ เช่น grid world ที่มีแค่ 100 ช่อง แต่ state ของ Tron1 คือเวกเตอร์ของเลขทศนิยม (base_ang_vel เป็น float 3 ค่า, joint_pos เป็น float 8 ค่า ฯลฯ) — เอียง 0.001 องศา กับ 0.002 องศา ถือเป็นคนละ state ทางคณิตศาสตร์ ถ้าจะทำตาราง ต้องมีจำนวนแถวเป็นอนันต์ ทำไม่ได้จริง

Remark. ปัญหานี้เรียกว่า **continuous state space** (สถานะต่อเนื่อง) ตรงข้ามกับ **discrete state space** (สถานะนับได้ เช่น ตำแหน่งบนกระดาน หมากรุก) Tron1 มีทั้ง continuous state และ continuous action space (offset มุมข้อต่อก็เป็น float เช่นกัน) — เป็นโจทย์ที่หนักที่สุดในสองมิตินี้ พร้อมกัน

ทางแก้คือแทนที่ตารางด้วยฟังก์ชันประมาณค่า (function approximator) — ให้ neural network รับ state เป็น input แล้วทำนาย $V(s)$ หรือ action ที่ดีที่สุด ออกมาแทน โครงข่ายหนึ่งตัวครอบคลุม state ทุกค่าได้ (ไม่ใช่แค่ค่าที่เคยเจอ) เพราะมันเรียนรู้ **ความสัมพันธ์** ไม่ใช่ท่องจำเป็นแถว ๆ — นี่คือจุดเริ่มต้นของ **deep RL** และเป็นเหตุผลที่ policy ของ Tron1 คือโครงข่าย $42 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 8$ (ภาค 2) ไม่ใช่ตาราง

Remark — ใน repo ไม่มีตารางสถานะเลย. สิ่งเดียวที่หน้าตาเหมือน "ตาราง" คือ RolloutStorage (rollout_storage.py) ซึ่งเก็บ transition ที่เพิ่งเกิด (ตาราง 2×4 ของเราคือมันฉบับย่อ) ไม่ใช่ตารางค่า V ต่อสถานะ — มันถูกล้างทิ้งทุก iteration (storage.clear() ppo.py:331) ความรู้ทั้งหมดอยู่ใน weight ของ NN สองก้อน

สรุปหัวข้อ — tabular methods ใช้ไม่ได้ เพราะต้องการพื้นที่ state-action ที่นับจำนวนได้จำกัด แต่ปัญหาหุ่นยนต์เป็น continuous — ต้องเปลี่ยนจากตารางเป็น neural network เพื่อ "สรุปความรู้" (generalize)ข้ามค่าที่ไม่เคยเห็นมาก่อน

3.2 Monte Carlo กับ TD — สามตัวเลขสำหรับสถานะเดียวกัน

ดูสถานะแรกของ env 1 ในตาราง 2×4 มีตัวเลขสามตัวที่ต่างกันอ้างว่าเป็น "ค่าของ s_0 ":

ตัวเลข	มาจากไหน	ค่า	ชนิดของหลักฐาน
ผลจริงทั้ง episode G_0	บวก reward ที่เกิดขึ้นจริง 3 step จนล้ม (หัวข้อ 1.7)	$0.5 + 0.9(1.0) + 0.81(-2.0) = -0.22$	ข้อเท็จจริง — แต่ต้องรอจนล้ม
ก้าวเดียวแล้วเดา	reward ก้าวแรก + $\gamma \times$ คำทำนายของ critic ที่ s_1	$0.5 + 0.9(2.0) = 2.3$	ผสม — รู้ทันทีหลังก้าวแรก
คำทำนายของ critic $V(s_0)$	สิ่งที่ critic พุดก่อนก้าว	3.0	ความเห็น ล้วน ๆ

สามตัวนี้ห่างกันมาก ($-0.22 / 2.3 / 3.0$) — ความต่างของมันคือทั้งหัวข้อนี้: จะเรียนรู้ จากตัวไหนดี

Monte Carlo — เชื้อผลจริง

สมมติเราไม่มีสมการ Bellman เลย แล้วอยากรู้ $V(s)$ วิธีตรงไปตรงมาที่สุดคือ: **ปล่อยให้ episode เล่นจนจบ แล้วนับผลรวม reward จริงที่เกิดขึ้น** जाएเป็น ค่าประมาณของ $V(s)$ — เรียกวิธีนี้ว่า **Monte Carlo (MC) estimation** (ตัวเลข -0.22 ในตารางข้างบน)

ปัญหาของ MC สองข้อ:

- ต้องรอ episode จบก่อนถึงจะเรียนรู้ได้อะไรเลย** — Tron1 หนึ่ง episode ยาว ถึง 1000 policy steps (20 วินาทีที่ 50Hz ตามหัวข้อ 1.4) ถ้าต้องรอจนจบ ทุกครั้งก่อนอัปเดต จะช้ามาก · ในตาราง 2×4 env 0 ยังไม่จบเลยตอนตารางเต็ม — MC ให้ค่าอะไรกับ env 0 ไม่ได้

2. **variance สูง** — ผลรวม reward ตลอด 1000 step ขึ้นอยู่กับความสุ่มของ *ทุก* action ตลอดทาง (การสุ่มจาก policy, การสุ่ม domain randomization ฯลฯ) ทำให้ตัวเลขที่ได้แกว่งไปมาเยอะ แม้จะเริ่มจาก state เดียวกันสองครั้ง ก็อาจได้ผลรวมต่างกันลิบลับ — โมเดลเรียนรู้ได้ช้า เพราะ "สัญญาณ" จมอยู่ใน "สัญญาณรบกวน"

TD — ก้าวเดียวแล้วเชื่อคำทำนาย

ทางแก้คือ **bootstrapping** ที่เห็นในสมการ Bellman (4) แล้ว — แทนที่จะรอ ผลจริงทั้ง episode เราใช้ **ค่าประมาณ ณ ตอนนี้** ของ state ถัดไปแทน:

$$V(s_t) \approx r_t + \gamma V(s_{t+1})$$

วิธีนี้เรียกว่า **temporal difference (TD) learning** — เรียนรู้ได้ทันทีจาก แค่หนึ่ง step ไม่ต้องรอจบ episode และ variance ต่ำกว่ามาก (เพราะ พังพาความสุ่ม แค่ 1 step ไม่ใช่ทั้ง 1000 step) แต่ก็มีข้อเสียของมันเอง: เพราะ $V(s_{t+1})$ เป็นแค่ค่า *ประมาณ* (อาจยังไม่แม่นยำ) การใช้มันมาคำนวณต่อ จึงมี **bias** (อคติ) ติดมาด้วย (ตัวเลข 2.3 ข้างบน — ถ้า critic ที่ s_1 ผิด ตัวเลขนี้ก็ผิดตาม)

TD error — วัฏว่า critic พลาดเท่าไร

สมการ Bellman (4) บอกว่า ถ้า *critic* *แม่นยำ* ข้างซ้ายเท่าข้างขวา · **ส่วนต่าง** ระหว่างสองข้างจึงวัฏว่า critic เดาพลาดเท่าไร — ตัวเลขนี้มีชื่อว่า **TD error** และมีข้อยกเว้นหนึ่งอย่างที่ต้องใส่เข้าไปในสูตรตั้งแต่ตอนนี้: ถ้า episode จบที่ช่องนี้ (done = 1) ไม่มี s_{t+1} ที่มีความหมาย — env ถูก reset ไปแล้ว — จึงต้องตัดพจน์ bootstrap ทิ้ง

สัญลักษณ์	ชนิด	ความหมาย	e1 t2
δ_t	scalar ต่อช่อง	ผลจริงก้าวนี้ + $\gamma \times$ ทำนายพรุ่งนี้ – ทำนายวันนี้	-3.00
r_t	scalar	reward ของช่องนี้	-2.0
$done_t$	scalar ต่อช่อง (0 หรือ 1)	episode จบที่ช่องนี้ไหม (หัวข้อ 1.4)	1
$(1 - done_t)$	scalar ต่อช่อง (0 หรือ 1)	mask — 1 = ยังมีอนาคต, 0 = ตัดอนาคตทิ้ง	0
$V(s_{t+1})$	scalar	คำทำนายของ critic ที่ช่องถัดไป — ที่ขอบตาราง (t = 3) ใช้ V_{last} ที่ critic ปรึะเป็น state หลังช่องสุดท้าย (ppo.py:176-178)	4.0 (แต่ถูก mask)

$$\delta_t = r_t + \underbrace{(1 - done_t)}_{mask} \gamma V(s_{t+1}) - V(s_t)$$

(19)

ความหมาย: $\delta_t > 0$ = ก้าวนี้ดีกว่าที่ critic คาด · $\delta_t < 0$ = แย่กว่าที่คาด · $\delta_t = 0$ = critic ทายถูกเป๊ะ · ในโค้ดคือ `delta` ที่ `rollout_storage.py:195-199` โดย `mask` ชื่อ `next_is_not_terminal (:194)` และ V_{last} ชื่อ `last_values (:190-191)`

ไล่ทั้ง 8 ช่องจากท้ายตารางขึ้นมา (ได้ควมย้อนหลังด้วยเหตุผลที่จะเห็นในหัวข้อ 4.5):

ช่อง	r_t	mask	$\gamma \cdot V_{t+1}$	V_t	δ_t
e0 t3	0.5	1	$0.9 \times 3.0 (=V_{last}) = 2.70$	3.5	$0.5 + 2.70 - 3.5 = -0.30$
e0 t2	1.0	1	$0.9 \times 3.5 = 3.15$	4.0	+0.15
e0 t1	0.5	1	$0.9 \times 4.0 = 3.60$	4.5	-0.40
e0 t0	1.0	1	$0.9 \times 4.5 = 4.05$	5.0	+0.05
e1 t3	0.0	1	$0.9 \times 3.5 (=V_{last}) = 3.15$	4.0	-0.85
e1 t2	-2.0	0	0 $\times 0.9 \times 4.0 = 0$	1.0	$-2.0 + 0 - 1.0 = -3.00$
e1 t1	1.0	1	$0.9 \times 1.0 = 0.90$	2.0	-0.10
e1 t0	0.5	1	$0.9 \times 2.0 = 1.80$	3.0	-0.70

ดูแถว e1 t2: ถ้าไม่มี mask จะได้ $\delta = -2.0 + 3.6 - 1.0 = +0.6$ — บอกว่า "ลื้ม แล้วดีกว่าที่คาด" ซึ่งผิดสิ้นเชิง เพราะ 3.6 นั่นคือ V ของ *ก้าวขึ้นใหม่หลัง reset* (env 1 เริ่ม episode ใหม่ที่ t3) ไม่ใช่อนาคตของท่าที่กำลังลื้ม mask ทำให้ได้ -3.0 ที่ถูกต้อง: ลื้มคือแย่มาก · ส่วนแถว e0 t0 ให้ $\delta = +0.05$ = ผิดต่าง 5.05 – 5.0 ที่เจอตอน ตรวจ Bellman ในหัวข้อ 1.6 พอดี — (19) คือ "ข้างขวาลบข้างซ้าย" ของ (4) นั่นเอง

Remark — done มีสองสาเหตุ และได้ปฏิบัติต่างกัน. ลื้ม (`base_contact`) ตัดอนาคต จริง จึง `mask` · แต่ *หมดเวลา* (`time_out`, `limx_base_env_cfg.py:450`) อนาคตยังมี แต่เรา หยุดดู — `ppo.py:163-168` จึงบวก $\gamma V(s_t)$ กลับเข้าไปใน `reward` ก่อนเก็บ เพื่อชดเชย `mask` ที่กำลังจะตัด · ตาราง 2×4 มีแต่กรณีลื้ม รายละเอียดของกรณีหมดเวลาอยู่หัวข้อ 4.5

bias กับ variance — ตารางที่ต้องจำ

วิธี	ต้องรอ episode จบไหม	Variance	Bias
Monte Carlo	ต้องรอ	สูง	ไม่มี (ใช้ผลจริง)
TD (1 step)	ไม่ต้องรอ	ต่ำ	มี (พึ่งค่าประมาณ)

นี่คือ **trade-off** ระหว่าง **bias** กับ **variance** ที่เป็นหัวใจของ RL เซิงปฏิบัติ เกือบทั้งหมด — **จำตารางนี้ไว้ให้ดี เพราะ GAE (หัวข้อ 4.5) ถูกออกแบบมาเพื่อ "เลือกจุดกึ่งกลาง" ระหว่างสองขั้วนี้โดยตรง** แทนที่จะเลือกสุดโต่งฝั่งใดฝั่งหนึ่ง — และวัตถุดิบของมันคือ δ_t ทั้ง 8 ช่องในตารางข้างบนนี้แหละ

สรุปหัวข้อ — MC เชื่อผลจริง (ช้า แกว่ง) TD เชื่อคำทำนาย (เร็ว นิ่ง แต่เอนเอียง); ส่วนต่างระหว่าง Bellman สองข้างคือ TD error δ_t สมการ (19) ที่มี mask ตัดอนาคตเมื่อ episode จบ ($e1\ t2 = -3.00$)

3.3 on-policy กับ off-policy — ข้อมูลที่เก็บมาแล้ว ใช้ได้นานแค่ไหน

ตาราง 2×4 ทั้งในถูกเก็บด้วย policy ที่มี μ ชุดเดิม (μ_{old} : 0.20 ที่ $e0\ t0$) ที่นี้สมมติเราปรับ weight ไปหนึ่งก้าว μ ที่ $e0\ t0$ กลายเป็น 0.25 — **8 ช่องนี้ยังเป็น "ตัวแทน" ของ policy ใหม่อยู่ไหม?** action 0.40 ที่สุ่มมาตอน $\mu = 0.20$ policy ใหม่ อาจไม่สุ่มออกมาบ่อยเท่าเดิม ค่าเฉลี่ยใด ๆ ที่คำนวณจาก 8 ช่องนี้จึงเป็นค่าเฉลี่ย *ของ policy เก่า* ไม่ใช่ของตัวที่เรากำลังจะปรับ

สองครอบครัวของอัลกอริทึม

	on-policy	off-policy
ข้อมูลที่ใช้เรียน	ต้องมาจาก policy ตัวที่กำลังปรับ เท่านั้น	มาจาก policy อื่น (ตัวเก่า, ตัวสุ่ม, มนุษย์) ก็ได้
หลังปรับ weight	ข้อมูลเก่าใช้ต่อไม่ได้ ต้องเก็บใหม่	ข้อมูลเก่าเก็บไว้ใช้ซ้ำได้ (replay buffer)
ตัวอย่าง	PPO, TRPO (trust region policy optimization), REINFORCE	Q-learning, SAC, DDPG
ใช้ข้อมูลค้มนไหม	ไม่ค้มน — ทิ้งบ่อย	ค้มน — แต่ต้องแก้ปัญหาค้มนข้อมูลไม่ตรงกับ policy

PPO อยู่ครอบครัว on-policy: คลาสที่รับการเรนชื่อ OnPolicyRunner (`on_policy_runner.py:46`) และ**ตารางถูกล้างทิ้งทุก iteration** (`self.storage.clear()` `ppo.py:331`) — ทุก iteration เก็บ 8 ช่อง (Tron1: 49,152 ช่อง) ใหม่ด้วย policy ล่าสุด

แต่มีช่องโหว่หนึ่งอย่าง: **ภายใน iteration เดียว** PPO ปรับ weight หลายครั้ง (toy: 4 gradient step, Tron1: 20 — หัวข้อ 4.7) บนตารางใบเดิม · หลัง gradient step แรก policy ก็ไม่ใช่ตัวที่เก็บข้อมูลแล้ว — ถ้ายึดกฎ on-policy อย่างเคร่งครัด step ที่ 2, 3, 4 จะใช้ ข้อมูลนี้ไม่ได้ ต้องมีเครื่องมือทางคณิตศาสตร์ที่ "แก้" ข้อมูลของ policy เก่าให้เป็น ตัวแทนของ policy ใหม่ได้ — เครื่องมือนั้นคือ importance sampling

importance sampling — ค่าเฉลี่ยได้ระดัั้งใหม่ จากตัวอย่างของระดัั้งเก่า

โจทก์ทั่วไป: อยากรู้ค่าเฉลี่ยของฟังก์ชัน $f(a)$ เมื่อ a ถูกสุ่มจากระดัั้ง**ใหม่** แต่ ตัวอย่างที่มีในมือถูกสุ่มจากระดัั้ง**เก่า**

สัญลักษณ์	ชนิด	ความหมาย	$e0\ t0$
$\pi_{old}(a s)$	scalar	ความหนาแน่นของระดัั้งที่ เก็บข้อมูล ณ a (สมการ (13))	$\mu_{old} = 0.20$: 0.7365
$\pi_{new}(a s)$	scalar	ความหนาแน่นของระดัั้งที่ เพิ่งปรับ ณ a ตัวเดิม	$\mu_{new} = 0.25$: $e^{-0.2708} = 0.7627$
$f(a)$	ฟังก์ชัน	อะไรก็ได้ที่อยากหาค่าเฉลี่ย — ในภาค 4 คือ advantage	—
$\mathbb{E}_{a \sim \pi_{old}}[\cdot]$	ตัวดำเนินการ	ค่าเฉลี่ยเมื่อ a สุ่มจากระดัั้ง เก่า = สิ่งที 8 ช่องประมาณได้	—

ขั้นตอน	เหตุผล
$\mathbb{E}_{a \sim \pi_{new}}[f(a)] = \int \pi_{new}(a s) f(a) da$	นิยามค่าเฉลี่ยได้ระดัั้งใหม่ — คำนวณตรง ๆ ไม่ได้ เพราะไม่มีตัวอย่างจาก π_{new}
$= \int \pi_{old}(a s) \frac{\pi_{new}(a s)}{\pi_{old}(a s)} f(a) da$	คูณและหารด้วย π_{old} ตัวเดียวกัน — ค่าไม่เปลี่ยน (ต้องมี $\pi_{old} > 0$ ทุกที่ที่ $\pi_{new} > 0$ ซึ่ง Gaussian เป็นจริงเสมอ)
$= \mathbb{E}_{a \sim \pi_{old}} \left[\frac{\pi_{new}(a s)}{\pi_{old}(a s)} f(a) \right]$	อ่านกลับเป็นค่าเฉลี่ยได้ระดัั้ง เก่า — ตัวที 8 ช่องประมาณได้

$$\mathbb{E}_{a \sim \pi_{new}}[f(a)] = \mathbb{E}_{a \sim \pi_{old}} \left[\frac{\pi_{new}(a | s)}{\pi_{old}(a | s)} f(a) \right]$$

(20)

ความหมาย: ตัวอย่างของ policy เก่ายังใช้ได้ **ถ้าถ่วงน้ำหนักแต่ละตัวอย่างด้วยอัตราส่วน ความหนาแน่น** — ตัวอย่างที่ policy ใหม่ชอบมากกว่าเดิมได้ น้ำหนักเกิน 1 ตัวอย่างที่ ชอบน้อยลงได้ น้ำหนักต่ำกว่า 1 · นี่คือ **importance sampling** (การสุ่มตามความสำคัญ; Sutton & Barto §5.5 ใช้มันแบบ

คูณกันทั้ง episode สำหรับ MC — PPO ใช้แบบต่อช่อง)

อัตราส่วนตัวนั้นสำคัญพอที่จะมีชื่อของตัวเอง:

$$\rho_t = \frac{\pi_{new}(a_t | s_t)}{\pi_{old}(a_t | s_t)} = \exp(\log \pi_{new}(a_t | s_t) - \log \pi_{old}(a_t | s_t))$$

(21)

ความหมาย: ρ_t (ρ) = "policy ใหม่ชอบ action ตัวเดิมในช่องนี้ มากขึ้นกี่เท่า" · ทั้งเศษและส่วนถูกคำนวณที่ a_t ตัวเดียวกันจาก buffer — ต่างกันแค่ ระบุว่า (ตัวเก่า vs ตัวใหม่) · รูปขวาคือสิ่งที่โค้ดทำ เพราะเก็บและคำนวณทุกอย่างใน log (สมการ (14))

ρ_t	อ่านว่า
= 1	policy ยังไม่ขยับ (gradient step แรกของทุก iteration เป็นแบบนี้ทุกช่อง)
> 1	policy ใหม่ให้ความหนาแน่นกับ a_t มากกว่าตอนเก็บ
< 1	น้อยกว่าตอนเก็บ

ρ ในโค้ด — เก็บตอนหนึ่ง ถ้ามั่วอีกตอนหนึ่ง

ตอนไหน	ทำอะไร	โค้ด	e0 t0
เก็บข้อมูล (ทุก env step)	หลังสุ่ม a ถ้ามั่วจะบันทึกว่า $\log \pi(a s)$ เท่าไร แล้ว <code>.detach()</code> เก็บลง buffer เป็น $\log \pi_{old}$	<code>ppo.py:147-149</code>	-0.3058
อัปเดต (ทุก gradient step)	สร้างระว่างใหม่จาก θ ปัจจุบัน (<code>act()</code>) ที่ :205-210 — ผลสุ่มถูกโยนทิ้ง) แล้วถ้ามั่ว $\log \pi$ ของ a ตัวเดิมจาก buffer (<code>actions_batch</code>)	:212-214	-0.2708 (หลัง $\mu \rightarrow 0.25$)
หาค่า	$\rho = \exp(\log \pi_{new} - \log \pi_{old})$	<code>ratio = torch.exp(...)</code> :252-254	$e^{-0.2708+0.3058} = e^{0.035} = 1.0356$

จะเห็นว่าไม่ต้องเก็บทั้งใบ — เก็บแค่ตัวเลข $\log \pi_{old}$ หนึ่งค่าต่อช่องก็พอ เพราะ (21) ต้องการแค่ความหนาแน่น ณ a_t ตัวเดียว

Remark — ทำไม policy ต้องสุ่ม เหตุผลข้อที่สอง. หัวข้อ 2.4 ให้เหตุผลข้อแรก (ไม่สุ่มก็ไม่มีข้อมูลเปรียบเทียบ) เหตุผลข้อสองอยู่ในสมการ (21): ถ้า policy ไม่สุ่ม มันจะให้ความหนาแน่น "ทั้งหมด" กับ action เดียวและ 0 กับที่เหลือ อัตราส่วนกลายเป็น $0/0$ หรือนิยามไม่ได้ทันทีที่ μ ขยับ $\rightarrow$ importance sampling ใช้ไม่ได้ $\rightarrow$ ข้อมูลใช้ ข้ามไม่ได้เลย PPO จึงต้องมีระว่างจริง ๆ ไม่ใช่แค่ต้องการ

Remark — ราคาของ importance sampling: variance. สมการ (20) เป็นจริงเป๊ะในทาง ทฤษฎี แต่ในทางปฏิบัติ ρ ไม่มีเพดาน — ค่าเฉลี่ยจาก 8 ช่อง (หรือ 12,288 ช่อง) อาจถูกช่องเดียวที่ ρ ใหญ่มากลากไปทั้งก้อน ยิ่ง π_{new} ห่างจาก π_{old} ยิ่งแย และในโค้ดจริง ρ คือผลคูณ 8 ข้อต่อ (จากสมการ (15)): ถ้าทุกข้อต่อขยับจนอัตราส่วน แต่ละตัวเป็น 1.2 คารวมคือ $1.2^8 \approx 4.3$ · ข้อสรุป: ค่าประมาณจาก (20) เชื่อได้ เฉพาะเมื่อ policy ใหม่ยังอยู่ใกล้ตัวเก่า — นี่คือเหตุผลทั้งหมดที่ PPO มีคำว่า "clip" ในชื่อ (หัวข้อ 4.6) ไม่ใช่ลูกเล่นเพิ่มเติม แต่เป็นเงื่อนไขที่ importance sampling บังคับ

สรุปหัวข้อ — PPO เป็น on-policy: ล้างตารางทุก iteration แต่ปรับ weight หลายจำนวนตารางใบเดิม; importance sampling (20) ทำให้ข้อมูลของ policy เก่ายังใช้ได้โดยถ่วงด้วย ρ_t (21) = อัตราส่วน ความหนาแน่น (e0 t0: 1.0356); ราคาคือ variance ที่โตเมื่อ policy ห่างกัน $\rightarrow$ ต้อง clip

สรุปภาค 3 — ข้อจำกัดสามข้อที่บังคับรูปร่างของคำตอบ:

ทางตัน	บังคับให้ต้องมีอะไร
tabular ใช้ไม่ได้ (continuous space)	$\rightarrow$ ต้องใช้ neural network เป็น function approximator (ภาค 2)
MC ขี้แฉะ variance สูง / TD มี bias	$\rightarrow$ ต้องมีวิธีผสมสองขั้ว = GAE (หัวข้อ 4.5) สร้างจาก δ_t (19)
on-policy ทั้งข้อมูลน้อย / importance sampling แกว่งเมื่อ policy ห่าง	$\rightarrow$ ต้องใช้ ρ_t (21) และจำกัดระยะที่ policy ขยับ = clip (หัวข้อ 4.6)

สามข้อนี้รวมกันคือโครงกระดูกของ PPO พอดี — ภาค 4 ประกอบมันขึ้นมาทีละชิ้น เริ่มจาก คำถามที่ภาค 2 ทิ้งไว้: loss $\mathcal{L}$ ในสมการ (12) หน้าตาเป็นยังไง

ภาค 4 — PPO ประกอบขึ้นมายังไง

ภาคนี้ประกอบขึ้นส่วนที่ภาค 3 บอกว่าจำเป็น เข้าด้วยกันทีละชิ้น จนกลายเป็น อัลกอริทึมที่รับอยู่จริงใน `train.py` · ลำดับคือ: ทิศที่ต้องขยับ θ (4.1) → ใครบอก ว่า action ดีหรือแย่ (4.2–4.3) → ซ่อมข้อมูลที่ขาด (4.4) → ตัวเลข advantage ที่ใช้จริง (4.5) → loss ที่ก้อน (4.6) → หน่วยนับของฟังก์ชันอัลกอริทึม (4.7)

4.1 Policy gradient — ทิศที่ต้องขยับ θ

ภาค 2 จบด้วยสมการ (12) $\theta \leftarrow \theta - \alpha \nabla_{\theta} L$ โดยยังไม่รู้ว่า L คืออะไร · ตรงนี้ตอบครั้งแรก: ถ้าอยากให้ผลตอบแทนคาดหวังสูงขึ้น ควรขยับ θ ไปทางไหน

ภาพก่อน: สี่กรณีของการขยับ μ

ดูข้อต่อเดียว ระวังหนึ่งใบ (จุดกลาง μ) และ action a ที่สุ่มได้ สมมติมีใครสักคน บอกเราแล้วว่า action นั้น "ดีกว่าที่คาด" หรือ "แย่กว่าที่คาด" (ตัวเลข A จากหัวข้อ 1.7 — ใครเป็นคนบอก อยู่หัวข้อ 4.2) — สามัญสำนึกบอกว่า:

กรณี	A	$a - \mu$	μ ควรเคลื่อนไป	แปลว่า
action ดี, a อยู่ขวาของ μ	+	+	ขวา (เข้าหา a)	"ทำแบบนั้นอีก"
action ดี, a อยู่ซ้ายของ μ	+	-	ซ้าย (เข้าหา a)	"ทำแบบนั้นอีก"
action แย่, a อยู่ขวาของ μ	-	+	ซ้าย (หนี a)	"อย่าทำแบบนั้น"
action แย่, a อยู่ซ้ายของ μ	-	-	ขวา (หนี a)	"อย่าทำแบบนั้น"

สี่แถวนี้สรุปเป็นกฎเดียว: ขยับ μ ไปทาง $A \cdot (a - \mu)$ — เข้าหา action ที่ดี หนีจาก action ที่แย่ แรงเป็นสัดส่วนกับว่าดี/แย่แค่ไหน · ที่เหลือของหัวข้อนี้คือการพิสูจน์ ว่าแคลคูลัสให้กฎเดียวกันนี้ออกมาเอง และให้สูตรที่ใช้กับ weight ทุกตัว ไม่ใช่แค่ μ

สิ่งที่อยากให้อัป: $J(\theta)$

สัญลักษณ์	ชนิด	ความหมาย
$J(\theta)$	ฟังก์ชัน $\mathbb{R}^{187,280} \rightarrow \mathbb{R}$	ผลตอบแทนคาดหวังของสมการ (2) มองเป็นฟังก์ชันของ θ — ใส่ weight ชุดหนึ่ง ได้ตัวเลขเดียวว่า policy ชุดนั้นเก็บคะแนนได้แค่ไหน
$\nabla_{\theta} J$	เวกเตอร์ยาว 187,280	ทิศที่ทำให้ J เพิ่มขึ้นที่เร็วที่สุด — ตัวเดียวกับ gradient ในหัวข้อ 2.2 แต่ของ J ไม่ใช่ของ L
$\mathbb{E}_{a \sim \pi_{\theta}}[\cdot]$	ตัวดำเนินการ	ค่าเฉลี่ยเมื่อ a สุ่มจากระยะของ policy ปัจจุบัน

ปัญหาคือ J ขึ้นกับ θ ผ่านการสุ่ม — θ กำหนดระยะ ระวังกำหนดว่า a ไหนออกบ่อย a กำหนด reward · จะหาอนุพันธ์ผ่านการสุ่มได้อย่างไร? เครื่องมือคือเอกลักษณ์เล็ก ๆ ข้อหนึ่ง

เอกลักษณ์ score function

$$\nabla_{\theta} \pi_{\theta}(a | s) = \pi_{\theta}(a | s) \nabla_{\theta} \log \pi_{\theta}(a | s) \quad (22)$$

ความหมาย: มาจากกฎลูกโซ่ของ log ตรง ๆ — $\nabla \log \pi = \nabla \pi / \pi$ ย้ายข้าง ก็ได้ (22) · ประโยชน์คือข้างขวามี $\pi_{\theta}(a | s)$ ยืนอยู่หน้าสุด ซึ่งคือ "น้ำหนักของ การสุ่ม" — พอเอาไปใส่ในอินทิกรัล มันจะรวมกลับเป็นค่าเฉลี่ยได้ · ตัว $\nabla_{\theta} \log \pi$ มีชื่อว่า **score function** (ฟังก์ชันคะแนน — คนละ "คะแนน" กับ reward)

derive ทิศของ J

ดูท้าวเดียวก่อน: อยากรู้ว่าค่าเฉลี่ยของ $f(a)$ (อะไรก็ได้ที่วัด "ดี" — เดี่ยวจะเป็น A) เปลี่ยนยังไงเมื่อขยับ θ

ขั้นตอน	เหตุผล
$\nabla_{\theta} \mathbb{E}_{a \sim \pi_{\theta}}[f(a)] = \nabla_{\theta} \int \pi_{\theta}(a s) f(a) da$	เขียนค่าเฉลี่ยเป็นอินทิกรัล — f ไม่ขึ้นกับ θ (reward มาจากโลก)
$= \int \nabla_{\theta} \pi_{\theta}(a s) f(a) da$	สลับอนุพันธ์กับอินทิกรัล (ระวัง Gaussian เรียบพอให้ทำได้)
$= \int \pi_{\theta}(a s) \nabla_{\theta} \log \pi_{\theta}(a s) f(a) da$	แทน (22)
$= \mathbb{E}_{a \sim \pi_{\theta}}[\nabla_{\theta} \log \pi_{\theta}(a s) f(a)]$	อ่านกลับเป็นค่าเฉลี่ย — ประมวลได้จากตัวอย่างที่สุ่มมา ไม่ต้องรู้อินทิกรัล

ต่อกันหลายก้าวตามสมการ (2) ผลลัพธ์มีรูปเดียวกับทุกก้าว (นี่คือ **policy gradient theorem** — Sutton & Barto §13.2) และตัว f ที่ถูกต้องของแต่ละก้าวคือ **advantage** $A^\pi(s_t, a_t)$ จากสมการ (8) — ตัวนี้ขึ้นกับ θ เอง (ด้วย π) จึงพันข้อสมมติ " f คงที่" ของ ตารางข้างบน ความถูกต้องมาจากทฤษฎีบทที่พิสูจน์ว่า ∇ ตกที่ π อย่างเดียว และในโค้ด $\hat{A}$ ถูก detach ไว้เป็นค่าคงที่แล้ว (advantages_batch): ใช้ G_t ก็ได้ในทางทฤษฎี แต่การลบ $V(s_t)$ ออก (baseline — §13.4) ไม่เปลี่ยนค่าเฉลี่ยและลด variance มหาศาล ด้วยเหตุผลเดียวกับที่ตารางสี่กรณีบอก: สิ่งที่เราควรหลีกเลี่ยงคือ "ดีกว่าที่คาด" ไม่ใช่ "ได้คะแนนสูง"

$$\nabla_\theta J(\theta) = \mathbb{E} \left[\nabla_\theta \log \pi_\theta(a_t | s_t) A^\pi(s_t, a_t) \right]$$

(23)

ความหมาย: ทิศที่ดีที่สุดสำหรับ θ = ค่าเฉลี่ยของ "ทิศทางที่ทำให้ a_t ออกบ่อยขึ้น" ถ่วงด้วย "action นั้นดีกว่าที่คาดแค่ไหน" · $A > 0 \rightarrow$ ดันให้ a_t ออกบ่อยขึ้น · $A < 0 \rightarrow$ ค่าติดลบคูณเข้าไป = ดันให้ออกน้อยลง · ตารางสี่กรณีคือ (23) เวอร์ชันพูด

ทิศนี้เป็นเลขเท่าไรสำหรับ Gaussian — $\partial \log \pi / \partial \mu$

(23) ต้องการ $\nabla_\theta \log \pi$ ซึ่งสำหรับ weight ทั้ง 187,280 ตัวมี autograd คำนวณให้ แต่ตัวที่ derive ด้วยมือได้และเห็นความหมายชัดที่สุดคืออนุพันธ์เทียบ μ — ใช้สมการ (14):

ขั้นตอน	เหตุผล
$\log \pi = -\frac{(a - \mu)^2}{2\sigma^2} - \log \sigma - \log \sqrt{2\pi}$	สมการ (14)
$\frac{\partial \log \pi}{\partial \mu} = -\frac{2(a - \mu)(-1)}{2\sigma^2}$	สองพจน์หลังไม่มี μ อนุพันธ์เป็น 0 · พจน์แรกใช้กฎลูกโซ่ผ่าน $(a - \mu)$

$$\frac{\partial \log \pi_\theta(a | s)}{\partial \mu} = \frac{a - \mu}{\sigma^2}$$

(24)

ความหมาย: เครื่องหมายคือ "a อยู่ข้างไหนของ μ " ขนาดคือ "ห่างกี่เท่าของ σ^2 " · เอาไปใส่ (23): $\partial J / \partial \mu \propto A \cdot (a - \mu) / \sigma^2$ — ตารางสี่กรณี **โผล่ออกมาจากแคลคูลัสเอง** เครื่องหมายของ A คูณเครื่องหมายของ $(a - \mu)$

ในตาราง 2×4 (ใช้ δ_t จากหัวข้อ 3.2 เป็นตัวประมาณ A แบบหยาบ — ตัวจริงที่โค้ดใช้ คือ $\hat{A}_t$ ของหัวข้อ 4.5 ซึ่งเครื่องหมายตรงกันในสองช่องนี้):

ช่อง	a	μ	$(a - \mu) / \sigma^2$	$A \approx \delta$	$A \cdot (a - \mu) / \sigma^2$	μ ควรไป
e0 t0	0.40	0.20	$0.20 / 0.25 = 0.80$	+0.05	+0.04	ขึ้น เข้าหา 0.40 (แถว 1 ของตารางสี่กรณี)
e1 t1	0.40	0.00	$0.40 / 0.25 = 1.60$	-0.10	-0.16	ลง หนีจาก 0.40 (แถว 3)

Remark — REINFORCE ไม่มีใน repo แต่ (23) ทำงานอยู่. อัลกอริทึมที่ทำ (23) ตรง ๆ ชื่อ REINFORCE (Sutton & Barto §13.3) — โค้ดไม่มีบรรทัดไหนชื่อนั้น สิ่งที่มีคือ `log_prob(...).sum(-1)` (`actor_critic.py:164`) เข้าไปอยู่ใน `loss` แล้ว `loss.backward()` (`ppo.py:288`) ให้ autograd คำนวณ $\nabla_\theta \log \pi$ ของ weight ทุกตัว — (23) จึงถูก ทำโดย autograd ไม่ใช่ด้วยสูตรที่พิมพ์ไว้ และ (24) คือสิ่งที่ autograd คำนวณได้ทีชั้น สุดท้ายของ `actor` · `loss` ตัวจริงมีของเพิ่มอีกสองอย่างจาก (23): ρ จากหัวข้อ 3.3 และการ clip — ทั้งหมดอยู่หัวข้อ 4.6

สรุปหัวข้อ — score function (22) ทำให้หาอนุพันธ์ผ่านการสุ่มได้; policy gradient (23) บอกว่าทิศ ของ θ = ค่าเฉลี่ยของ $\nabla \log \pi \cdot A$; สำหรับ Gaussian ที่ μ คือ $(a - \mu) / \sigma^2$ (24) $\rightarrow$ เข้าหา action ที่ $A > 0$ หนีจาก action ที่ $A < 0$; สิ่งที่ยังขาดคือใครให้ค่า A

4.2 Actor-Critic — ทำไมต้องมี network สองก้อน

หัวข้อ 2.1 บอกไปแล้วว่า Tron1 มี actor กับ critic — ตรงนี้คือ**เหตุผลเชิง อัลกอริทึม**ว่าทำไมถึงต้องเป็นสองก้อน และสมการ (23) เพิ่งบอกเหตุผลนั้นไปครึ่งหนึ่ง

จนถึงตอนนี้เรามีสองสิ่งที่แยกกันอยู่:

- **Value function** $V^\pi(s)$ — บอกว่า state นี้ดีแค่ไหนโดยรวม (หัวข้อ 1.6)
- **Policy** π_θ — ตัดสินใจว่าจะทำ action อะไร (ภาค 2)

วิธีคิดแบบแรก ๆ (value-based RL เช่น Q-learning) ฝึกแค่ value function แล้ว "เลือก action ที่ทำให้ value สูงสุด" — ใช้ได้ดีเมื่อ action มีจำนวนจำกัด (นับได้) แต่ Tron1 มี action เป็น continuous (offset มุมข้อต่อ float 8 ค่า) การจะ "ลองทุก action แล้วเลือกตัวที่ดีที่สุด" ทำไม่ได้ เพราะ

มีอนันต์ทางเลือก · ทางกลับกัน (23) ปรับ policy ได้โดยตรงโดยไม่ต้องลองทุก action — แต่มันต้องการ $\mathbf{A}$ และ $\mathbf{A} = \mathbf{Q} - \mathbf{V}$ (สมการ (8)) ต้องมีใครสักคนประเมิน $\mathbf{V}$ ให้

ทางแก้คือฝึก **สองเครือข่ายคู่กัน**:

ชื่อ	หน้าที่	เรียนรู้จากอะไร	ใน Tron1	แถวในตาราง 2×4
Actor (π_θ)	ตัดสินใจ action โดยตรงจาก state — คือ policy ตัวจริงที่ deploy ไปยังหุ่นยนต์	policy gradient (23) โดยใช้ $\mathbf{A}$ ที่ critic ช่วยคำนวณ	โครงข่าย 42 → 512 → 256 → 128 → 8 + logstd	แถว μ
Critic (V_ϕ)	ประเมินว่า state ที่ actor เจอนั้นดีแค่ไหน ใช้เป็น "ครู" ช่วยสอน actor	ผลรวม reward ที่เกิดขึ้นจริง (หัวข้อ 4.3)	โครงข่าย 230 → 512 → 256 → 128 → 1	แถว V

ในโค้ดสองก้อนทำงานในเมรอตเดียวกันทุก env step (ppo.py:133-149): actor ที่ :137-139 critic ที่ :144 แล้วเก็บ $\log \pi_{old}$ ที่ :147-149 — ทุกช่องของตารางจึงมีทั้ง μ และ V พร้อมกัน · critic คอยบอกว่า "ดีกว่าที่คาดไว้" แปลว่าอะไร — นี่คือที่มาของคำว่า **advantage** และตัวเลข $\hat{A}_t$ ที่ใช้จริงจะประกอบจาก V ทั้งตารางในหัวข้อ 4.5

Remark. รายละเอียดว่า actor เห็น 42 มิติ ส่วน critic เห็น 230 มิติ (**asymmetric actor-critic**) อยู่ในตารางหัวข้อ 2.1 แล้ว — สรุปสั้น ๆ คือ critic โกงดูข้อมูลที่มีแค่ในซิมได้ เพราะมันถูกทิ้งตอน deploy ส่วน actor ต้องเก่งจากสิ่งที่เซนเซอร์จริงวัดได้เท่านั้น

สรุปหัวข้อ — continuous action ทำให้ argmax ทำไม่ได้ จึงปรับ policy ตรง ๆ ด้วย (23); (23) ต้องการ $\mathbf{A}$ ซึ่งต้องการ $\mathbf{V} \rightarrow$ critic; actor กับ critic คือแถว μ กับแถว V ของตารางเดียวกัน

4.3 Critic ต่างจาก Reward function ยังไง

สองอย่างนี้สับสนกันง่ายมาก เพราะทั้งคู่คายตัวเลขที่ "บอกว่าดีแค่ไหน" ออกมา เหมือนกัน — แต่อยู่คนละฝั่งของปัญหาเลย

Reward function คือ โจทย์ — Critic คือ เครื่องมือแก้โจทย์

	Reward function $R(s, a)$	Critic $V_\phi(s)$
ใครสร้าง	มนุษย์เขียน (วิศวกร LimX)	เรียนรู้เองจากข้อมูล
เป็นอะไร	โค้ด Python ที่คำนวณตามสูตรตายตัว	neural network 230 → 512 → 256 → 128 → 1 (actor_critic.py:92-111)
อยู่ตรงไหนใน MDP	ช่อง R ในแบบฟอร์ม 5 ช่อง (หัวข้อ 1.2)	ไม่อยู่ในแบบฟอร์มเลย — เป็นส่วนของอัลกอริทึม
ตอบคำถามอะไร	"step ที่เพิ่งผ่านไป ได้กี่คะแนน"	"จากตรงนี้ไปจนจบ น่าจะได้อีกกี่คะแนน"
มองไปทางไหน	อดีต (สิ่งที่เพิ่งเกิด)	อนาคต (สิ่งที่ยังไม่เกิด)
เปลี่ยนระหว่างเทรนไหม	ไม่เปลี่ยนเลย ตายตัวตั้งแต่ต้นจนจบ	เปลี่ยนทุก iteration (20 gradient step — หัวข้อ 4.7)
ผิดได้ไหม	ผิดไม่ได้ — มันคือ <i>นิยาม</i> ของคำว่า "ดี"	ผิดได้ตลอด เป็นแค่การคำนวณ
ตอน deploy	ไม่ใช่ (ไม่มีใครให้คะแนนหุ่นจริง)	ถูกทิ้ง
อยู่ไฟล์ไหน	mdp/rewards.py + RewardsCfg (limx_base_env_cfg.py:368, 20 พจน์)	rsl_rl/modules/actor_critic.py
ในตาราง 2×4	แถว r	แถว V

ดูที่จังหวะเดียวกันจะเห็นชัด — สมมติหุ่นอยู่ที่ step 300 กำลังเดินอยู่ดี ๆ:

step 300 →

Reward function พูดว่า —

"step นี้ทรงตัวดี +0.8
เก้ากระแทกแรงไปหน่อย -0.1
รวม step นี้ได้ 0.7" ← ข้อเท็จจริง

Critic พูดว่า —

"ท่าทางแบบนี้ ดูแล้วน่าจะเก็บได้อีกราว
650 แต่มันจบ episode" ← การทำนาย

Reward = ข้อเท็จจริงที่วัดได้แล้ว (คำนวณจากฟิสิกส์ที่เพิ่งเกิด ไม่มีทางผิด) **Critic = การเดาอนาคต** (อาจเดาผิดเป็น 400 ก็ได้ ถ้าจริง ๆ อีก 3 step หน้าน่าจะล้ม) — ในตาราง 2×4 ช่อง e1 t0 คือแบบนั้นพอดี: critic พุด 3.0 แต่ผลจริง $G_0 = -0.22$ (หัวข้อ 3.2)

ความสัมพันธ์: critic เรียนรู้ จาก reward

สองอย่างนี้ไม่ได้แยกกันอยู่ — **reward คือ "ครู" ของ critic**

$$\underbrace{V_\phi(s)}_{\text{critic ทำนาย}} \longleftarrow \text{เรียนรู้ ให้ เข้าใจ} \longrightarrow \underbrace{\mathbb{E}\left[\sum_k \gamma^k r_{t+k}\right]}_{\text{ผลรวม reward จริงในอนาคต — สมการ (3)}}$$

```
reward function (มนุษย์เขียน)
| เป็นเป้าหมายให้เรียนรู้ (value loss - หัวข้อ 4.6)
▼
critic (เรียนรู้ที่จะทำนายผลรวมของ reward)
| ให้ค่า V(s) ไปคำนวณ advantage (หัวข้อ 4.5)
▼
actor (เรียนรู้จาก advantage ด้วยสมการ (23))
```

เทียบกับ LQR — สำหรับคนที่มาจากสาย control

ถ้าเคยผ่าน optimal control มาแล้ว ความสัมพันธ์นี้คุ้นมาก:

Optimal control (LQR — linear quadratic regulator)	RL (PPO)
stage cost $x^T Q x + u^T R u$ — เลือก Q, R เอง	reward function — วิศวกรเขียนเอง
cost-to-go $V(x) = x^T P x$ — ได้จากแก้สมการ Riccati	critic $V_\phi(s)$ — ได้จากการฝึก neural network
P คำนวณได้ตรง ๆ เพราะระบบเป็นเชิงเส้น	หาตรง ๆ ไม่ได้ เพราะระบบไม่เชิงเส้นและไม่รู้สมการ → ต้องประมาณด้วย NN

critic คือ cost-to-go function ที่หาด้วยวิธีวิเคราะห์ไม่ได้ เลยต้องเดาเอาจาก ข้อมูล — แนวคิดเดียวกันเป๊ะกับ P ใน Riccati equation ต่างกันแค่วิธีได้มา (ตัว Q, R ของ LQR เป็นเมทริกซ์น้ำหนัก คนละตัวกับ Q^π และ $R(s, a)$ ของโน้ตนี้)

ผลที่ตามมาในทางปฏิบัติ — สำคัญที่สุดในหัวข้อนี้

critic แก้ reward function ที่ผิดไม่ได้

ถ้า reward function เขียนไว้ผิด (เช่น ให้คะแนนความเร็วสูงมากจนหุ่นเรียนรู้ที่จะ กระโดดไปข้างหน้าแล้วล้ม) critic ก็จะเรียนรู้ที่จะ **ทำนายคะแนนของพฤติกรรมผิด ๆ นั้นได้อย่างแม่นยำ** — มันไม่มีทางรู้ว่าเป้าหมายที่ตั้งไว้ผิด

นี่คือเหตุผลที่ **งานวิศวกรรมส่วนใหญ่ของ RL อยู่ที่ reward function ไม่ใช่ที่ อัลกอริทึม** — Tron1 มี reward ถึง 20 terms พร้อมน้ำหนักที่จูนมาอย่างละเอียด (03-sf-task-anatomy.md) ในขณะที่ critic เป็นแค่ MLP ธรรมดาที่ไม่มีอะไรพิเศษเลย

```
เชื่อมกับตารางใน 04-file-structure-vs-ros2.md: การเปลี่ยน "เป้าหมาย" ทำที่ cfg/SF/limx_base_env_cfg.py → RewardsCfg ส่วนการเปลี่ยน "วิธีเรียนรู้" ทำที่ rs1_rl/ — คนละไฟล์ คนละความรับผิดชอบ
```

สรุปหัวข้อ — reward function คือโจทย์ (มนุษย์เขียน มองอดีต ไม่เปลี่ยน) critic คือเครื่องมือ (เรียนรู้ มองอนาคต ผิดได้); critic เรียนจาก reward ผ่านสมการ (3) และเป็น cost-to-go ที่หาด้วย Riccati ไม่ได้; reward ผิด critic ช่วยไม่ได้

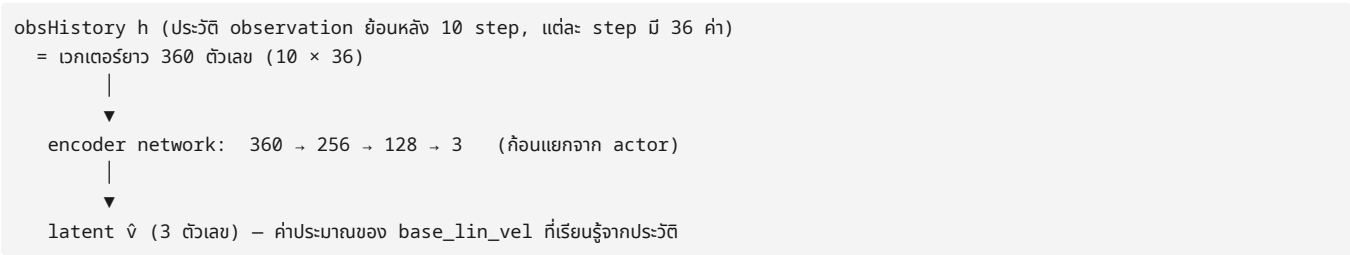
4.4 Encoder กับ latent ๖ — ซ่อมปัญหาไม่ Markov

หัวข้อ 1.3 ถึงหน้าไว้ว่า Tron1 ไม่ Markov แท้ และหัวข้อ 2.2 พุดถึง "latent 3 ค่าจาก encoder" ลอย ๆ — ตรงนี้คือการขำระหนักทั้งสองอย่าง

ปัญหาที่ encoder ถูกสร้างมาแก้: เซนเซอร์จริงบนหุ่นยนต์ (IMU + joint encoder) **วัดความเร็วเชิงเส้นของลำตัว (base linear velocity) ไม่ได้โดยตรง** — วัดได้แค่ความเร็วเชิงมุม (base_ang_vel), แรงแโน้มถ่วงที่ฉายลงมา (proj_gravity), มุม/ความเร็วข้อต่อ แต่ "ตอนนี้ลำตัวกำลังเคลื่อนที่ไปข้างหน้า เร็วแค่ไหน" ไม่มีเซนเซอร์ตัวไหนบอกตรง ๆ แต่ policy จำเป็นต้องรู้ค่านี้ทางอ้อม เพื่อควบคุมความเร็วการเดินทางให้แม่นยำ

ทางแก้: ถ้าเราดูประวัติของสิ่งที่วัดได้ย้อนหลังไปหลาย step (เช่น ขาไหนกำลังเคลื่อนไหวแบบไหน ต่อเนื่องกันมา 10 step) จริง ๆ แล้วสามารถ "เดา" ความเร็วลำตัวได้ในระดับหนึ่ง — คล้ายกับที่คนเราหลังตาแล้วยังรู้สึกได้ คร่าว ๆ ว่ากำลังเดินเร็วแค่ไหน จากจังหวะการก้าวขา ไม่ใช่จากการมองเห็น

encoder คือ neural network **ก้อนที่สาม แยกจาก actor และ critic** ที่ทำหน้าที่โดยเฉพาะ (คลาส `MLP_Encoder` `mlp_encoder.py:39`, พารามิเตอร์ ψ 125,699 ตัว):



สัญลักษณ์/ค่า	ชนิด	ความหมาย	ที่
h obsHistory	เวกเตอร์ (360,)	ประวัติของ observation หลัก (36 มิติ) ย้อนหลัง 10 step ล่าสุด เอามาเรียงต่อกัน	HistoryObsCfg limx_base_env_cfg.py:154-176 (history_length = 10 :175) · obs_history_len = 10 limx_rsl_rl_ppo_cfg.py:106 · 360 = 10 × 36 on_policy_runner.py:63
encoder f_ψ	ฟังก์ชัน $\mathbb{R}^{360} \rightarrow \mathbb{R}^3$	neural network ขนาด 360 → 256 → 128 → 3 — โครงสร้างข้างในเหมือน node/layer ในหัวข้อ 2.2 ทุกประการ เพียงแต่เป็นคนละก้อนที่มีหน้าที่ต่างจาก actor	hidden_dims = [256, 128], num_output_dim = 3 limx_rsl_rl_ppo_cfg.py:110-111
$\hat{v}$ latent	เวกเตอร์ (3,)	output ของ encoder — เรียกว่า "latent" (แฝงอยู่) เพราะเป็นค่าที่ network สรุปลงมาให้อยู่ ไม่ใช่ค่าที่มนุษย์นิยามเหมือน joint_pos · หมวกอ่านว่า "ค่าประมาณของ" — สิ่งที่มีนัยประมาณคือ $\mathbf{x} = \text{base_lin_vel}$	encoder.encode(obs_history) ppo.py:136

จากนั้น $\hat{v}$ 3 ค่านี้ถูก**ต่อท้าย (concatenate)** เข้ากับ observation หลัก 36 ค่าและ command 3 ค่า กลายเป็น input ก้อนเดียวกับที่ actor เห็น — นี่คือการมาของตัวเลข "42" ที่พูดถึงตลอดทั้งโน้ตนี้ (3 + 36 + 3 — `ppo.py:137-139`, ดูหัวข้อ 2.2 และโน้ต 06 §1)

encoder เรียนรู้จากอะไร — และทำไมมันถึงรู้ว่าต้องเดา "ความเร็วลำตัว"

ตรงนี้ไม่ต้องเดา เพราะโค้ดบอกตรง ๆ (`ppo.py:313-315`):

```
extra_loss = (
    (encode_batch[:, 0:3] - critic_obs_batch[:, 0:3]).pow(2).mean()
)
```

สามช่องแรกของ critic observation คือ `base_lin_vel` (พจน์แรกของ `CriticCfg limx_base_env_cfg.py:183`) — encoder จึงถูกสอนให้ output ของมันตรงกับความเร็วลำตัว **จริง** ที่ซิมรู้ (ค่าเฉลี่ยของผลต่างกำลังสอง = mean squared error, MSE) · นี่คือการเรียนแบบ **supervised** ตรงไปตรงมา: มีเฉลย (ground truth จากฟิสิกส์) ให้เทียบทุก step ต่างจาก actor ที่ไม่มีเฉลยว่า action ที่ถูกคือเท่าไร

Remark — ทำไม encoder ถึงเป็น network แยกก้อน ไม่ใช่ชั้นแรกของ actor. เพราะมันฝึกด้วยเป้าหมายคนละแบบและมี optimizer ของตัวเอง: `output_detach = True` (`limx_rsl_rl_ppo_cfg.py:109`) ทำให้ `encode()` คืนค่าที่ `.detach()` แล้ว (`mlp_encoder.py:95-100`) — gradient จาก loss ของ policy **ไม่ไหลย้อน กลับเข้าไปปรับ ψ** สายระหว่างสองก้อนถูกตัด · encoder ถูกฝึกแยกด้วย `extra_optimizer` (`ppo.py:88-89`) ในลูปของมันเอง (:299-321) ด้วย MSE ข้างบน

เป็นเทคนิคที่เรียกว่า **auxiliary task / state estimation** ซึ่งแยกออกไปจาก PPO core ที่โน้ตนี้โฟกัสอยู่ — รู้แต่ว่ามันคือ "ตัวเดาความเร็วลำตัว" ก็พอ สำหรับเข้าใจภาพรวม · ผลข้างเคียงที่ต้องรู้: กฎปรับ lr ของ PPO (หัวข้อ 4.6) **ไม่แตะ** lr ของ encoder เพราะเป็นคนละ optimizer

ทางเลือกอื่นที่ใช้ได้จริงเหมือนกัน. ปัญหา "ประมาณความเร็วลำตัวจาก IMU + joint encoder" เป็นโจทย์คลาสสิกของ **sensor fusion** — คุ้นเคยดีสี่ขา/สองขา จำนวนมาก (MIT Cheetah, ANYmal รุ่นแรก ๆ) ใช้ **Extended Kalman Filter (EKF)** ทำ งานนี้แทน โดยผสม IMU + forward kinematics + ข้อสมมติว่าเท้าที่แตะพื้นไม่ลื่น

Tron1 เลือก learned encoder แบบ ด้วยเหตุผลหลักว่า **ในซิมมี ground truth ให้ ฝึกฟรี** (physics engine รู้ `base_lin_vel` จริงอยู่แล้ว — บรรทัด :314 ข้างบน) จึงไม่ต้อง derive dynamics model หรือ tune noise covariance เอง และมันทนต่อ domain randomization แบบเดียวกับที่ policy เจอโดยอัตโนมัติ — แลกกับการที่ interpretability ต่ำกว่า KF (ไม่มี covariance matrix ให้ดูว่า "ตอนนี้ไม่มันใจ") ทั้งสองวิธีถูกต้องและใช้จริงในวงการ ไม่ใช่ว่าอันใดอันหนึ่งผิด

สรุปหัวข้อ — encoder เป็น NN ก้อนที่สาม รับประวัติ h (360) คืน $\hat{v}$ (3) ที่ถูกสอนแบบ supervised ให้ตรงกับ `base_lin_vel` จริงของซิม; ถูก detach ออกจาก loss ของ policy และมี optimizer แยก; $\hat{v}$ ต่อเข้า input ของ actor เป็น 3 + 36 + 3 = 42 — คืนความเป็น Markov ให้ policy โดย

4.5 Advantage และ GAE — ให้เครดิต action ยังไงให้แม่นยำและนิ่ง

สมการ (23) ต้องการ A ของทุกช่อง และหัวข้อ 3.2 ให้ δ_t มาเป็นตัวประมาณแบบหยาบ · หัวข้อนี้สร้างตัวประมาณที่โคตใช้จริง — จากตาราง 2×4 ที่มีแค่ $r, V, done$:

	t0	t1	t2	t3	V_{last}
env 1: r	0.5	1.0	-2.0	0.0	
env 1: V	3.0	2.0	1.0	4.0	3.5
env 1: done	0	0	1	0	

คำถามของหัวข้อนี้: env 1 ลืมที่ t2 — action ไหนควรถูกโทษ? t2 เองแน่นอน แต่ t1 ที่พาไป ทำนั้นล่ะ? แล้ว t0? และควรโทษมากแค่ไหน — นี่คือปัญหา **credit assignment** (การให้ เครดิต/โทษย้อนหลัง) และ GAE คือคำตอบเชิงตัวเลขของมัน

จุดที่ควรสังเกต: สิ่งที่ขับเคลื่อนการเรียนรู้ไม่ใช่ reward ดิบ แต่เป็น **advantage** — action ที่ได้ reward 0.5 อาจถือว่า "แย่" ถ้า critic คาดไว้ว่าควรได้ 0.8 และ action ที่ได้ reward ตีตกก็อาจถือว่า "ดี" ถ้าสถานการณ์นั้น แย่อยู่แล้วและ critic คาดว่าจะแย่กว่านั้นอีก

สัญลักษณ์	ชนิด	ความหมาย	toy · Tron1
δ_t	scalar ต่อช่อง	TD error สมการ (19) — critic พลาดเท่าไรที่ก้าวนี้	ตารางในหัวข้อ 3.2
λ GAE lambda	ค่าคงที่	เชื่อ critic แค่ไหน vs สอผลจริง (0 = TD ล้วน, 1 = MC ล้วน)	$0.8 \cdot 0.95$ (lam limx_rsl_rl_ppo_cfg.py:103)
$\gamma\lambda$	ค่าคงที่	ตัวคูณลดน้ำหนักต่อก้าวของ δ ในอนาคต	0.72 · 0.9405
A_t	scalar ต่อช่อง	advantage ที่ประมาณด้วย GAE (ยังไม่ normalize)	e0 t0: -0.2722
R_t	scalar ต่อช่อง	เป้าที่ critic ต้องเรียนรู้ = $A_t + V_t$ · ไม่ใช่ G_t	e0 t0: 4.7278
$\hat{A}_t$	scalar ต่อช่อง	A_t ที่ normalize ทั้งตารางแล้ว — ตัวที่ loss ใช้จริง	e0 t0: +0.7975

จาก TD ก้าวเดียวไปถึง Monte Carlo — ตระกูลเดียวกัน

เริ่มจากนิยาม (8) $A = Q - V$ และประมาณ Q ด้วยก้าวจริง k ก้าวแล้วให้ critic ทำนายที่เหลือ:

ขั้นตอน	เหตุผล
$A_t^{(1)} = r_t + \gamma V(s_{t+1}) - V(s_t) = \delta_t$	ก้าวจริง 1 ก้าว — สมการ (6) แทน Q แล้วเอา V = TD error (19)
$A_t^{(2)} = r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2}) - V(s_t)$	ก้าวจริง 2 ก้าว
$= [r_t + \gamma V(s_{t+1}) - V(s_t)] + \gamma [r_{t+1} + \gamma V(s_{t+2}) - V(s_{t+1})]$	บวกและลบ $\gamma V(s_{t+1})$ — พจน์ตัดกันพอดี
$= \delta_t + \gamma \delta_{t+1}$	ก้าวจริง 2 ก้าว = พบบวก TD error 2 ตัว
$A_t^{(\infty)} = \sum_{k \geq 0} \gamma^k \delta_{t+k} = G_t - V(s_t)$	ทำต่อเนื่องจน episode — V ตรงกลางตัดกันหมด เหลือผลจริงลบค่าทำนาย = Monte Carlo

ตรวจกับ env 1 ที่ $t = 0$ (episode จบที่ t2, ใช้ δ จากตารางหัวข้อ 3.2): $\delta_0 + 0.9 \delta_1 + 0.81 \delta_2 = -0.70 - 0.09 - 2.43 = -3.22$ และ $G_0 - V(s_0) = -0.22 - 3.0 = -3.22$ — เท่ากันเป๊ะ ตามที่ derive

ยิ่ง k มาก ยิ่งใช้ผลจริงมาก (bias ต่ำ variance สูง) ยิ่ง k น้อย ยิ่งพึ่ง critic (bias สูง variance ต่ำ) — ตารางในหัวข้อ 3.2 นั่นเอง · **Generalized Advantage Estimation (GAE)** ไม่เลือก k ตัวเดียว แต่ถ่วงน้ำหนักทุก k ด้วย $(1 - \lambda)\lambda^k$ (น้ำหนักรวมกัน ได้ 1 — Schulman et al. 2016 สมการ (16)) ซึ่งรวมอนุกรมแล้วยุบเป็นผลรวมเดียว:

$$A_t = \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k} \quad (25)$$

ความหมาย: เอา TD error ของก้าวนี้ บวก TD error ของก้าวถัดไปทีละน้ำหนักเหลือ $\gamma\lambda$ บวกก้าวถัดจากนั้นที่ $(\gamma\lambda)^2 \dots \cdot \lambda = 0$ เหลือแค่ δ_t (TD ล้วน) · $\lambda = 1$ กลายเป็นแถวสุดท้ายของตารางข้างบน (MC ล้วน — ตรวจแล้วกับ env 1: -3.22) · lam = 0.95 ที่ Tron1 ใช้คือการเลือกจุดที่ "เชื่อ critic พอสมควร แต่ก็ยังพึ่งผลจริงในระยะ สั้น ๆ ถัดไปด้วย" — จุดนี้ในทางปฏิบัติมักให้การเรียนรู้ที่เร็วและนิ่งกว่าสุดโต่งทั้งสองฝั่ง

recursion ที่ใช้ได้ใช้ — และ mask ที่หยุดผลรวมตรงขอบ episode

โค้ดไม่ได้บวกอนุกรมอนันต์ มันเดินย้อนหลังจากแถวสุดท้ายขึ้นมา ทีละก้าว:

$$A_t = \delta_t + (1 - \text{done}_t) \gamma \lambda A_{t+1}, \quad A_T = 0 \quad (26)$$

ความหมาย: A ของช่องนี้ = δ ของช่องนี้ + $\gamma \lambda$ เท่าของ A ช่องถัดไป — และถ้า episode จบที่ช่องนี้ (mask = 0) ก็ไม่เอา A ของช่องถัดไปมาเลย เพราะช่องถัดไปเป็น คนละ episode · $A_T = 0$ คือ "หลังขอบตารางไม่มี δ ให้บวกแล้ว" (ส่วนอนาคตหลังขอบ อยู่ใน V_{last} ของ δ_{T-1} แล้ว)

พิสูจน์ว่า (26) คือ (25) (ไม่มี done): แทน A_{t+1} ด้วยนิยามของมันเองซ้ำ ๆ

ขั้นตอน	เหตุผล
$A_t = \delta_t + \gamma \lambda A_{t+1}$	นิยาม (26)
$= \delta_t + \gamma \lambda (\delta_{t+1} + \gamma \lambda A_{t+2})$	แทน A_{t+1}
$= \delta_t + \gamma \lambda \delta_{t+1} + (\gamma \lambda)^2 A_{t+2}$	กระจาย
$= \delta_t + \gamma \lambda \delta_{t+1} + (\gamma \lambda)^2 \delta_{t+2} + (\gamma \lambda)^3 A_{t+3}$	แทนอีกครั้ง
$= \sum_{k=0}^{T-1-t} (\gamma \lambda)^k \delta_{t+k}$	ทำจนถึง $A_T = 0$ ■

recursion จึงเป็นแค่วิธีคำนวณผลรวม (25) ให้เร็ว (ดูลบวกทีละก้าวแทนที่จะยกกำลังใหม่ ทุกครั้ง) และ mask ($1 - \text{done}$) ทำให้ผลรวมหยุดที่ขอบ episode

คำนวณด้วย recursion ($\gamma \lambda = 0.72$, เริ่ม $A_4 = 0$, δ จากหัวข้อ 3.2):

ช่อง	δ_t	mask $\times 0.72 \times A_{t+1}$	A_t
e0 t3	-0.30	$1 \times 0.72 \times 0 = 0$	-0.3000
e0 t2	+0.15	$1 \times 0.72 \times (-0.30) = -0.2160$	-0.0660
e0 t1	-0.40	$1 \times 0.72 \times (-0.066) = -0.0475$	-0.4475
e0 t0	+0.05	$1 \times 0.72 \times (-0.4475) = -0.3222$	-0.2722
e1 t3	-0.85	$1 \times 0.72 \times 0 = 0$	-0.8500
e1 t2	-3.00	$0 \times 0.72 \times (-0.85) = 0$	-3.0000
e1 t1	-0.10	$1 \times 0.72 \times (-3.0) = -2.1600$	-2.2600
e1 t0	-0.70	$1 \times 0.72 \times (-2.26) = -1.6272$	-2.3272

ตรวจด้วย closed-form (25) (ทำเองบนกระดาษได้):

$$A_0^{(e0)} = \delta_0 + 0.72 \delta_1 + 0.72^2 \delta_2 + 0.72^3 \delta_3 = 0.05 - 0.288 + 0.0778 - 0.1120 = -0.2722 \checkmark$$

$$A_0^{(e1)} = \delta_0 + 0.72 \delta_1 + 0.72^2 \delta_2 + \underbrace{0 \cdot \delta_3}_{\text{mask ตัด}} = -0.70 - 0.072 - 1.5552 = -2.3272 \checkmark$$

คำตอบของคำถามเปิดหัวข้อ: การลื่นที่ e1 t2 ($\delta = -3.00$) ถูกส่งย้อนไปโทษ t1 เต็ม ๆ 0.72 เท่า (-2.16 ใน A_1) และ t0 อีก 0.72^2 เท่า (-1.56 ใน A_0) — ส่วน $\delta_3 = -0.85$ ของ episode ใหม่ไม่ถูกนับใน $A_0 \dots A_2$ เลย เพราะเป็นคนละ episode · ถ้าไม่มี mask จะเอา ความลื่นเหลวของ episode หน้าไปโทษ action ใน episode ก่อน

Remark — กรณีหมดเวลา (time-out) ที่ตาราง 2×4 ไม่มี. done จาก time_out (limx_base_env_cfg.py:450) ก็ทำให้ mask = 0 เหมือนกัน ทั้งที่อนาคตยังมี · ppo.py:163-168 ขดเชยด้วยการบวก $\gamma \cdot V(s_t)$ เข้าไปใน r_t ก่อนเก็บลง buffer (ใช้ V ของช่อง นั้นเองที่ critic เพิ่งคำนวณตอน act() แทน $V(s_{t+1})$ ที่ไม่ได้คำนวณ — เป็นการประมาณ ของ rsl_rl) ผลคือ δ_t ของช่องหมดเวลายังมีพจน์ bootstrap อยู่ ไม่ถูกตัดเหมือนกรณีลื่น

เป้าของ critic และการ normalize — สองบรรทัดสุดท้ายของ compute_returns

$$R_t = A_t + V_t \quad (27)$$

ความหมาย: จากนิยาม $A = (\text{พลจ. งโดยประมาณ}) - (\text{ที่ 'critic' เดา}) \rightarrow \text{พลจริง} = A + V$ นี่คือ "เฉลี่ย" ที่จะเอาไปสอน critic ในหัวข้อ 4.6 (ง) · ไม่ใช่ G_t — มีคำทำนาย ของ critic เองปนอยู่ (bootstrap) จึงเป็นเฉลี่ยที่ดีขึ้นเรื่อย ๆ ตามที่ critic ดีขึ้น

ช่อง	A_t	V_t	R_t
e0 t0	-0.2722	5.0	4.7278
e0 t1	-0.4475	4.5	4.0525
e0 t2	-0.0660	4.0	3.9340
e0 t3	-0.3000	3.5	3.2000
e1 t0	-2.3272	3.0	0.6728
e1 t1	-2.2600	2.0	-0.2600
e1 t2	-3.0000	1.0	-2.0000
e1 t3	-0.8500	4.0	3.1500

$$\hat{A}_t = \frac{A_t - \text{mean}(A)}{\text{std}(A) + 10^{-8}} \quad (28)$$

ความหมาย: ทำให้ขนาดของ A อยู่ราว ๆ ± 1 เสมอไม่ว่า reward จะใหญ่หรือเล็ก → learning rate ตัวเดียวใช้ได้ทุกงาน · mean กับ std คำนวณจากทั้งตารางพร้อมกัน (8 ค่า; Tron1: 49,152 ค่า) ครั้งเดียวต่อ iteration · 10^{-8} กันหารศูนย์

$$\text{mean} = \frac{-0.2722 - 0.4475 - 0.0660 - 0.3000 - 2.3272 - 2.2600 - 3.0000 - 0.8500}{8} = -1.1904$$

$$\text{std} = \sqrt{\frac{\sum (A_i - \text{mean})^2}{8 - 1}} = 1.1513$$

กับดัก: torch.std() หารด้วย $n - 1$ (unbiased) ไม่ใช่ n — ถ้าคุณคำนวณด้วยมือ แล้วได้ 1.077 แปลว่าหารด้วย 8 ต้องหารด้วย 7 ถึงจะตรงกับโค้ด

ช่อง	A	$\hat{A} = (A + 1.1904)/1.1513$
e0 t0	-0.2722	+0.7975
e0 t1	-0.4475	+0.6452
e0 t2	-0.0660	+0.9766
e0 t3	-0.3000	+0.7733
e1 t0	-2.3272	-0.9874
e1 t1	-2.2600	-0.9290
e1 t2	-3.0000	-1.5718
e1 t3	-0.8500	+0.2956

อ่านผล: ทุก action ของ e0 กลายเป็น "ดีกว่าค่าเฉลี่ย" (บวก) และ e1 t0-t2 เป็น "แยกว่า" (ลบ) — แม้ A ดิบของ e0 จะติดลบทั้งหมด! เพราะ normalize วัดเทียบกับ เพื่อนในตารางเดียวกัน ไม่ใช่เทียบกับศูนย์ · เครื่องหมายของ e0 t0 (+) และ e1 t1 (-) ตรงกับตัวประมาณหยาบ δ ที่ใช้ในหัวข้อ 4.1 แต่ขนาดต่างกันมาก

โค้ดบรรทัดต่อบรรทัด — rollout_storage.py: 187-207

บรรทัด	โค้ด	สมการ
:187	def compute_returns(self, last_values, gamma, lam)	รับ V_{last} (จาก ppo.py:176-178), γ , λ
:189	for step in reversed(range(...))	เดินย้อนจากแถวสุดท้าย — (26) ต้องรู้ A_{t+1} ก่อน
:190-193	next_values = last_values ถ้าแถวสุดท้าย ไม่ขึ้น self.values[step+1]	$V(s_{t+1})$ ของ (19)
:194	next_is_not_terminal = 1.0 - self.dones[step].float()	mask ($1 - done_t$)
:195-199	delta = rewards + mask * gamma * next_values - values	(19)
:200	advantage = delta + mask * gamma * lam * advantage	(26) — ตัวแปร advantage คือ A_{t+1} ก่อนบรรทัดนี้และ A_t หลังบรรทัดนี้
:201	self.returns[step] = advantage + self.values[step]	(27)

บรรทัด	โค้ด	สมการ
:204	<code>self.advantages = self.returns - self.values</code>	ได้ A_t กลับมาจาก (27)
:205-207	<code>(advantages - mean) / (std + 1e-8)</code>	(28)

สรุปหัวข้อ — GAE (25) = พหุคูณ TD error ถ่วงด้วย $(\gamma\lambda)^k$ คำนวณตั้งแต่ TD ($\lambda = 0$) ถึง MC ($\lambda = 1$); โค้ดคำนวณด้วย recursion (26) ย้อนหลังพร้อม mask; $R_t = A_t + V_t$ (27) คือเป้า ของ critic; $\hat{A}_t$ (28) normalize ทั้งตารางแล้วส่งให้ loss (e0 t0: -0.2722 → +0.7975)

4.6 PPO — loss ทั้งก่อน และทำไมต้อง "clip"

ตอนนี้มีครบแล้ว: ทิศจาก (23), $\hat{A}_t$ ทุกช่องจาก (28), และ ρ_t จาก (21) ที่ทำให้ ใช้ตารางซ้ำได้ · หัวข้อนี้ประกอบ **loss L ที่สมการ (12) รออยู่ตั้งแต่ภาค 2** จากห้าส่วน (ก)-(จ) — แต่ก่อนอื่นต้องเข้าใจว่าทำไม policy gradient ธรรมดาถึงยังไม่พอ

Policy gradient ธรรมดา (REINFORCE) มีปัญหาสำคัญ: ถ้าอัปเดตน้ำหนักของ actor แรงเกินไป ในการอัปเดตครั้งเดียว policy ใหม่อาจ "กระโดด" ไปไกลจาก policy เดิมมากจนพฤติกรรมพังไปเลย (เช่น หุ่นยนต์ที่กำลังเริ่มเดินได้ จู่ ๆ ล้มทุกครั้ง) และเพราะ action ถัดไปถูกเลือกจาก policy ที่พังแล้ว ข้อมูลที่เก็บมาต่อจากนี้ก็ใช้ไม่ได้ต่อเนื่องกันไปอีก — วนเป็นปัญหาลูกโซ่ · ขำร้าย หัวข้อ 3.3 บอกว่า ρ แกว่งแรงเมื่อ policy ห่างจากตัวเก่า

Proximal Policy Optimization (PPO) ("proximal" = ใกล้เคียง) แก้ปัญหานี้ด้วยการ**จำกัด ขนาดของการเปลี่ยนแปลง policy ในแต่ละรอบอัปเดต** ไม่ให้ policy ใหม่ต่างจาก policy เดิม มากเกินไป มีสามกลไกที่ Tron1 ใช้ร่วมกัน: clip (ก), เบรก lr ด้วย KL (ข), โบนัส entropy (ค) — บวกกับ loss ของ critic (ง) รวมเป็น loss เดียว (จ)

(ก) Clipped surrogate objective

surrogate (ตัวแทน) คือฟังก์ชันที่คำนวณได้จากข้อมูลเก่า และ**มีความชันเท่ากับ** เป้าหมายจริง J ตรงจุดที่ยังไม่ขยับ: เอา $\hat{A}_t$ คูณ ρ_t — จาก (22) $\nabla \rho = \rho \nabla \log \pi$ และที่ $\theta = \theta_{old}$ ค่า $\rho = 1$ ดังนั้น $\nabla(\rho_t \hat{A}_t) = \hat{A}_t \nabla \log \pi$ ตรงกับ (23) เป๊ะ · ไกลจากจุดนั้นความชันเริ่ม ไม่ตรงและ ρ เริ่มแกว่ง — จึง **clip ρ** ไม่ให้หลุดช่วง $[1 - \epsilon, 1 + \epsilon]$:

รูป	สมการ	ทิศ
เปเปอร์ (Schulman et al. 2017)	$L^{CLIP} = \mathbb{E}[\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)]$	maximize
โค้ด (ppo.py: 256-260)	$L^{CLIP} = \text{mean}[\max(-\hat{A}_t \rho_t, -\hat{A}_t \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon))]$	minimize — ติดลบทั้งสองพจน์แล้ว min กลายเป็น max

$$L^{CLIP} = \text{mean}_{t \in \text{minibatch}} \left[\max(-\hat{A}_t \rho_t, -\hat{A}_t \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon)) \right] \tag{29}$$

ความหมาย: สองพจน์คือ "ผลักดันแรง" กับ "ผลักดันที่ ρ ถูกหนีไว้" — เลือกพจน์ที่ **แย่กว่า** (ค่า loss ใหญ่กว่า) เสมอ = มองโลกในแง่ร้าย · ผลคือพอ ρ หลุดกรอบใน ทิศที่กำลังได้กำไร พจน์ที่ถูกหนีเป็นค่าคงที่ (ไม่มี θ) ขณะ → gradient เป็นศูนย์ → หยุดผลัก · clip_param = 0.2 (limx_rsl_rl_ppo_cfg.py:96) คือ ϵ — ρ ขยับได้ไม่เกิน ±20% ต่อ rollout

สัญลักษณ์	ชนิด	ความหมาย	ที่
ρ_t	scalar ต่อช่อง	อัตราส่วนความหนาแน่น (21)	ratio ppo.py:252-254
$\hat{A}_t$	scalar ต่อช่อง	advantage ที่ normalize แล้ว (28) — ค่าคงที่ ระหว่าง update ไม่มี θ	advantages_batch
ϵ	ค่าคงที่	ขอบ clip 0.2 — ไม่ใช่ ϵ noise ของ (16)	clip_param cfg:96
$\text{clip}(x, a, b)$	ฟังก์ชัน	หนีบ x ให้อยู่ใน $[a, b]$	torch.clamp :257-259

ไล่ e0 t0 — $\log \pi_{old} = -0.3058$ ถูกเก็บตอน rollout (ppo.py: 147-149) · สมมติ gradient step ก่อนหน้าดัน μ จาก 0.20 → 0.25 (ทิศที่ (24) สั่ง เพราะ $\hat{A} > 0$ และ $a > \mu$):

ขั้นตอน	ค่า	ที่
$\log \pi_{new}(0.40 \mid s)$ ที่ $\mu = 0.25$: $-(0.40 - 0.25)^2/0.5 - 0.2258$	-0.2708	: 212-214
$\rho = e^{-0.2708+0.3058} = e^{0.035}$	1.0356 — อยู่ใน [0.8, 1.2]	: 252-254
$-\hat{A}\rho = -0.7975 \times 1.0356$ · $-\hat{A} \text{clip}(\rho) = -0.7975 \times 1.0356$	-0.8259 ทั้งคู่ — clip ยังไม่ทำงาน	: 256-259
$L_i = \max(-0.8259, -0.8259)$	-0.8259	: 260

ขั้นตอน	ค่า	ที่
$\partial L_i / \partial \mu = -\hat{A} \rho (a - \mu_{new}) / \sigma^2 = -0.7975 \times 1.0356 \times 0.15 / 0.25$	-0.4955 → SGD ดัน μ ขึ้นต่อ	จาก (24) ผ่าน ρ

clip ทำงานเมื่อไหร่ — คำนวณจากคู่แฝดในตาราง

ในตัวอย่างข้างบน clip ไม่ทำงานเลยเพราะ μ ขยับแค่ $0.05 \cdot$ ตาราง 2×4 จึงไม่มี คู่แฝด ไว้ให้ทดสอบ: **e0 t2** กับ **e1 t1** มี $a - \mu = 0.40$ เท่ากัน จึงมี $\log \pi_{old} = -0.5458$ เท่ากัน และเมื่อ μ ขยับเท่ากันก็ได ρ เท่ากันเป๊ะ — ต่างกันแค่เครื่องหมายของ $\hat{A}$ (+0.9766 กับ -0.9290) · เท่านั้นก็เห็น clip ครบทั้งสี่กรณีจากตัวเลขในตารางเดียวกัน

กฎการขยับ: สมมติว่าผ่านไปหลาย gradient step แล้ว μ เดินตามทิศทางที่ (24) สั่งจนสะสมได้

$\mu_{new} = \mu_{old} + \text{sign}(\hat{A}) \cdot \text{sign}(a - \mu_{old}) \cdot 0.15$ — เข้าหา a ถ้า action ดี หนีจาก a ถ้า action แย่ · ทั้งสองช่องมี $a > \mu_{old}$ "ถูกทาง" ของ e0 t2 จึงเป็น +0.15 ส่วนของ e1 t1 คือ -0.15 · แล้วลองกลับทิศทางดูด้วยว่าถ้า "เดินผิดทาง" โกลเท่ากัน clip จะทำอะไร

ช่อง	$\hat{A}$	ขยับ	μ_{new}	ρ	ในช่วง?	$-\rho \hat{A}$	$-\text{clip}(\rho) \hat{A}$	$L = \max$	$\partial L / \partial \mu$
e0 t2	+0.9766	ถูกทาง +0.15	0.25	1.2153	$X > 1.2$	-1.1869	-1.1719	-1.1719 (คงที่)	0
e1 t1	-0.9290	ถูกทาง -0.15	-0.15	0.7520	$X < 0.8$	+0.6986	+0.7432	+0.7432 (คงที่)	0
e0 t2	+0.9766	ผิดทาง -0.15	-0.05	0.7520	$X < 0.8$	-0.7344	-0.7813	-0.7344	-1.6157 โหล
e1 t1	-0.9290	ผิดทาง +0.15	0.15	1.2153	$X > 1.2$	+1.1291	+1.1148	+1.1291	+1.1291 โหล

อ่านสองแถวบน: พอ ρ หลุดกรอบ $[0.8, 1.2]$ ใน ทิศที่ถูก **max** เลือกพจน์ที่ถูก clip ซึ่งเป็นค่าคงที่ → อนุพันธ์ = 0 → **หยุดหลัก** · ระบบพูดว่า "เข้าหา/หนี action นี้นา 20% แล้ว พอ รอข้อมูลใหม่"

อ่านสองแถวล่าง: ρ หลุดกรอบเท่ากันเป๊ะ แต่ใน ทิศที่ผิด — **max** กลับเลือกพจน์ดิบ gradient โหลเต็ม และเครื่องหมายของมันดึง μ กลับ · clip ไม่เคยขวางการแก้ผิด

$\hat{A}$	$\rho > 1 + \epsilon$ (ขอบมากขึ้น)	$\rho < 1 - \epsilon$ (ขอบน้อยลง)
+ (action ดี)	clip · gradient 0 — "ดีพอแล้ว หยุด"	ไม่ clip · gradient ดึงกลับ — "ทำไหมหนี action ดี"
- (action แย่)	ไม่ clip · gradient ผลักออก — "ทำไหมชอบ action แย่"	clip · gradient 0 — "หนีพอแล้ว หยุด"

ปลายการเรน σ เล็กจน clip จะทำงานถี่ขึ้นมาก — จาก (14) สองครั้งได้ $\rho = \exp\left(\frac{(a-\mu_{old})^2 - (a-\mu_{new})^2}{2\sigma^2}\right)$ · ถ้า $\sigma = 0.25$ การขยับ +0.15 เดียวกันบน e0 t2 ($a - \mu_{old} = 0.40, a - \mu_{new} = 0.25$) ให้ $\rho = \exp\left(\frac{0.40^2 - 0.25^2}{2 \cdot 0.25^2}\right) = \exp(0.78) = 2.18$ นะถูกรอบไป โกล · นี่คือการหมายเชิงปฏิบัติของ trade-off เรื่องจำนวน epoch ในหัวข้อ 4.7

ทำไมต้องเก็บ $\log \pi_{old}$ ไว้ตั้งแต่ตอน rollout — เพราะต้องมี "ค่าอ้างอิงจากอดีต" ไว้เทียบว่า policy เคลื่อนไปเท่าไรแล้ว ถ้าไม่เก็บ ก็วัด ρ ไม่ได้ · และนี่ก็เป็นเหตุผล เดียวกับที่ตารางใบเดิมถูกใช้ซ้ำได้ 5 epoch (หัวข้อ 4.7) — ทุก epoch π_{new} เคลื่อนออก จาก π_{old} ไปอีก clip คือสิ่งที่คอยจำกัดไม่ให้เคลื่อนไกลเกินจนข้อมูลเก่าใช้ไม่ได้ (หัวข้อ 3.3)

(ข) KL-adaptive learning rate — เบนกัตโนมิติ

clip ดูรายชื่อ — ยังไม่มีอะไรบอกว่า policy ทั้งใบ ขยับไปไกลแค่ไหนโดยรวม · ตัววัดนั้นคือ **KL divergence** (Kullback–Leibler; อ่านว่า "เค-แอล") ซึ่งเป็น

สัญลักษณ์	ชนิด	ความหมาย
$D_{KL}(p \parallel q)$	ฟังก์ชัน (การแจกแจง, การแจกแจง) → scalar ≥ 0	"ระยะ" จากระวัง p ไปหาระวัง q — 0 ก็ต่อเมื่อเหมือนกันทุกจุด · ไม่สมมาตร สลับลำดับได้คนละค่า · นิยามทั่วไป $\mathbb{E}_{a \sim p}[\log p(a) - \log q(a)]$
μ_o, σ_o	เวกเตอร์ (8,) ต่อช่อง	ระวังเก่า ที่ใช้เก็บข้อมูล — ได้เก็บ μ, σ ลง buffer ไว้ด้วย (rollout_storage.py:145-146)
μ_n, σ_n	เวกเตอร์ (8,) ต่อช่อง	ระวังใหม่ ที่ ปัจจุบันให้ (ppo.py:217-218)

สำหรับ Gaussian สองใบ นิยามทั่วไปข้างบนคำนวณออกมาเป็นรูปปิด (สูตรมาตรฐาน — derive ด้วย การแทน (14) ลงในนิยามแล้วใช้ $\mathbb{E}[(a - \mu_o)^2] = \sigma_o^2$):

$$D_{KL}(\mathcal{N}(\mu_o, \sigma_o) \parallel \mathcal{N}(\mu_n, \sigma_n)) = \ln \frac{\sigma_n}{\sigma_o} + \frac{\sigma_o^2 + (\mu_o - \mu_n)^2}{2\sigma_n^2} - \frac{1}{2}$$

(30)

ความหมาย: พจน์กลางคือ "จุดกลางขยับไปที่เท่าของความกว้าง (ยกกำลังสอง)" — ตัวหลัก · พจน์ $\log$ และ $-\frac{1}{2}$ วัดความกว้างที่เปลี่ยน (ตัดกันเป็นศูนย์เมื่อ σ เท่ากัน) · ตรงกับ ppo.py:223-232 ทุกพจน์: $\log(\text{sigma}/\text{old_sigma} + 1e-5) + (\text{old_sigma}^2 + (\text{old_mu} - \text{mu})^2) / (2 \text{ sigma}^2) - 0.5$ แล้ว $\text{sum}(\text{axis}=-1)$ รวม 8 ข้อต่อ (+1e-5 กันหารศูนย์) · $\text{kl_mean} (:233)$ เฉลี่ยทั้ง minibatch — **ตัวนี้**ที่กฎข้างล่างใช้ (ส่วน $\text{mean_kl} :295$ แค่อัปเดต)

กฎปรับ lr (ppo.py:236-244, เปิดด้วย `schedule = "adaptive"` `cfg:101`, เป้า `desired_kl = 0.01` `cfg:104`) — เช็ก**ทุก gradient step** ก่อนคำนวณ loss:

เงื่อนไข	ทำอะไร	ขีดจำกัด
$KL > 2 \times 0.01 = 0.02$ (policy รวดเร็วเกิน)	<code>lr /= 1.5</code>	พื้น 10^{-5}
$KL < 0.01/2 = 0.005$ และ > 0 (ช้าเกิน)	<code>lr *= 1.5</code>	เพดาน 10^{-2}
ระหว่างนั้น	คงเดิม	

ในตาราง 2×4 (σ เท่ากัน สองพจน์ข้างตัดกัน เหลือ $(\Delta\mu)^2/(2\sigma^2)$): ขยับ 0.05 ทุกช่อง → $KL = 0.0025/0.5 = \mathbf{0.0050}$ → ไม่ถึง 0.02 และไม่ต่ำกว่า 0.005 → lr คงเดิม · ขยับ ± 0.15 ตามกฎคู่แฝด → $KL = 0.0225/0.5 = \mathbf{0.045} > 0.02$ → $lr \div 1.5$

สองข้อที่มักเข้าใจผิด. (1) KL **ไม่อยู่ใน loss** — มันปรับแค่ความยาวก้าว α ของ สมการ (12) ไม่ได้ผลลัพท์ · (2) KL วัดระยะ**สะสม**จากะวังที่เก็บข้อมูล (μ_o จาก buffer) ไม่ใช่ระยะของก้าวล่าสุด — gradient step แรกของทุก iteration จึงได้ $KL \approx 0$ เสมอ (ะวังยังไม่ ขยับ) กฎจึง**เพิ่ม** $lr \times 1.5$ ที่จุดนั้นทุกครั้ง แล้ว step ต่อ ๆ ไปค่อยดึงลงถ้าสะสมเกิน 0.02 · พฤติกรรมลดการเกรนที่เกิดจากสองข้อนี้อยู่ใน 06 หัวข้อ 3.4 · clip กับ KL ไม่เข้ากัน: clip ดูรายช่องและขึ้นกับ $a - \mu$ ส่วน KL ดูค่าเฉลี่ยทั้ง minibatch และขึ้นกับ $\Delta\mu/\sigma$ อย่างเดียว

(ค) Entropy bonus — กันไม่ให้หยุดสำรวจเร็วเกินไป

ปัญหาสุดท้ายคือ policy อาจ "มันใจเกินไป" เร็วเกินไป — หด σ จนเลือก action เดิมซ้ำ ๆ ก่อนที่จะสำรวจพอ (explore/exploit จากหัวข้อ 2.4) · ตัววัด "ยังสับสนอยู่แค่ไหน" คือ **entropy** (เอนโทรปี)

สัญลักษณ์	ชนิด	ความหมาย
$H(p)$	ฟังก์ชัน (การแจกแจง) → scalar	ความไม่แน่นอนของการแจกแจง · นิยาม $H = -\mathbb{E}_{a \sim p}[\log p(a)]$ = ค่าเฉลี่ยของ "ความประหลาดใจ" — $\log p$

สำหรับ Gaussian แทน (14) ลงในนิยามแล้วใช้ $\mathbb{E}[(a - \mu)^2] = \sigma^2$:

ขั้นตอน	เหตุผล
$H = -\mathbb{E}\left[-\frac{1}{2} \ln 2\pi - \ln \sigma - \frac{(a-\mu)^2}{2\sigma^2}\right]$	แทน (14)
$= \frac{1}{2} \ln 2\pi + \ln \sigma + \frac{\mathbb{E}[(a - \mu)^2]}{2\sigma^2}$	กระจายลบ ค่าคงที่ออกจาก $\mathbb{E}$
$= \frac{1}{2} \ln 2\pi + \ln \sigma + \frac{\sigma^2}{2\sigma^2}$	ค่าเฉลี่ยของระยะห่างกำลังสองคือความแปรปรวน

$$H(\mathcal{N}(\mu, \sigma)) = \frac{1}{2} + \frac{1}{2} \ln 2\pi + \ln \sigma \quad (\text{ต่อข้อ ต่อ; รวม 8 ข้อ ต่อดี วยการบวก})$$

(31)

ความหมาย: ขึ้นกับ σ อย่างเดียว ไม่เกี่ยวกับ μ — ยิ่งระวังกว้าง ยิ่งไม่แน่นอน · ที่ $\sigma = 0.5$: $H = 0.5 + 0.9189 - 0.6931 = \mathbf{0.7258}$ · ที่ $\sigma = 1.0$ (เริ่มเกรน): 1.4189 · ที่ $\sigma = 0.1$: -0.8836 — **ติดลบได้** เพราะเป็น entropy ของความหนาแน่น (ค่าที่เกิน 1 ได้ตามหัวข้อ 2.3) ไม่ใช่ของ ความน่าจะเป็น · ได้ด: `distribution.entropy().sum(-1)` (actor_critic.py:151-153, สูตรใน `torch/distributions/normal.py:104-105`) แล้วเฉลี่ย minibatch (ppo.py:273)

PPO ลบ $c_H H$ ออกจาก loss (= บวกโบนัสดู objective) ด้วย `entropy_coef = 0.01` (`cfg:97`) — เล็กน้อย เพราะเป็นแค่ตัวช่วยไม่ใช่เป้าหมายหลัก · **นี่คือกลไกที่ดัน σ ไม่ให้หดจนเหลือศูนย์เร็วเกินไป** — กราฟ Mean action noise std ที่ค่อย ๆ ลดลง (หัวข้อ 2.4) คือผลของการชกเยื่อระหว่าง L^{CLIP} (อยากให้ σ เล็กเมื่อ action ที่ต่ออยู่ใกล้ μ) กับ entropy bonus (ดัน ℓ ขึ้นคงที่ 0.01 ต่อข้อต่อทุก step เพราะ $\partial H/\partial \ell = 1$ จาก (31)) — อนุพันธ์ทั้งสองข้างของการชกเยื่อนี้ derive ไว้ใน 06 หัวข้อ 3.4

(ง) Value loss — สอน critic

critic ต้องเรียนด้วย ไม่จั้น $\hat{A}$ ในรอบถัดไปจะไม่แม่นยำ · เป้าคือ R_t จาก (27) และ critic ค่ายค่าใหม่ V_{new} จาก ϕ ที่เพิ่งขยับ (ppo.py:216):

$$L_V = \text{mean} \left[\max \left((V_{new} - R_t)^2, (V_{clip} - R_t)^2 \right), \quad V_{clip} = V_{old} + \text{clip}(V_{new} - V_{old}, -\epsilon, \epsilon) \right]$$

(32)

ความหมาย: พื้นฐานคือความคลาดกำลังสอง (mean squared error) ระหว่างค่าทำนายกับเฉลย — การเรียนแบบ supervised ที่เฉลยมีตัวเองปนอยู่ (bootstrap) · ส่วน **max** กับ V_{clip} (เปิดด้วย `use_clipped_value_loss = True` `cfg:95`, โค้ด :263-269) ทำให้ critic ถูก "ลงโทษแบบระวัง": ถ้ากระโดดไกลกว่า ϵ จาก V_{old} ก็ยังโดนคิดตามที่กระโดดจริง ไม่ได้ผ่อนลง · ถ้าปีดจะ เหลือ MSE รรรมดา (: 271)

ตัวอย่างช่อง e0 t0 ($V_{old} = 5.0, R = 4.7278$):

V_{new}	$(V_{new} - R)^2$	V_{clip}	$(V_{clip} - R)^2$	L_V ช่องนี้ = max
4.9	0.0297	$5.0 + \text{clip}(-0.1) = 4.9$	0.0297	0.0297
4.5	0.0519	$5.0 + \text{clip}(-0.5) = 4.8$	0.0052	0.0519 (เลือกตัวใหญ่)

gradient $\partial L_V / \partial V = 2(V - R) \rightarrow$ ดัน V เข้าหา R · ตัวเลขนี้ไหลไปที่ ϕ ของ critic เท่านั้น (`.detach()` ที่ :144 และ :177 กับ V ที่อยู่ใน $\hat{A}$, R ไม่ใช่เป็น ทางผ่านของ gradient)

(จ) Loss รวม — สิ่งที่สมการ (12) รออยู่

$$L = L^{CLIP} + c_V L_V - c_H H, \quad c_V = 1.0, c_H = 0.01$$

(33)

ความหมาย: เข็มวัดความเผล่ตัวเดียวทั้งระบบ (scalar) — สามพจน์ดึงดูดคนละชุด: L^{CLIP} ดึง θ (actor + ℓ), L_V ดึง ϕ (critic), H ดึง ℓ · ค่าคงที่มาจาก `value_loss_coef = 1.0` `cfg:94`, `entropy_coef = 0.01` `cfg:97` (คำปรัยายของคลาส `ppo.py:54` คือ 0.0 — ถูก `cfg` กับ) · โค้ด `ppo.py:274-278` · KL ไม่อยู่ในนี้

จากนั้นคือสี่คำสั่งของหัวข้อ 2.2 (`ppo.py:287-290`): `zero_grad` $\rightarrow$ `backward` (autograd คำนวณ ∇L ของลูกบิดทั้ง 469,905 ตัวใน optimizer หลัก = actor + critic + ℓ) $\rightarrow$ `clip_grad_norm(1.0)` $\rightarrow$ `step()` — **จบหนึ่ง gradient step** · ในตาราง 2×4 คือ 1 ใน 4 ต่อ iteration (Tron1: 1 ใน 20)

สรุปเป็นภาพเดียว

PPO = policy gradient (23) เขียนใหม่ด้วย ρ (21) ให้ใช้ตารางซ้ำได้ + advantage จาก GAE (28) + clip (29) กันพลังโกลเกิน + KL (30) เบส lr + entropy (31) กันหยุดสำรวจ + value loss (32) สอน critic $\rightarrow$ loss เดียว (33) $\rightarrow$ สมการ (12)

— ทุกชิ้นส่วนที่พูดมาทั้งหมดในโค้ดนี้ ประกอบกันเป็นบรรทัดใน `agents/limx_rsl_rl_ppo_cfg.py`

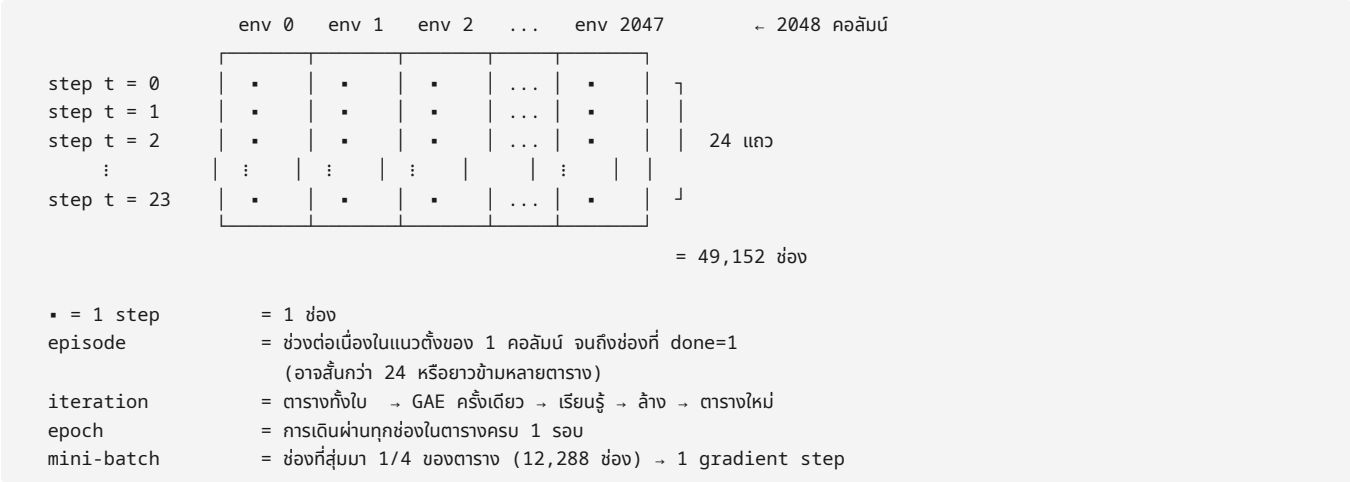
สรุปหัวข้อ — loss ของ PPO (33) มีสามพจน์: surrogate ที่ clip แล้ว (29) สำหรับ actor, value loss (32) สำหรับ critic, entropy (31) สำหรับ σ ; clip หยุด gradient เฉพาะทิศที่ถูกละโกลพอ; KL (30) ไม่อยู่ใน loss แต่ปรับ lr; ทั้งหมดถูกทำที่ระดับ minibatch

4.7 หน่วยของฝั่งอัลกอริทึม: iteration · rollout · epoch · mini-batch

หัวข้อ 1.4 แนะนำ **step** กับ **episode** ไปแล้ว — สองอันนั้นเป็นหน่วยของ *ปัญหา* ตรงนี้คือหน่วยของ *การเรียนรู้* ซึ่งเพิ่งจะอธิบายได้ตอนนี้ เพราะต้อง รู้จัก GAE กับ clip ก่อน

ห้าหน่วยนับคือห้าวิธีหั่นตารางเดียวกัน

ข้อมูลของหนึ่ง iteration คือตารางสองมิติ แถว = step ($T = 24$) คอลัมน์ = env ($B = 2048$) แต่ละช่อง = หนึ่ง transition — ตาราง 2×4 ของโค้ดนี้คือมันฉบับย่อ:



หน่วย	คืออะไรในตาราง	คำนวณอะไร "ที่ระดับนี้"	ใน toy	ใน Tron1
step	1 ช่อง	forward pass: $\mu, a, \log \pi, V \cdot \text{env: } r, \text{done}$	8 ช่อง	49,152 ช่อง
episode	ช่วงแนวตั้งใน 1 คอลัมน์	ไม่มีสูตร — แค่ reset env และจดความยาว · แต่ done ตัดสาย GAE (26)	e1 มี 2 episode	~1000 step ≈ 42 ตาราง
iteration	ตารางทั้งใบ = rollout	GAE + normalize ครั้งเดียว ทุกช่องพร้อมกัน แล้ว epoch × minibatch gradient step แล้ว KL → lr	1 ตาราง 2×4	1 ตาราง 24×2048
epoch	เดินครบทุกช่อง	หันเป็น minibatch (สืบลำดับครั้งเดียวต่อ iteration rollout_storage.py:230)	2 epoch × 2 mb	5 epoch × 4 mb
mini-batch	ช่องสุ่ม $1/M$ ของ ตาราง	$\rho, L^{CLIP}, L_V, H \rightarrow$ 1 gradient step	4 ช่อง	12,288 ช่อง

ไม้ดชุดนี้เรียกตารางทั้งใบว่า **rollout** (49,152 transition) — ไม่ใช่คำว่า "batch" กับหน่วยนี้ เพราะคำนี้นั้นชนกับ mini-batch

สองสิ่งที่คนมักเข้าใจผิด:

- episode ไม่ใช่หน่วยของอัลกอริทึม** — PPO ไม่เคยรอ episode จบ ไม่เคยคำนวณอะไร "ต่อ episode" มันเป็นหน่วยของ *environment* (เมื่อไหร่ที่ต้อง reset) และของ *การวัดผล* (Mean episode length on_policy_runner.py:327) เท่านั้น สิ่งเดียวที่ episode มีผลต่อ คณิตศาสตร์คือ done ที่ตัดสาย bootstrap ใน (19) และ (26)
- gradient step เกิดที่ระดับ mini-batch ไม่ใช่ iteration** — 1 iteration มี $5 \times 4 = 20$ gradient step; GAE ต่างหากที่เกิดครั้งเดียวต่อ iteration

สามหน่วยเทียบกัน

iteration คือ **หนึ่งรอบเต็มของวงจร "ออกไปเก็บประสบการณ์ แล้วกลับมาเรียนรู้จากมัน"**

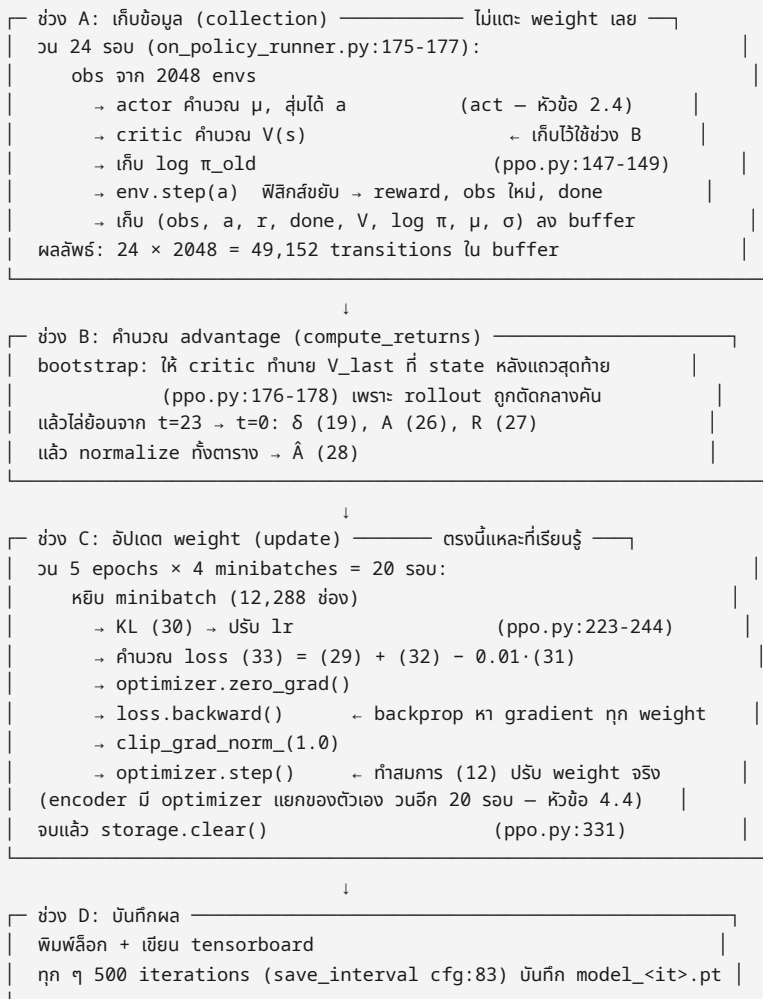
	step	episode	iteration
คือหน่วยของอะไร	การตัดสินใจ 1 ครั้ง	การเล่น 1 รอบ ของหุ่น 1 ตัว	1 รอบของ เก็บข้อมูล → เรียนรู้
ขนาดใน Tron1	0.02 วินาที	สูงสุด 1000 step	24 step × 2048 env
จบเมื่อไหร่	ทุก 0.02 s	ล้ม หรือครบ 20 วินาที	เก็บครบ 24 step ต่อ env แล้วอัปเดตเสร็จ
เกี่ยวกับหุ่นกี่ตัว	1 ตัว	1 ตัว	ทั้ง 2048 ตัวพร้อมกัน
weight เปลี่ยนไหม	ไม่	ไม่ (แต่เปลี่ยนระหว่างทาง!)	เปลี่ยน 20 ครั้ง (ดูกายวิภาคด้านล่าง)
เห็นในล็อกยังไ้	ไม่แสดง	Mean episode length	Learning iteration 137/15000 (on_policy_runner.py:314)

จุดสำคัญคือ **rollout ไม่ใช่ episode** และ **หนึ่ง rollout ไม่ต้องรอ episode จบ** — Isaac Lab รัน 2048 หุ่นยนต์พร้อมกันแบบขนาน แต่ละตัวเดินไปแค่ 24 step แล้วหยุด เก็บข้อมูล ไม่ว่า episode ของมันจะจบหรือยัง

ทำไมไม่รอ episode จบก่อนค่อยอัปเดต? เพราะ episode ยาวถึง 1000 step ถ้ารอจบก่อนทุกครั้งจะช้ามาก — **นี่คือปัญหาแบบ Monte Carlo ในหัวข้อ 3.2 นั่นเอง** การตัดเป็น rollout สั้น ๆ ทำให้อัปเดตบ่อยขึ้นมาก episode ที่ยังไม่จบ ตอนตัด rollout จะถูก "ตัดกลางคัน" แล้ว critic ช่วยทำนายส่วนที่เหลือแทนด้วย V_{last} — bootstrap ที่ขอบตารางในหัวข้อ 4.5

กายวิภาคของ 1 iteration — ข้างในเกิดอะไรขึ้นบ้าง

หนึ่ง iteration ไม่ใช่การอัปเดตครั้งเดียว แต่แบ่งเป็น **4 ช่วงที่ทำงานกัน** (ไล่ตามโค้ดใน [on_policy_runner.py:170-240](#) และ [ppo.py:180-331](#)):



จุดที่มักเข้าใจผิด: 1 iteration ≠ 1 gradient update

`num_learning_epochs = 5 (cfg:98) × num_mini_batches = 4 (cfg:99) = 20 ครั้ง` ที่ `optimizer.step()` ถูกเรียกต่อหนึ่ง iteration — ข้อมูลชุดเดียวกัน (49,152 transition) ถูกใช้ซ้ำ 5 รอบ โดยแต่ละรอบหันเป็น 4 ก้อนย่อย

ทำไมใช้ข้อมูลซ้ำได้ 5 รอบ ทั้งที่ PPO เป็น on-policy? — คำตอบเดิมอยู่หัวข้อ 3.3: importance sampling (ρ) ทำให้ข้อมูลของ policy เก่ายังใช้ได้ **ตรงใดที่ policy ใหม่ ยังอยู่ใกล้** และ clip (29) กับ KL (30) คือสิ่งที่บังคับให้ยังอยู่ใกล้ · พอ epoch ที่ 2, 3, 4, 5 policy เปลี่ยนไปจากตอนเก็บข้อมูลมากขึ้นเรื่อย ๆ **ถ้าไม่มี clip จะใช้ซ้ำได้แค่ 1 epoch เท่านั้น** — การใช้ซ้ำได้ 5 รอบคือสิ่งที่ทำให้ PPO ประหยัดข้อมูลกว่า policy gradient รุ่นก่อนหน้ามาก

เวลาที่ใช้ในแต่ละช่วง — `train.py` พิมพ์ให้ดูทุก iteration ในรูปแบบ ([on_policy_runner.py:320](#)):

```
Computation: 81920 steps/s (collection: 0.420s, learning 0.180s)
└─ ช่วง A ─┬─ ช่วง B+C ─┘
```

และบันทึกลง tensorboard แยกเป็น `Perf/collection time` กับ `Perf/learning_time` · ตัวเลข steps/s คือ

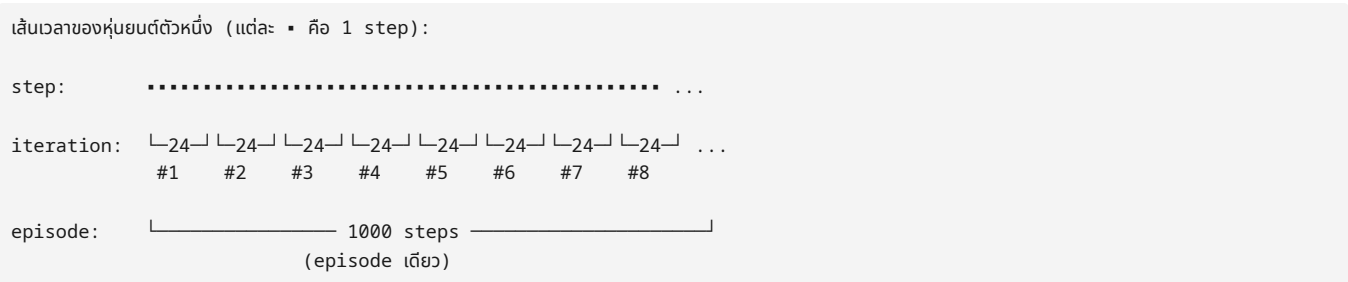
$49,152 / (0.420 + 0.180) = 81,920$ ([on_policy_runner.py:270-274](#))

ปกติช่วง A จะกินเวลามากกว่า เพราะต้องรับฟลักส์จริง ๆ — นี่คือเหตุผลที่การเพิ่ม `--num_envs` ช่วยให้เรียนรู้เร็วขึ้นต่อ iteration (ได้ข้อมูลเยอะขึ้นโดยเวลาช่วง A เพิ่มขึ้นมานิด เพราะ GPU ทำงานน้อยแล้ว) จนกว่าจะชน VRAM

สิ่งที่ควรจดตอนรัน exp_001. `exp_001_baseline_sf.md` มีช่อง "steps/s reported at iteration 100" รออยู่ — ตัวเลข steps/s ในบรรทัดนี้แหละคือค่าที่ ต้องกรอก และการดูสัดส่วน collection ต่อ learning จะบอกได้ว่าคอบวดยอยู่ที่ ฟลักส์ (เพิ่ม envs ไม่ช่วยแล้ว) หรือที่การอัปเดต network

episode กับ iteration ไม่ซ้อนกัน

step ซ้อนอยู่ใน episode และซ้อนอยู่ใน iteration ด้วย แต่สองอันหลัง เป็นการ "หั่น" ระยะเวลาเดียวกันคนละแบบ ที่ตัดกันไปมา:



หนึ่ง episode กินเวลาประมาณ $1000 \div 24 \approx 42$ iterations — แปลว่า ระหว่างที่หุ่นยนต์ตัวหนึ่งเดินอยู่ใน episode เดียว weight ของมันถูกอัปเดตไป แล้วราว $42 \times 20 = 840$ gradient step หุ่นตัวนั้นจึง "เปลี่ยนสมอง" หลายร้อยรอบระหว่างที่ ยังเดินรอบเดียว — ฟังดูแปลกแต่ไม่มีปัญหา เพราะแต่ละครั้งเปลี่ยนนิดเดียว (มี clip (29) กับ KL (30) คุ่มอยู่) · ในตาราง 2×4 : env 1 จบ episode ที่ t2 แล้วเริ่มใหม่ที่ t3 ภายในตาราง เดียว ส่วน env 0 ยังไม่จบ — สองแบบที่เจอในโลกจริงอยู่ครบล

นี่คือคำตอบเชิงตัวเลขของเรื่อง "เห็นหุ่นค่อย ๆ ตั้งได้ระหว่างเทรน" ในหัวข้อ 2.4 — weight ถูกอัปเดตทุก ~0.5 วินาทีของเวลาจำลอง ไม่ใช่ทุก episode

ภาพรวมทั้งการเทรน:

	จำนวน
1 iteration	$24 \times 2048 = 49,152$ transition
15,000 iteration (max_iterations cfg:82)	737,280,000 transition (~740M)
หุ่น 1 ตัวได้ประสบการณ์	$15,000 \times 24 = 360,000$ step = 2 ชั่วโมง ของเวลาจำลอง
รวมทั้ง 2048 ตัว	2 ขม. $\times 2048 \approx 170$ วัน ของประสบการณ์การเดิน

ตัวเลข ~740M ตรงกับที่ exp_001_baseline_sf.md ประเมินไว้พอดี — และแถวสุดท้าย คือเหตุผลทั้งหมดที่ต้องเทรนในซิม: เก็บประสบการณ์ 170 วัน ภายในไม่กี่ชั่วโมง โดยไม่มีหุ่นยนต์จริงพังสักตัว

เลข 2048 มาจากไหน. ค่าปริยายใน cfg คือ num_envs = 4096 (limx_base_env_cfg.py:484) — 2048 คือค่าที่ exp_001 สั่งกับผ่าน CLI -- num_envs 2048 (../README.md:265, ../experiments/exp_001_baseline_sf.md:28) เพื่อให้พอดี VRAM 12 GB ของเครื่องนี้

สรุปว่าแต่ละคำตอบคำถามอะไร:

- **step** = หน่วยเวลาที่เล็กที่สุด (policy ตัดสินใจ 1 ครั้ง)
- **episode** = หน่วยของ "การลองหนึ่งรอบ" → ตอบว่า *หุ่นเก่งแค่ไหน* (Mean episode length ยิ่งยาวยิ่งดี)
- **iteration** = หน่วยของ "การเรียนรู้หนึ่งครั้ง" → ตอบว่า *ฝึกไปแล้วเท่าไร* (Learning iteration 137/15000)

ทำไม 1 iteration ถึงเก็บ 24 step — ไม่ใช่ 4 หรือ 200

ค่านี้มาจาก num_steps_per_env = 24 ใน limx_rsl_rl_ppo_cfg.py:81 — เป็น hyperparameter เหมือน 512/256/128 และ scale=0.25 คือไม่มีสูตร คำนวณออกมาตรง ๆ แต่มี ข้อจำกัดจริงมันจากทั้งสองด้าน:

ถ้าน้อยเกินไป (เช่น 4)	ถ้ามากเกินไป (เช่น 200)
GAE มองอนาคตได้สั้นเกินไป → advantage ฟัง critic เกือบทั้งหมด → bias สูง	ข้อมูลช่วงต้น rollout "เก่า" — เก็บด้วย policy เวอร์ชันก่อน แต่เอาไปอัปเดต policy ปัจจุบัน (on-policy — หัวข้อ 3.3)
อัปเดตทีละทีละครั้งข้อมูลน้อย → gradient แหว่ง	เปลืองแรม — buffer โตตามตรง
ค่าใช้จ่ายคงที่ต่อการอัปเดต (backprop, optimizer) มากเกินไป	feedback ช้า — จำนวนการอัปเดตต่อเวลาลดลง

เหตุผลเชิงปริมาณที่ดีที่สุด: ต้องยาวพอคลุม "ระยะมองเห็น" ของ GAE

GAE (25) ถ่วงน้ำหนัก TD error ที่ไกลออกไปด้วย $(\gamma\lambda)^k$ ซึ่งลดลงแบบเอ็กซ์โพเนนเชียล · ผลรวมของน้ำหนักทั้งหมด $1 + \gamma\lambda + (\gamma\lambda)^2 + \dots$ คืออนุกรมเรขาคณิต และค่าของ มันคือ "จำนวนก้าวที่ GAE สนใจจริง ๆ" (effective horizon):

$$\text{horizon} = \frac{1}{1 - \gamma\lambda} \quad (34)$$

ความหมาย: toy: $1/(1 - 0.72) \approx 3.6$ ถ้า — ตาราง 4 แถวคลุมพอดิ · Tron1: $\gamma\lambda = 0.99 \times 0.95 = 0.9405 \rightarrow 1/0.0595 \approx 17$ ถ้า — ถ้าเก็บ rollout สั้นกว่านั้น เท่ากับตัดข้อมูลที่ GAE ยังต้องใช้ทั้ง advantage จะพึ่ง bootstrap จาก critic มากเกินไป (bias สูง ตามตารางใน หัวข้อ 3.2) · $24 > 17 \rightarrow$ มีระยะเพื่อพอดิ ที่ 24 step น้ำหนักเหลือ $0.9405^{24} \approx 0.23$ คือลดเหลือ ~23% แล้ว ส่วนที่เก็บจากนั้นให้ critic ทำนายต่อ ด้วย V_{last} (ใน toy: $0.72^4 \approx 0.27$ — สัดส่วนใกล้เคียงกันโดยตั้งใจ)

ข้อจำกัดเรื่อง VRAM — เกี่ยวกับเรื่องนี้โดยตรง

rollout buffer ต้องเก็บทุก transition ไว้ในหน่วยความจำ GPU:

```
ต่อ 1 transition: policy obs 36 + critic obs 230 (รวม commands แล้ว) + obsHistory 360 + commands 3
                  + next_observations 36 (สำเนา obs ถัดไป) + action 8 + reward + value
                  + return + advantage + logprob +  $\mu$  8 +  $\sigma$  8 + done (1 byte)
                  = 694 floats  $\times$  4 bytes + 1  $\approx$  2.7 KB      (rollout_storage.py:71-120)
```

num_steps_per_env	ขนาด buffer (ที่ 2048 envs)
24 (ปัจจุบัน)	~137 MB
100	~570 MB
200	~1.1 GB

README บันทึกว่าการ์ด 12 GB ของเครื่องนี้อยู่ต่ำกว่าที่ Isaac Lab แนะนำ (16 GB) อยู่แล้ว — การเพิ่ม num_steps_per_env กับ VRAM เพิ่มแบบเป็นเส้นตรง

ขนาด minibatch ที่ได้ออกมาก็ต้องพอ

```
49,152 transitions ÷ num_mini_batches 4 = 12,288 ต่อ minibatch
× num_learning_epochs 5 → ข้อมูลแต่ละชิ้นถูกใช้ 5 รอบ
```

12,288 ตัวอย่างต่อ gradient step ถือว่าเยอะมากสำหรับงานแบบนี้ gradient จึงนิ่ง **จำนวน env ที่มาก (2048) ขดเซยความสั้นของ rollout (24) ได้** — ถ้าใช้แค่ 16 envs ตอน smoke test จะได้แค่ 384 transitions ต่อ iteration ซึ่งน้อยเกินไปจะเรียนรู้จริง (นั่นคือเหตุผลที่ smoke test ใน README ตั้ง --max_iterations 2 แค่นี้ก็ว่า รันผ่าน ไม่ได้หวังผลการเรียนรู้)

สรุป: 24 ไม่ได้มาจากการคำนวณ แต่มาจากการอยู่ในช่วงที่เงื่อนไขทุกข้อพอใจ พร้อมกัน:

- ยาวกว่าระยะมองเห็นของ GAE (~17 step จาก (34))
- สั้นพอที่ข้อมูลยังไม่เก่าเกินไปสำหรับ on-policy (หัวข้อ 3.3)
- buffer ไม่เกิน VRAM
- คู่กับ 2048 env แล้วได้ minibatch ใหญ่พอให้ gradient นิ่ง

ค่าในช่วง 16-32 เป็นค่ามาตรฐานที่ใช้กันทั่วไปในงาน legged locomotion บน Isaac Lab / legged_gym ไม่ใช่ตัวเลขที่ LimX คิดขึ้นเอง

ถ้าจะทดลองเปลี่ยน: แก้ที่ agents/limx_rsl_rl_ppo_cfg.py ตามตารางใน 04-file-structure-vs-ros2.md — แต่ถ้าเปลี่ยน num_steps_per_env ควรพิจารณา lam ควบคู่ไปด้วย เพราะทั้งคู่กำหนดระยะมองเห็นร่วมกันตาม (34)

สรุปหัวข้อ — ถ้าหน่วยคือหน่วยขั้นตอนตารางเดียว (step ช่อง · episode คอลัมน์จนถึง done · iteration ทั้งใน · epoch เดียวกัน · minibatch หั่น 1/4); 1 iteration = 4 ช่วง A-D และ 20 gradient step; 24 มาจากขอบเขต 4 ข้อ โดยข้อสำคัญคือ horizon (34) ≈ 17

ภาค 5 — สรุปและเชื่อมกลับเข้ากับ config จริง

5.1 ตาราง hyperparameter — แต่ละค่ามาจากหัวข้อไหน

ทุกแถวอ้าง block ของ SF (limx_rsl_rl_ppo_cfg.py:80-114) เว้นที่ระบุไฟล์อื่น · ค่าใน toy ของ ตาราง 2 × 4 อยู่ในคอลัมน์สุดท้าย

Hyperparameter	ที่	ค่าที่ใช้	คืออะไร	อธิบายที่หัวข้อ	toy
num_steps_per_env	:81	24	ความยาว rollout T ต่อ env ต่อ iteration	4.7 (สมการ 34)	4
max_iterations	:82	15000	ฝึกกี่ iteration	4.7	1
save_interval	:83	500	บันทึก checkpoint ทุกกี่ iteration	4.7 ช่วง D	—
init_noise_std	:87	1.0	σ เริ่มต้น — ค่านี้ไม่ถูกใช้ (actor_critic.py:117 ถูกคอมเมนต์; :118 ใช้ zeros $\rightarrow \sigma = e^0 = 1.0$ ตรงกันโดยบังเอิญ)	2.1	0.5 คงที่
actor_hidden_dims · critic_hidden_dims	:88-89	[512, 256, 128]	ขนาดชั้นกลางของ actor/critic	2.2	—
activation	:90	"elu"	ฟังก์ชันกระตุ้น	2.2 (สมการ 11)	—
value_loss_coef	:94	1.0	c_V น้ำหนักของ value loss ใน loss รวม	4.6 (สมการ 33)	1.0
use_clipped_value_loss	:95	True	ใช้ max กับ V_{clip} ใน value loss	4.6 (สมการ 32)	ใช้
clip_param	:96	0.2	ϵ ใน clipped objective	4.6 (สมการ 29)	0.2
entropy_coef	:97	0.01	c_H น้ำหนัก entropy bonus — กันไม่ให้ σ หดเร็วเกินไป	4.6 (สมการ 31, 33)	0.01
num_learning_epochs	:98	5	ใช้ข้อมูลชุดเดิมซ้ำกี่รอบ	4.7	2
num_mini_batches	:99	4	หั่น rollout เป็นกี่ก้อนย่อยต่อ epoch	4.7	2
learning_rate	:100	1e-3	α เริ่มต้น — ถ้ายาวแค่ไหนต่อ gradient step	2.2 (สมการ 12)	0.1
schedule	:101	"adaptive"	เปิดกฎปรับ lr ด้วย KL	4.6 (ข)	เปิด
gamma	:102	0.99	discount γ — อนาคตสำคัญเท่าปัจจุบันแค่ไหน	1.5 (สมการ 2), 1.6 (สมการ 3)	0.9
lam	:103	0.95	λ ใน GAE — จุดกึ่งกลาง bias/variance	4.5 (สมการ 25)	0.8
desired_kl	:104	0.01	เป้าหมายของ KL-adaptive learning rate	4.6 (สมการ 30)	0.01
max_grad_norm	:105	1.0	เพดานความยาวของเวกเตอร์ gradient ก่อน <code>step()</code>	2.2	—
obs_history_len	:106	10	กี่ step ใน obsHistory h (360 = 10 × 36)	4.4	—

Hyperparameter	ที่	ค่าที่ใช้	คืออะไร	อธิบายที่หัวข้อ	toy
output_detach · num_output_dim	:109-110	True · 3	ตัด gradient ของ encoder ออกจาก loss หลัก · ขนาด $\hat{\mathbf{z}}$	4.4	—
num_envs	cfg/SF/limx_base_env_cfg.py:484	4096 (CLI กับ เป็น 2048)	จำนวนคอลัมน์ B ของ rollout	4.7	2
scale (action)	limx_base_env_cfg.py:119	0.25	ตัวแปลง action ไร้หน่วย → เรเดียน	2.5 (สมการ 17)	—
stiffness / damping	solefoot_cfg.py:50-51, :61-62	45.0 / 1.5, 0.8	k_p, k_d ของ PD controller	2.5 (สมการ 18)	—

Remark — คำปரியายของคลาส PPO ไม่ใช่ค่าที่รับ. ppo.py:48-64 มี default ของตัวเอง (gamma = 0.998, entropy_coef = 0.0, schedule = "fixed", num_learning_epochs = 1) ซึ่งถูกค่าใน cfg กับทั้งหมดตอนสร้างอ็อบเจกต์ — เวลาอ่านโค้ดให้ดูที่ cfg ไม่ใช่ที่ signature

5.2 ดัชนีสมการทั้งหมด

เลขสมการเป็น **เลขเดี่ยวทั้งชุด** ใช้ร่วมกับโน้ต 06, 07 และอภิธานศัพท์ (00-glossary.md หัวข้อ 9) · โน้ตนี้ครอบคลุม (1)–(34) · โน้ต 06 ต่อด้วย (35)–(37) และโน้ต 07 มี (G)

สมการ	ชื่อ	หัวข้อ
(1)	Markov property	1.3
(2)	Objective — ใจถ่ยทั้งหมดของ RL	1.5
(3)	นิยาม value function V^π	1.6
(4)	สมการ Bellman	1.6
(5)	นิยาม $Q^\pi(s, a)$	1.6
(6)	$Q = \mathbb{E}[r + \gamma V(s')]$	1.6
(7)	$V(s) = \mathbb{E}[G_t \mid s]$	1.7
(8)	Advantage $A = Q - V$	1.7
(9)	$\pi_\theta(\cdot \mid s) = \mathcal{N}(\mu_\theta(s), \sigma), \sigma = e^\ell$	2.1
(10)	Node output ของ neural network	2.2
(11)	ELU	2.2
(12)	Gradient descent update	2.2
(13)	Gaussian PDF	2.3
(14)	$\log \pi(a \mid s)$ สามพจน์	2.3
(15)	รวม 8 ข้อต่อ	2.3
(16)	การสุ่ม $a = \mu + \sigma \odot \varepsilon$	2.4
(17)	$q_{\text{target}} = 0.25 a + q_{\text{default}}$	2.5
(18)	PD controller	2.5
(19)	TD error δ_t (มี mask)	3.2
(20)	Importance sampling	3.3
(21)	Ratio ρ_t	3.3
(22)	Score function $\nabla \pi = \pi \nabla \log \pi$	4.1
(23)	Policy gradient	4.1
(24)	$\partial \log \pi / \partial \mu$	4.1
(25)	GAE พาราม	4.5
(26)	GAE recursion (มี mask)	4.5
(27)	$R_t = A_t + V_t$	4.5
(28)	Normalize $\hat{A}_t$	4.5

สมการ	ชื่อ	หัวข้อ
(29)	PPO clipped surrogate	4.6 (ก)
(30)	KL ของเกาส์เซียน	4.6 (ข)
(31)	Entropy ของเกาส์เซียน	4.6 (ค)
(32)	Value loss	4.6 (ง)
(33)	Loss รวม	4.6 (จ)
(34)	Horizon $1/(1 - \gamma\lambda)$	4.7

ตารางแปลงเลขเก่า → ใหม่ (สำหรับผู้ที่ยังจำเลขเดิมของโน้ตนี้ได้):

เดิม	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)
ใหม่	(2)	(3)	(4)	(7)	(10)	(12)	(13)	(16)	(18)	(8)	(25)	(29)

5.3 สิ่งที่ยังไม่ได้พูดถึง

- **Domain randomization** — ไม่ใช่ส่วนหนึ่งของ PPO เอง แต่เป็นเทคนิคแยก ต่างหากที่ทำให้ policy ที่ฝึกในซิมูเลเตอร์ใช้งานได้ดีบนหุ่นยนต์จริง (sim-to-real) — เห็นได้ใน EventsCfg ตามที่ 03-sf-task-anatomy.md บันทึกไว้ พารามิเตอร์ที่ถูกสุ่มเหล่านี้คือสิ่งที่ critic เห็นใน 227 มิติ (หัวข้อ 2.1)
- **Gait command** — gait_phase 2 มิติ กับ gait_command 4 มิติ ใน observation มาจาก mdp/commands/gait_command.py ซึ่งเป็น term ที่ LimX เขียน เองเอง จังหวะการเดินของ Tron1 จึงเป็นสิ่งที่ถูกสั่ง ไม่ใช่สิ่งที่ เกิดขึ้นเองจากการเรียนรู้
- **รายละเอียดของ 20 reward terms** — อยู่ใน 03-sf-task-anatomy.md พร้อม น้ำหนักครบทุกตัว โน้ตนี้อ้างถึงแต่ไม่ได้แจกแจงซ้ำ
- **กลไกระดับโค้ดของการปรับ weight** — l กับ .grad เป็นคนละช่อง, อนุพันธ์ $\partial L / \partial l$ (35), Adam (36) ที่ปรับก้าวต่อลูกบิต, และพหุติกรรมของกฎ lr (37) ตลอดการเทรน — โน้ต 06 หัวข้อ 3.4
- **การไล่ทั้ง iteration ด้วยตัวเลข** — ทุกสมการของโน้ตนี้ถูกแทนค่าครบทั้ง 8 ช่อง รวมทั้ง mini-batch ที่สอง — โน้ต 07

5.4 เส้นทางอ่านต่อ

อยากรู้อะไร	ไปที่ไหน
นิยามของทุกคำ สัญลักษณ์ ค่าคงที่ และเฉลี่ยของตาราง 2 × 4	00-glossary.md
สายส่งค่าจาก obs ถึง loss ที่ละบรรทัดโค้ด พร้อมเวกเตอร์ 8 มิติจริง	06-training-walkthrough-bridge.md
คำนวณหนึ่ง iteration ด้วยมือทุกระดับ	07-compute-at-every-level.md
ตัวเลข config ของ SF task ควบ ๆ (obs/action/reward/randomisation)	03-sf-task-anatomy.md
จะแก้อะไรต้องไปแก้ไฟล์ไหน	04-file-structure-vs-ros2.md
repo ไหนทำอะไรในระบบนิเวศ TRON1	01-repo-map.md
ทำไม deploy ต้องผ่าน SDK ไม่ใช่เขียน mj_data.ctrl ตรง ๆ	02-sdk-vs-sim.md
จะรับ baseline ยังไง ต้องจดอะไรบ้าง	experiments/exp_001_baseline_sf.md
อ่าน PPO paper ดันฉบับ	Schulman et al. 2017, arXiv:1707.06347

แหล่งอ้างอิงแนวคิด: Sutton & Barto, "Reinforcement Learning: An Introduction" (ตอนที่ 3–4 สำหรับ MDP/Bellman, §5.5 สำหรับ importance sampling, §13.2–13.4 สำหรับ policy gradient / REINFORCE / baseline); Schulman et al. 2017, "Proximal Policy Optimization Algorithms"; Schulman et al. 2016, "High-Dimensional Continuous Control Using Generalized Advantage Estimation". ตัวเลข config และเลขบรรทัดโค้ดทั้งหมดตรงกับ repo จริงใน tron1-rl-isaacsim/ และ IsaacSim/ ของเครื่องนี้ — สิ่งที่ได้เป็น relative path จึงใช้ได้เฉพาะในเครื่องที่ clone ทั้งสอง repo ไว้แล้ว (ทั้งคู่อ้างใน .gitignore ตาม README) · ตัวเลขของตาราง 2 × 4 ทั้งหมดคำนวณด้วย scripts/toy_answer_key.py.